

CURSO INTEGRADOR II · OCTUBRE 2026 · EDICIÓN AMPLIADA



Cómo funciona la plataforma de monitoreo fitosanitario

Arquitectura, flujo completo, tecnologías y herramientas, app móvil, plataforma web, API, base de datos, backend, inteligencia artificial, seguridad, despliegue y pendientes. Explicado desde cero y, después de cada idea, con su explicación técnica.

Agrícola Riachuelo S.A.C. · Ica, Perú

monitoreo.agricolariachuelo.org



Estado al 08/10/2026 · plataforma web v1.3 · app móvil v0.5.0 · modelo de IA 2026.10-int

Cómo leer este documento

Cada tema se explica dos veces: primero en sencillo y luego «por dentro». Estos son los elementos que vas a ver.

**Explicación sencilla**

Las tarjetas, pasos y tablas cuentan la idea con palabras simples y ejemplos de la vida diaria. Si solo lees esto, ya entiendes el sistema.

**Explicación técnica**

El panel gris de abajo, con la barra de color, explica lo mismo «por dentro»: archivos, tablas, comandos, valores y reglas reales del código.

**Herramienta usada y por qué**

Cuando se usa una herramienta, se dice **para qué** sirve y **por qué** se eligió frente a otras opciones.

**Un color por bloque**

Cada bloque tiene su color (barra izquierda y etiqueta de arriba). En los casos, el color indica el **estado de revisión**, nunca la gravedad.

**Dato clave:** un dato que conviene recordar.

**Para entenderlo:** una comparación con algo cotidiano.

**Importante:** una regla que no se debe romper.

**Pendiente:** algo que todavía falta hacer.

**Nuevo:** lo agregado en la web v1.3 y la app v0.5.0 (08/10/2026).

**Estados de un caso (colores de la web y la app)**

 Pendiente de revisión

 Confirmado por IA

 Confirmado por especialista

 Posible plaga

 Evidencia insuficiente

 Descartado

**EXPLICACIÓN TÉCNICA**

Los datos de este documento salen del código de los dos repositorios (`agricola-riachuelo-platform` , rama `prueba2` , y `agricola-riachuelo-mobile` , rama `fase4-sincronizacion`), de sus decisiones de arquitectura (ADR-W-001 a ADR-W-007 y ADR 0001 a 0009), de los maestros de la web y de la app y del informe del curso. Los nombres en `código` son nombres reales de archivos, tablas, variables o estados.

**HERRAMIENTA USADA Y POR QUÉ**

**Este documento** — **Para qué:** explicar el sistema a todo el equipo.
Por qué: se genera desde el mismo material del proyecto, así no se desactualiza a mano.

1



El problema y la idea

Qué resuelve el sistema, en qué consiste, quién lo usa y cómo se va a medir si funciona.

¿Qué problema resolvemos?

Revisar la salud de miles de plantas de vid a simple vista es lento, depende de la memoria y deja poco registro.

 **Hoy**

El personal recorre las hileras y anota lo que ve. Es difícil saber **dónde exactamente** estaba el problema y comparar con semanas anteriores.

 **La propuesta**

Una mochila con **3 celulares** toma **fotos de cada hilera**, por sus dos lados, con su lote, hilera, lateral, segmento, marcador y posición GPS.

 **La ayuda de la IA**

Un modelo de visión (YOLO) **sugiere** en qué fotos hay indicios de **chanchito blanco** o de **melaza y fumagina**, y marca dónde están.

 **La decisión humana**

Un **especialista** revisa la foto en la web y decide. Si confirma, se avisa por WhatsApp al jefe de fundo y la alerta aparece en la app para **ir al lugar**.

 **Dato clave:** la IA no diagnostica: **sugiere indicios** y ordena el trabajo del especialista para que revise primero lo más sospechoso. Desde la v1.3 puede confirmar sola **solo si** el administrador activa un umbral de confianza para el modelo (hoy está apagado).

 **EXPLICACIÓN TÉCNICA**

La plaga objetivo es el **chanchito blanco** (*Planococcus ficus*, cochinilla harinosa de la vid): se alimenta de la planta, deja **melaza** donde crece la **fumagina** (hongo negro que tapa la hoja) y transmite el virus del enrollamiento de la hoja (GLRaV-3). Cada foto queda unida a su contexto (sesión, pasada, lateral, secuencia, lote, hilera, segmento, marcador y GPS) y a un identificador único `captureId` : por eso un hallazgo se puede volver a encontrar y seguir en el tiempo.

 **HERRAMIENTA USADA Y POR QUÉ**

 **Celulares Android comunes** — **Para qué:** capturar en campo con cámara, GPS y Wi-Fi. **Por qué:** son baratos y conocidos; no hace falta hardware especial ni drones.

Cómo se trabaja hoy en el fundo

El registro actual es una hoja de Excel: cuenta plantas con chanchito por lote, pero no guarda foto, hora ni ubicación exacta.

Lote	Plantas evaluadas	Plantas con chanchito	Incidencia
SWG 1	9 987	97	0,97 %
SWG 2	4 843	331	6,83 %
SWG 3	6 017	80	1,33 %
SWG 4	13 398	604	4,51 %
SWG 5	4 836	587	12,14 %
LOTE 6	1 656	0	0 %
LOTE 7	1 645	0	0 %
LOTE 8	10 521	30	0,29 %
TOTAL	52 903	1 729	3,27 %

👥 Mucha gente, poco registro

Hoy recorren **entre 10 y 20 personas**. El resultado es un número por lote: no queda la foto, la hora ni la planta exacta.

📍 Difícil volver al lugar

Si una planta tenía chanchito, no hay forma segura de volver a ella ni de saber si mejoró la semana siguiente.

🕒 Comparar depende de la memoria

Las comparaciones entre semanas o lotes se hacen a mano y sin evidencia que se pueda revisar después.

🔧 EXPLICACIÓN TÉCNICA

Fuente: archivo `MAPEO CHANCHITO MOLINOS actualizado 1.xlsx`, hoja `RESUMEN` (una hoja más por lote y un plano). Incidencia = plantas con chanchito ÷ plantas evaluadas. El piloto trabaja en **SWG1, SWG2 y SWG5** (83 hileras). SWG5 tiene la mayor incidencia (12,14 %): por eso sus hileras H31–H36 (y H01, H02 y H07 de SWG2) son las elegidas para tomar fotos con plaga y entrenar la IA.

El sistema en una frase

«Del campo a la decisión, y de vuelta al campo»: fotos ordenadas por ubicación, sugeridas por IA, validadas por una persona y fáciles de volver a encontrar.

**App móvil**

Captura las fotos en el campo, incluso **sin internet**, y luego las envía. Desde la v0.5.0 también muestra en un **mapa satelital** dónde están las plagas y guía hasta ellas.

**Plataforma web**

Recibe las fotos, las guarda, las analiza con IA y permite **revisarlas y decidir** en un navegador. Tiene panel, bandeja, mapa, plano, reportes y administración.

**Servicios en la nube**

Base de datos (**Supabase**), fotos (**Cloudinary**), avisos (**WhatsApp**), imágenes satelitales (**Esri**) y rutas a pie (**Google Maps**).

**Para entenderlo:** piensa en un hospital: la app es la ambulancia que trae los casos (y ahora también lleva al médico hasta el paciente), la web es la sala de diagnóstico y la nube es el archivo de historias clínicas.

**EXPLICACIÓN TÉCNICA**

Son **dos programas propios** que solo se hablan por una API (`/api/v1` , con HTTPS y tokens JWT): la app en React Native + Expo y la plataforma en Django. La plataforma corre en un VPS con Docker detrás de Traefik y es la **única** que entra a la base de datos. Las fotos viajan directo del celular a Cloudinary con un permiso firmado por Django, así el servidor no carga con los megabytes de cada foto.

**HERRAMIENTA USADA Y POR QUÉ**

 **Un solo backend (Django)** — **Para qué:** web, API y procesamiento de IA en un mismo código. **Por qué:** una sola fuente de reglas; antes se pensaban 4 proyectos separados.

El ciclo completo, de vuelta al campo

Con «Ubicar plaga» el sistema cierra el círculo: lo que se detecta en la foto termina con una persona parada frente a la planta.



 **EXPLICACIÓN TÉCNICA**

Pasos 1 y 2 ocurren en el celular controlador (base SQLite y cola de sincronización); el 3 en el proceso `worker_ia`; el 4 en la web (`revision/services.py`) o en el worker si el modelo tiene umbral de autoconfirmación; el 5 en el mismo worker (cola `notifications`); y el 6 en la pantalla `app/pests` , que consulta `GET /api/v1/mobile/pest-reports` . Ningún paso espera al siguiente: cada uno deja su resultado guardado y el siguiente lo toma cuando puede.

 **HERRAMIENTA USADA Y POR QUÉ**

 **Google Maps por URL** — **Para qué:** guiar a pie hasta la planta. **Por qué:** no necesita clave ni pago; abre la app de mapas que el celular ya tiene.

Objetivo, hipótesis e indicadores

Lo que el proyecto del curso quiere demostrar: 3 dimensiones y 9 indicadores.

🎯 Objetivo general

Determinar en qué medida la implementación de un sistema tecnológico de monitoreo fitosanitario **mejora el desempeño** del proceso de monitoreo de plagas en los cultivos de vid de Agrícola Riachuelo S.A.C., Ica, 2026.

🧪 Hipótesis general

La implementación del sistema **mejora significativamente** el desempeño del proceso de monitoreo de plagas. Hay una hipótesis específica por cada dimensión de abajo.

🕒 Eficiencia operativa

- Tiempo total de monitoreo (min u h)
- Personas-hora empleadas
- Plantas o segmentos inspeccionados por hora

✅ Efectividad del monitoreo

- Cobertura alcanzada (%)
- Evidencia utilizable (%)
- Registros correctamente documentados (%)

🔍 Trazabilidad y seguimiento

- Hallazgos correctamente localizables (%)
- Tiempo para localizar un hallazgo (min)
- Casos con seguimiento completo (%)

🔧 EXPLICACIÓN TÉCNICA

Cada indicador sale de datos que el sistema ya guarda: duración de sesiones y pasadas (`monitoring_sessions` , `monitoring_passes`), cobertura por hilera (ambos laterales cerrados, RN-13), calidad de cada foto (`quality_results` : UTILIZABLE o repetir), ubicación de cada caso (`location_source` : GPS, MARCADOR o NINGUNA), tiempo hasta la decisión (panel de la web) e historial completo (`audit_events`). «Ubicar plaga» apunta directo al indicador **tiempo para localizar un hallazgo**.

Cómo se va a medir: pretest y posttest

Diseño preexperimental de un solo grupo ($O_1 - X - O_2$): medir antes, aplicar el sistema y medir después en los mismos lugares.



Muestra

12 segmentos de 25 plantas consecutivas (300 plantas) en los lotes del piloto, elegidos de forma intencional (acceso y operación).

Análisis

12 pares antes/después. Shapiro-Wilk para ver si las diferencias son normales; luego **t de Student** o **Wilcoxon**.

Control

Se anotan fecha, luz, cultivo, personas, conectividad e incidencias; el pretest no guía el recorrido del posttest.

EXPLICACIÓN TÉCNICA En el sistema, un segmento es exactamente eso: la tabla <code>field_segments</code> con <code>start_plant</code>, <code>end_plant</code> y la marca <code>is_pilot</code>. Hoy cada hilera del piloto tiene un segmento «Hxx completa» (<code>-S1</code>); para el posttest se crean los de 25 plantas con «Dividir en segmentos» (Administración → Catálogos), sin borrar nada: lo anterior se desactiva.	HERRAMIENTA USADA Y POR QUÉ Excel / hoja de registro — Para qué: anotar el pretest manual (tiempos y hallazgos). Por qué: es lo que el equipo ya usa; el posttest sale de los datos del sistema.
--	--

¿Quién usa qué?

Cada persona tiene un rol; el rol decide qué pantallas y acciones ve.

Rol	Dónde trabaja	Qué hace
Operador de campo	App móvil	Lleva la mochila, inicia la sesión de monitoreo, recorre las hileras y sincroniza. También puede abrir «Ubicar plaga».
Especialista fitosanitario	Web + app nuevo	Revisa las fotos sugeridas y decide: confirmar, posible plaga, descartar o evidencia insuficiente. En la app solo ve alertas en el mapa.
Supervisor	Web	Ve el panel, el mapa, el plano y los reportes; sigue el avance; gestiona catálogos y dibuja lotes e hileras en el mapa.
Administrador	Web + app	Crea, aprueba y edita cuentas, revoca celulares, catálogos, destinatarios, cola de IA y auditoría. Puede usar la app.
Jefe de fundo / supervisor de zona	WhatsApp	Recibe el aviso cuando el especialista confirma un caso (o la IA, si el umbral está activo) y abre el enlace al caso.

🔧 EXPLICACIÓN TÉCNICA

Los roles se guardan en la tabla `user_roles` (una cuenta puede tener varios). Roles fijos: `ADMINISTRADOR` , `OPERADOR_CAMPO` , `ESPECIALISTA_FITOSANITARIO` y `SUPERVISOR` . Entran a la app `MOBILE_ROLES` (operador, administrador y, desde v1.3, especialista); el trabajo de campo exige `MOBILE_FIELD_ROLES` (operador y administrador). El jefe de fundo no tiene cuenta: es un **destinatario** (`notification_recipients`) con teléfono, lotes y consentimiento.

Roles y permisos, uno por uno

Lo que cada rol puede hacer en la web. El servidor lo comprueba siempre, no solo la pantalla.

Permiso	Qué permite	A	E	S
dashboard.ver	Panel del fundo	✓	✓	✓
bandeja.ver	Bandeja de casos	✓	✓	✓
caso.ver	Ver un caso	✓	✓	✓
caso.decidir	Decidir un caso	—	✓	—
caso.corregir	Corregir una decisión	—	✓	—
caso.abrir_manual	Abrir caso desde una foto	—	✓	—
mapa.ver	Mapa del fundo	✓	✓	✓
sesiones.ver	Sesiones de monitoreo	✓	✓	✓
reportes.exportar	Reportes y CSV	✓	✓	✓

Permiso	Qué permite	A	E	S
notificaciones.ver	Avisos enviados	✓	✓	✓
ia.ver	Estado de la IA	✓	✓	—
ia.reencolar	Reencolar una tarea	✓	—	—
usuarios.gestionar	Cuentas (crear, aprobar, editar)	✓	—	—
dispositivos.gestionar	Celulares (revocar)	✓	—	—
destinatarios.gestionar	Destinatarios WhatsApp	✓	—	—
catalogos.gestionar	Catálogos y editor del mapa	✓	—	✓
auditoria.ver	Auditoría	✓	—	—

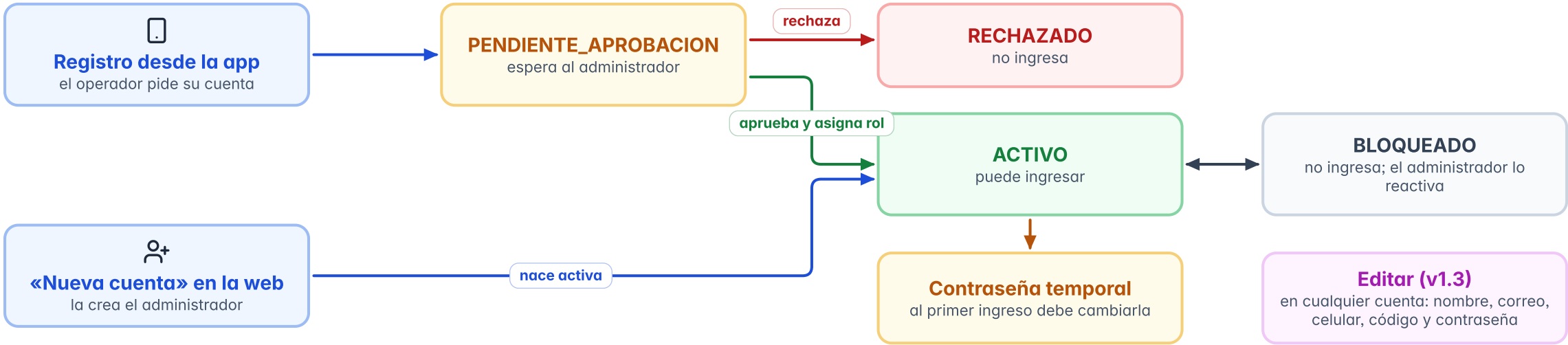
 **Dato clave:** **A** = administrador, **E** = especialista fitosanitario, **S** = supervisor. Solo el especialista decide y corrige casos. El operador de campo no entra a la web.

 **EXPLICACIÓN TÉCNICA**

La matriz vive en `web/permissions.py` (única fuente de verdad). Cada vista lleva `@web_view("permiso")` (regla W-07) y una prueba automática falla si alguna ruta no lo tiene. Además, los servicios vuelven a comprobar las reglas críticas (RN-W01): aunque alguien fabrique una petición, solo un especialista ACTIVO decide. Sin permiso se muestra la página 403 propia, y ocultar un botón nunca es la única protección.

Cuentas: cómo nacen y qué estados tienen

Una cuenta pasa por estados. El administrador es quien aprueba, bloquea y corrige.



⚠ Importante: el **operador** va siempre solo (cuenta exclusiva de la app); los roles de la web se pueden combinar. Nadie cambia el estado de su propia cuenta ni se quita su rol de administrador (RN-W14). No hay registro público en la web.

🕒 Nuevo: solo el **administrador** edita los datos de ingreso de una cuenta (botón «Editar»). En `/gestion/` esos campos quedaron en solo lectura.

 **EXPLICACIÓN TÉCNICA**

Servicios en `cuentas/services.py`: `create_account`, `update_account` y `set_password_by_admin` (v1.3). Todo se audita (`CUENTA_CREADA`, `CUENTA_APROBADA`, `CUENTA_ESTADO`, `ROLES_CAMBIADOS`, `CUENTA_EDITADA`, `CONTRASENA_ASIGNADA`) y la contraseña jamás llega a la auditoría. Cambiar el correo o la contraseña cierra la sesión de la app (el refresh entra a la lista negra).

2



Conceptos básicos

Las ideas que hay que tener claras antes de ver la arquitectura. Cada una con un ejemplo de la vida diaria y, abajo, su versión técnica.

Frontend, backend y base de datos

Casi todo sistema se divide en tres capas que cumplen funciones distintas.

 **Frontend (lo que se ve)**

Pantallas, botones y formularios. En nuestro caso: la **web** en el navegador y la **app** en el celular.

 **Backend (el que piensa)**

Programa que corre en un servidor: valida, aplica reglas y decide qué guardar. Aquí es **Django**, con su proceso de IA (worker).

 **Base de datos (la memoria)**

Guarda todo ordenado en tablas: usuarios, sesiones, fotos, decisiones. Aquí es **PostgreSQL** en Supabase (y SQLite en el celular).



 **EXPLICACIÓN TÉCNICA**

En la web, el HTML se arma **en el servidor** con plantillas de Django y el navegador solo lo muestra; HTMX reemplaza pedazos de la página sin recargarla. En la app, el frontend es React Native: dibuja pantallas nativas y guarda primero en SQLite. Los dos frontends llegan al mismo backend: la web con **cookie de sesión** y la app con **token JWT**. Solo Django habla con PostgreSQL.

 **HERRAMIENTA USADA Y POR QUÉ**

 **Plantillas Django + HTMX** — **Para qué:** armar la web en el servidor.
Por qué: sin frameworks pesados de JavaScript ni npm (regla W-01): más simple de mantener.

¿Qué es una API?

Una API es un «menú» de pedidos que un programa acepta. La app no entra a la base de datos: le pide cosas al backend por la API.

📍 La app pide

```
# la app pide
GET https://monitoreo.agricolariachuelo.org
/api/v1/health
```

Cada pedido tiene una **dirección** (URL) y un **verbo**: GET para leer, POST para enviar datos.

>

⬅ El backend responde

```
# el backend responde (JSON)
{ "status": "UP",
  "serverTime": "2026-10-05T22:02:24Z" }
```

La respuesta llega en **JSON**: texto con pares «nombre: valor» que cualquier lenguaje entiende.

Pedido real de nuestra API	Verbo	Para qué
/api/v1/mobile/bootstrap	GET	bajar lotes, hileras, segmentos y marcadores para trabajar sin internet
/api/v1/sessions	POST	enviar una sesión de monitoreo (se puede repetir sin duplicar)
/api/v1/captures/{id}/upload-ticket	POST	pedir permiso firmado para subir una foto
/api/v1/mobile/pest-reports v1.3	GET	traer las alertas de plaga para el mapa de la app

📄 **Dato clave:** nuestra API se llama /api/v1 . El «v1» es la versión: si algún día cambia de forma incompatible se crea /api/v2 sin romper la app ya instalada en los celulares.

🔧 **EXPLICACIÓN TÉCNICA**

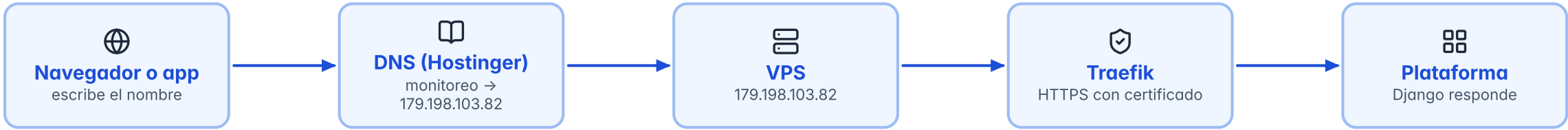
Una API REST usa los verbos de HTTP sobre direcciones. Cada respuesta trae un **código de estado**: 200/201 bien, 202 aceptado, 400 datos inválidos, 401 sin token o vencido, 403 sin permiso, 404 no existe, 409 conflicto, 413 archivo muy grande, 429 demasiados intentos, 5xx error del servidor. La app se identifica con `Authorization: Bearer <token>` y `X-Device-Id` (el celular).

🔗 **HERRAMIENTA USADA Y POR QUÉ**

`{}` **JSON** — **Para qué:** intercambiar datos entre la app y el servidor.
Por qué: es texto simple; TypeScript y Python lo leen sin librerías extra.

Dominio, DNS y HTTPS

Así encuentra el navegador nuestro servidor y por qué la conexión es segura.



@ **Dominio**

El nombre fácil de recordar:
monitoreo.agricola-riachuelo.org. Lo compramos en Hostinger.

📖 **DNS**

La «guía telefónica» de internet: traduce el nombre a la dirección numérica del servidor (**179.198.103.82**).

🔒 **HTTPS**

Conexión cifrada: nadie en el camino puede leer contraseñas ni fotos. El candado del navegador lo confirma.

🛡️ **Certificado**

Documento digital que prueba que el sitio es quien dice ser. Lo emite gratis Let's Encrypt y se renueva solo.

🔧 **Para entenderlo:** el dominio es el nombre de una tienda, el DNS es el directorio que dice en qué calle está, y HTTPS es el sobre cerrado en el que viajan tus cartas.

🔧 **EXPLICACIÓN TÉCNICA**

El dominio `agricolariachuelo.org` está registrado en Hostinger (activo hasta el 05/10/2027). El registro **A** `monitoreo -> 179.198.103.82` tiene TTL de 300 s (un cambio se propaga en minutos). Traefik pide el certificado a Let's Encrypt con el reto HTTP-01, lo renueva solo y redirige el puerto 80 al 443. Django añade **HSTS**: el navegador recuerda que este sitio solo se abre con HTTPS.

🔧 **HERRAMIENTA USADA Y POR QUÉ**

🔒 **Let's Encrypt** — **Para qué:** certificado HTTPS. **Por qué:** gratis y automático; Traefik lo renueva sin que nadie intervenga.

Servidor, VPS y contenedores

El backend necesita una computadora encendida las 24 horas, con internet y una dirección fija.

**VPS**

«Servidor privado virtual»: una computadora alquilada en un centro de datos (Hostinger). La administramos por comandos.

**Docker y contenedores**

Un contenedor es una caja cerrada con el programa y todo lo que necesita. Funciona igual en cualquier servidor.

**Proxy (Traefik)**

El «portero» del servidor: recibe lo que llega por internet, pone el HTTPS y lo envía al contenedor correcto.

**Aislamiento**

Nuestros contenedores tienen límites de memoria y permisos mínimos: no afectan a otros servicios del VPS.

 **Para entenderlo:** el VPS es un edificio; cada contenedor es un departamento con su propia llave; Traefik es el portero que recibe las visitas y las lleva al departamento correcto.

**EXPLICACIÓN TÉCNICA**

Una **imagen** de Docker es la receta (código + Python + librerías); un **contenedor** es esa receta funcionando. De la misma imagen `riachuelo-plataforma:latest` salen dos contenedores, `riachuelo-web-1` y `riachuelo-worker-1`, que se levantan juntos con Docker Compose (`deploy/docker-compose.yml`). El VPS tiene 48 GB de disco y se comparte con otro servicio que la plataforma no modifica.

**HERRAMIENTA USADA Y POR QUÉ**

 **Docker + Docker Compose** — **Para qué:** empaquetar y levantar web y worker. **Por qué:** un comando instala, actualiza o retira todo; los límites protegen al VPS compartido.



Git y GitHub

La forma en que el equipo guarda, comparte y publica el código: siempre PC → GitHub → VPS.

 **Git**
Guarda el historial del código: cada cambio (**commit**) queda registrado con autor, fecha y descripción. Se puede volver atrás.

 **GitHub**
El lugar en internet donde viven los repositorios del equipo. Es el punto de encuentro entre la PC de cada uno y el servidor.

 **push**
Subir tus cambios de tu PC a GitHub.

 **pull**
Bajar a tu PC (o al servidor) los cambios que están en GitHub.



 **Importante:** los secretos (contraseñas, claves de Cloudinary, Supabase y WhatsApp) **nunca** se suben a GitHub. El repositorio de la plataforma es público y el archivo `.gitignore` los excluye. El código nunca se edita directo en el VPS.

 **EXPLICACIÓN TÉCNICA**
Repositorios: `OskitarBB/agricola-riachuelo-platform` (rama `prueba2`) y `OskitarBB/agricola-riachuelo-mobile` (rama `fase4-sincronizacion`). `.gitignore` excluye `.env`, `db.sqlite3`, `media/`, `.venv/`, los pesos de IA (`*.pt`, `*.onnx`) y los respaldos. Si dos personas cambian las mismas líneas, Git avisa un **conflicto** y VS Code muestra las dos versiones para elegir.

 **HERRAMIENTA USADA Y POR QUÉ**
 **Git + GitHub** — **Para qué:** historial y trabajo en equipo. **Por qué:** estándar de la industria y gratis; el VPS se actualiza con un simple `git pull`.

Sin internet primero, y sin duplicados

Dos ideas que protegen los datos del campo: offline-first e idempotencia.

**Offline-first: primero guardar**

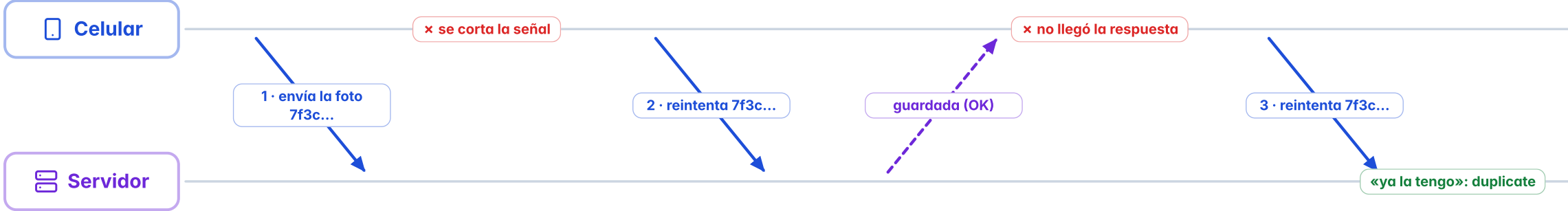
La app guarda todo en el celular aunque no haya señal y lo envía cuando vuelve la conexión. El trabajo de campo nunca se detiene.

**Idempotencia: sin duplicados**

Enviar lo mismo dos veces da el mismo resultado que enviarlo una vez. Si se corta la señal a mitad de un envío, se repite sin miedo.

**Identificadores únicos**

Cada sesión, pasada, secuencia, foto e incidencia nace en el celular con un código único (por ejemplo `captureId`).



 **Para entenderlo:** es como el número de un pedido de delivery: si llamas dos veces con el mismo número, no te traen dos pizzas.

**EXPLICACIÓN TÉCNICA**

Los identificadores son **UUID** (36 caracteres al azar) generados con `Crypto.randomUUID()` . En el servidor esos UUID son la clave primaria y hay restricciones `UNIQUE` (por ejemplo `cloudinary_public_id` , o pasada + número de secuencia). Si llega una foto con el mismo `captureId` y el mismo contenido, responde `duplicate: true` ; con otro contenido, `409 CAPTURE_CONFLICT` . En Cloudinary el nombre de la foto es su `captureId` y no se sobrescribe (`overwrite=false`).

**HERRAMIENTA USADA Y POR QUÉ**

 **SQLite (expo-sqlite)** — **Para qué:** guardar en el celular sin internet. **Por qué:** viene con Expo, es una base completa en un archivo y soporta transacciones.

¿Cómo sabe el sistema quién eres?

La web y la app prueban tu identidad de forma distinta, y ninguna guarda tu contraseña.

**Web: sesión con cookie**

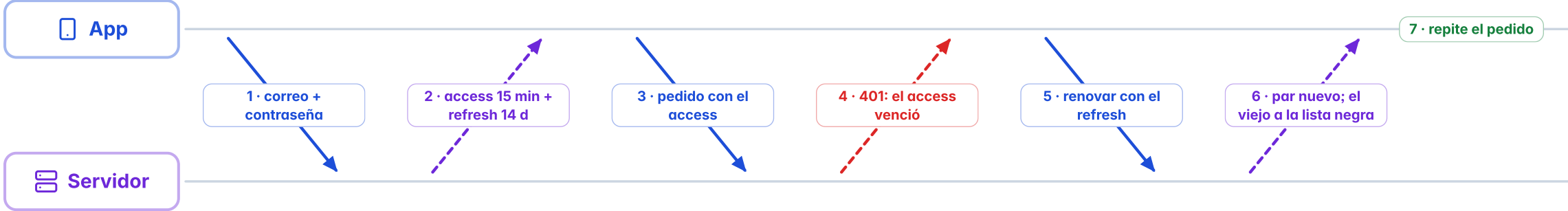
Al ingresar, el navegador recibe una cookie de sesión que dura **8 horas**. Cada clic la envía y Django sabe quién eres.

**App: token JWT**

La app recibe dos «pases» firmados: uno corto (**15 min**) para cada pedido y uno largo (**14 días**) para renovarlo sin escribir la contraseña.

**Contraseña con hash**

Nunca se guarda la contraseña: solo una huella (hash) que no se puede revertir. Al ingresar se calcula la huella y se compara.



🌟 **Para entenderlo:** la cookie es el sello en la mano de una discoteca; el JWT es la pulsera de un evento con hora de vencimiento; el hash es una huella digital: sirve para reconocerte, no para reconstruirte.

**EXPLICACIÓN TÉCNICA**

Web: `SESSION_COOKIE_AGE` = 8 h, cookie `HttpOnly` (JavaScript no la lee), `SameSite=Lax` y `Secure` con HTTPS. App: SimpleJWT con **rotación** (cada renovación entrega un refresh nuevo) y **lista negra** (el anterior deja de servir). Contraseñas con **PBKDF2**, el algoritmo por defecto de Django. Para entrar sin internet el celular guarda un verificador PBKDF2 (40 000 iteraciones) en SecureStore, nunca la contraseña.

**HERRAMIENTA USADA Y POR QUÉ**

 **SimpleJWT** — **Para qué:** emitir y renovar los tokens de la app. **Por qué:** se integra con Django REST Framework y trae rotación y lista negra listas.

La inteligencia artificial en palabras simples

Lo mínimo para entender qué hace YOLO y qué no hace.

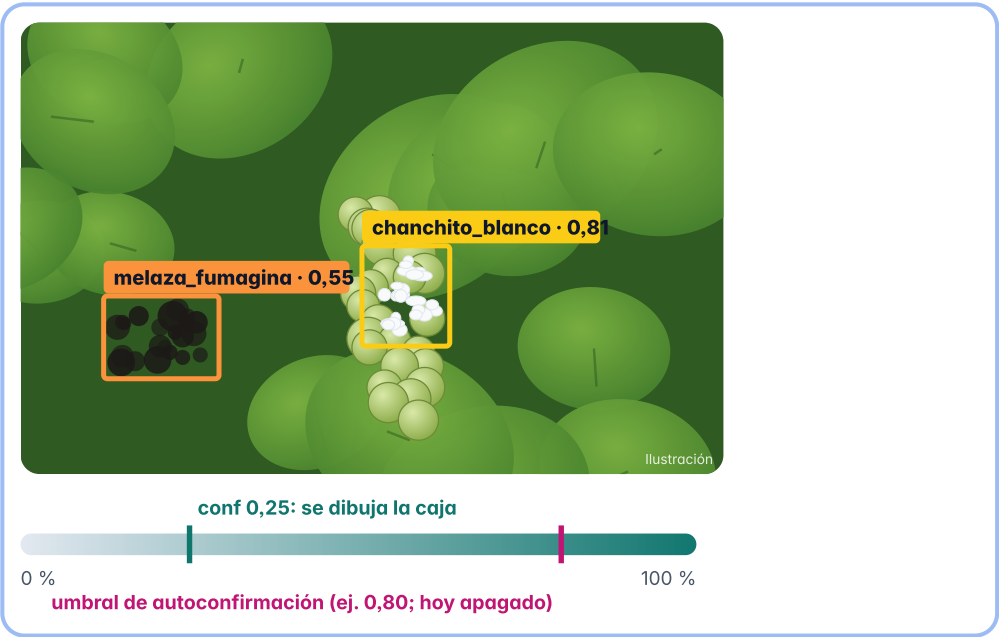
 **Visión por computadora**
Programas que «miran» una foto y encuentran cosas en ella. **YOLO** es un modelo muy rápido para eso.

 **Caja, clase y confianza**
Por cada cosa encontrada da una **caja** (dónde), una **clase** (qué cree que es) y una **confianza** de 0 a 100 %.

 **Entrenar**
Se le muestran muchas fotos con cajas dibujadas a mano (**etiquetado**) hasta que aprende a encontrarlas solo. Se hace una vez, con GPU.

 **Inferir**
Usar el modelo ya entrenado con fotos nuevas. Lo hace nuestro servidor con cada foto que llega, sin GPU.

🌟 **Para entenderlo:** entrenar es estudiar para el examen; inferir es rendir el examen; el umbral es la nota mínima para aprobar.



 **EXPLICACIÓN TÉCNICA**
Modelo **YOLO26n** (versión «nano», la más liviana) exportado a **ONNX**, un formato estándar que se ejecuta con ONNX Runtime en CPU. Clases: `chanchito_blanco` y `melaza_fumagina`. Confianza mínima 0,25 (debajo no hay caja) e IoU 0,45 (si dos cajas se solapan más de 45 %, queda la mejor). Las fotos de 12 MP se analizan por recortes de 1280 px con 20 % de solape (**tiling**) para no perder insectos pequeños.

 **HERRAMIENTA USADA Y POR QUÉ**
 **Ultralytics YOLO** — **Para qué:** entrenar y exportar el modelo. **Por qué:** es el estándar más usado en detección de objetos, rápido y bien documentado.

Coordenadas, teselas y GeoJSON

Lo básico de mapas para entender el mapa satelital de la web y «Ubicar plaga» en la app.

📍 Coordenadas

Latitud y longitud ubican un punto en la Tierra. El centro del fundo es **14°01'40.1"S 75°41'57.2"W** (−14,027806; −75,699222).

🗺 Teselas

Un mapa web se arma con muchas imágenes cuadradas pequeñas (**teselas**) que se descargan según el zoom. Las satelitales vienen de Esri.

📶 Precisión del GPS

Un celular común se equivoca entre **5 y 30 m**. Entre hileras separadas por pocos metros no basta: por eso también se guarda lote, hilera, lado y plantas.

🏠 GeoJSON

Texto para dibujar en un mapa: **puntos** (casos y lugares), **líneas** (hileras) y **polígonos** (contornos de lotes).



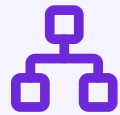
🔧 EXPLICACIÓN TÉCNICA

Coordenadas en grados decimales WGS84 (las del GPS), guardadas como `double precision` (unos 15 dígitos, de sobra para 6 decimales ≈ 10 cm). GeoJSON usa el orden **longitud, latitud** y un polígono termina en el mismo punto donde empieza. La distancia y el rumbo que muestra la app se calculan en el celular con la fórmula del **haversine**, sin internet. En el dibujo: contornos de lotes, hileras (inicio verde, fin rojo), un punto con nombre, alertas por color de estado y tu posición en azul.

🔗 HERRAMIENTA USADA Y POR QUÉ

📄 **Leaflet 1.9.4** — **Para qué:** dibujar mapas en la web y en la app. **Por qué:** liviano, libre y sin clave; se copia dentro del proyecto (sin CDN).

3



Arquitectura

Las piezas del sistema, cómo se comunican, por dónde pasa cada pedido y las reglas que no se rompen.

Arquitectura lógica: Django + Supabase + Clouinary

App móvil y plataforma Django en repositorios separados, con **una sola fuente de reglas** y un flujo lineal, sin callbacks · web **v1.3** · app **v0.5.0**

Repositorio 1: App móvil

- React Native Expo SDK 57 · TypeScript estricto · **v0.5.0**
- Operador de campo · **offline-first** (SQLite + archivos)
- Cámara, GPS, SQLite, FileSystem, SecureStore
- Wi-Fi local: WebSocket (órdenes) + HTTP (fotos)
- Sincroniza por /api/v1 - foto directa con ticket
- **¡NUEVO!** «Ubicar plagas»: mapa satelital y «Cómo llegar»
- **¡NUEVO!** Especialista: solo alertas, sin trabajo de campo

CONTROLADOR

📷 CAMERA_1

④ CAMERA_2

 **ESPECIALISTA · Ubicar plaga**

mismo lateral por pasada · la función se elige después de cada login

 Web de revisión (servida por Django)

- Plantillas Django + HTMX · Leaflet con **imágenes satelitales** (Esrri)
- Bandeja «Por revisar», revisión con cajas, mapa y plano
- Panel, historial de sesiones, reportes CSV y auditoría
- **NUEVO** Decisiones: confirmar · **posible plaga** · descartar · evidencia insuficiente
- **NUEVO** Editor del mapa: lotes, hileras y puntos con nombre
- **NUEVO** Cuentas: el administrador edita los datos de ingreso
- Django Admin (/gestion/): modelos de IA y ajustes avanzados

 Especialista

👉 **Supervisor**

✔ **Administrador**

🔗 Repositorio 2: Plataforma Django (backend central)

un solo código, dos procesos

Worker IA · proceso apart

- Toma tareas de la cola `ai_tasks`
- Descarga el original de Cloudinary
- YOLO (ONNX) cargado una sola vez
- Guarda cajas, confianza y estado
- **NUEVO** Abre el caso y, si supera el umbral, lo confirma
- Envía los WhatsApp ya confirmados

Proceso web · Django + DRF

- API REST versionada /api/v1
- JWT para la app · sesión para la web
- Reglas, permisos y estados oficiales
- Idempotencia por captureId · auditoría
- Firma los tickets de subida
- **NUEVO** Alertas para la app y editor del mapa

💡 ¿Por qué un solo código

- 1 Una sola fuente de verdad
- 2 La IA usa las mismas tablas
- 3 Sin callbacks entre servicios
- 4 Sin RabbitMQ, FastAPI ni Angular
- 5 Django Admin cubre la gestión

solo tras confirmar

 Cloudinary

Fotos de evidencia

- Subida directa con ticket (≤ 1 h)
- `public_id = captureId`: sin duplicados
- Privadas (authenticated) + CDN
- La BD guarda `public_id`, versión y bytes

descarga el origen

PostgreSQL gestionado · BD única

Cola de análisis

Tabla **ai_tasks** (SKIP LOCKED). Se encola en la misma transacción que la captura. **Reemplaza a RabbitMQ.**

- usuarios, roles y catálogos
- sesiones, pasadas y capturas
- detecciones, casos y auditoría
- sin acceso público: solo Django

WhatsApp Cloud AP

Aviso externo

- Avisa al confirmar: especialista o IA con umbral
- Un fallo de envío no cambia la decisión
- Enlaza al caso en Django (con login), no a la foto

☆ Ventajas prácticas

- 2 repositorios en vez de 4
- Sin RabbitMQ, FastAPI ni Angular que mantener
- Encolar = guardar: misma transacción
- El servidor no recibe los MB de cada foto
- La web carga miniaturas por CDN

Regla de arquitectura

- La app y el navegador **NO acceden a Supabase**: solo a Django
- La app sube fotos con un **ticket firmado** por Django, nunca con el API secret
- YOLO no es un servicio aparte: el worker escribe en la BD y **nadie hace callbacks**
- La sincronización no espera a YOLO

Las piezas, una por una

Qué hace cada parte del diagrama y con quién conversa.

Pieza	Qué hace	Con quién habla	Hecha con
App móvil	Captura en campo, guarda sin internet, sincroniza y muestra alertas	Django (API), Cloudinary (subir), Esri y Google Maps	React Native + Expo
Web de revisión	Pantallas para revisar, decidir, ver mapas, reportes y administrar	Django (HTML), CDN de Cloudinary, Esri	Plantillas + HTMX + Leaflet
Proceso web (Django)	Atiende la web y la API: reglas, permisos, tickets y auditoría	Supabase	Django 5.2 + DRF + gunicorn
Worker IA (Django)	Analiza fotos con YOLO, abre casos y envía avisos	Supabase, Cloudinary, WhatsApp	Python + ONNX Runtime
Supabase	Base de datos central: todo lo que no es una foto	Solo recibe a Django	PostgreSQL gestionado
Cloudinary	Guarda y entrega las fotos (miniatura y revisión)	Recibe la app (con ticket) y el worker	Almacenamiento + CDN
WhatsApp Cloud API	Lleva el aviso al jefe de fundo cuando se confirma	Lo llama solo el worker	API oficial de Meta
Mapas externos	Imágenes satelitales y rutas a pie	Los usa el navegador y la app	Esri + Google Maps (URL)

 **EXPLICACIÓN TÉCNICA**

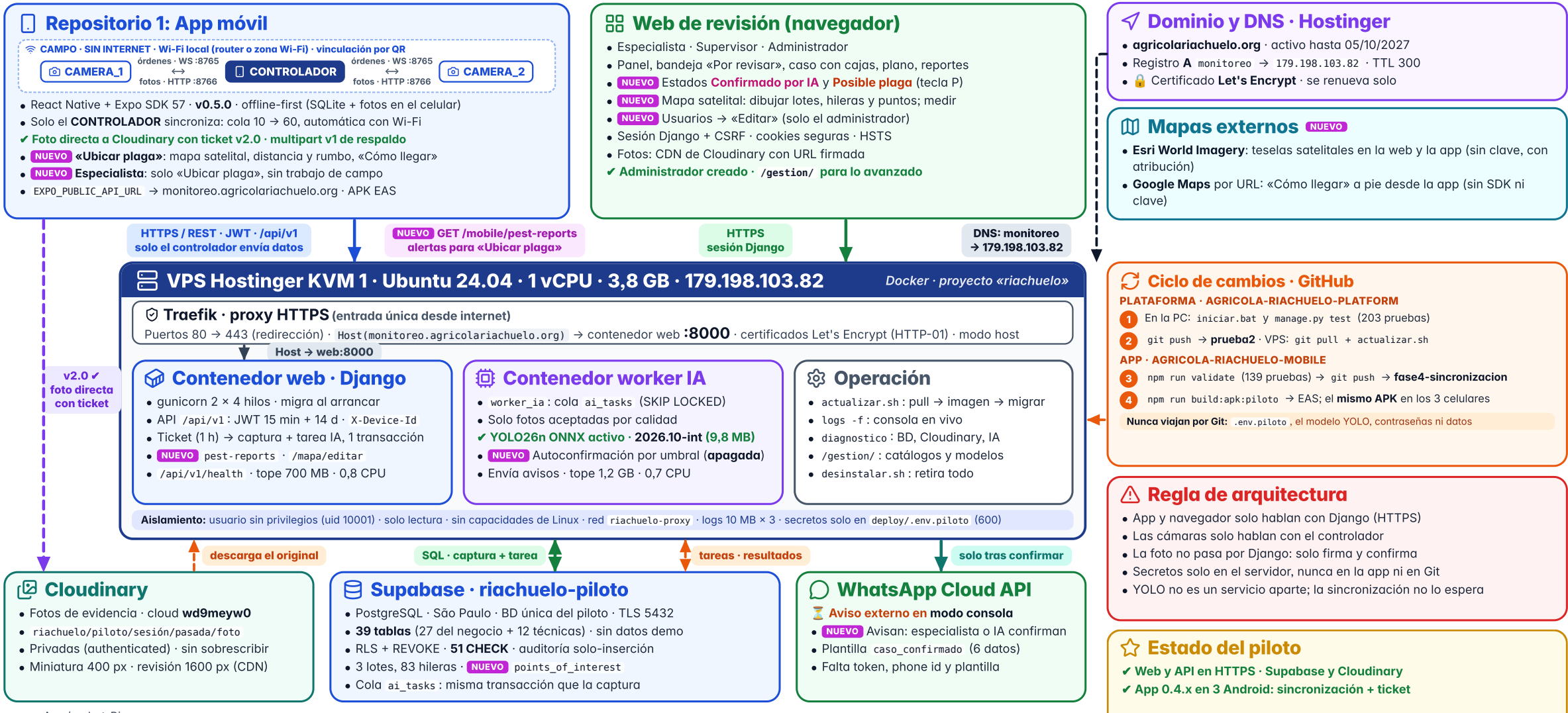
Hay **dos programas propios** (app y plataforma) y el resto son servicios administrados por terceros. La plataforma es **un solo código** que arranca como dos procesos: `gunicorn config.wsgi` (web y API) y `python manage.py worker_ia` (IA y avisos). Ambos usan los mismos modelos y servicios, así que una regla se escribe una vez. Ningún servicio externo llama de vuelta a la plataforma (sin callbacks): el sistema avanza porque cada parte deja su resultado en la base.

 **HERRAMIENTA USADA Y POR QUÉ**

 **Un monolito modular** — **Para qué:** tener web, API y worker en un repositorio. **Por qué:** para un equipo pequeño es más fácil de probar, desplegar y mantener que microservicios.

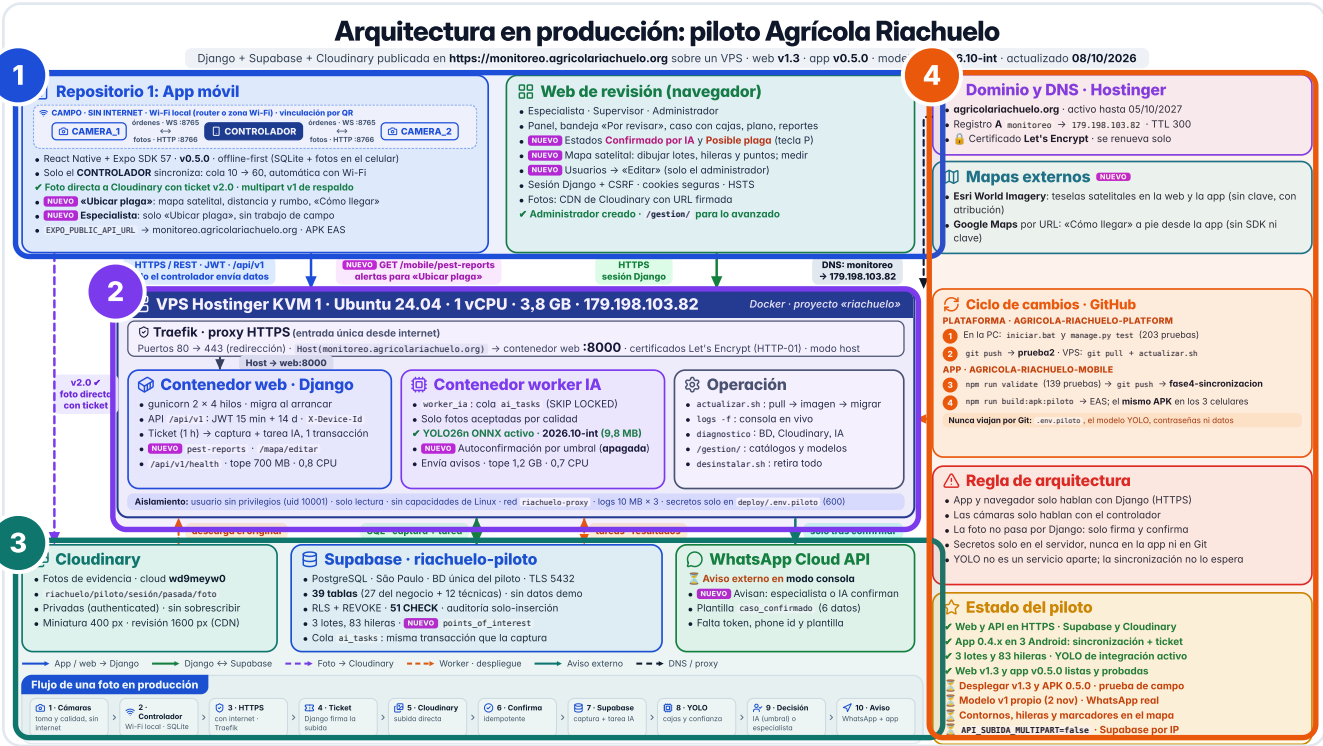
Arquitectura en producción: piloto Agrícola Riachuelo

Django + Supabase + Cloudinary publicada en <https://monitoreo.agricolariachuelo.org> sobre un VPS · web **v1.3** · app **v0.5.0** · modelo IA **2026.10-int** · actualizado **08/10/2026**



Cómo leer el diagrama de producción

El diagrama tiene cuatro zonas. Leerlo en este orden ayuda a no perderse.



➔ 1 · Quién entra

Arriba: la **app** (con sus 3 celulares en el Wi-Fi del campo), el **navegador**, el **dominio** y los **mapas externos**. Todo llega por HTTPS.

🗄 2 · El servidor (VPS)

Al centro: **Traefik** recibe todo y lo pasa al contenedor **web**; el **worker** trabaja aparte; abajo, cómo se aíslan.

☁ 3 · Servicios en la nube

Abajo: **Cloudinary** (fotos), **Supabase** (datos) y **WhatsApp** (avisos), con las flechas de quién usa qué.

⚙ 4 · Cómo se mantiene

A la derecha: el **ciclo de cambios** con GitHub, las **reglas** que no se rompen y el **estado** del piloto.

⚙ EXPLICACIÓN TÉCNICA

Las flechas usan colores fijos: **azul** app y web hacia Django (HTTPS), **verde** Django con Supabase (SQL con TLS), **morado punteado** foto directa a Cloudinary, **naranja punteado** el worker y el despliegue, **verde azulado** el aviso externo y **negro punteado** DNS y proxy. Las etiquetas **NUEVO** marcan lo agregado en la web v1.3 y la app v0.5.0. El mismo diagrama se entrega como imagen PNG aparte (3200 × 1800).

Quién habla con quién

Cada conexión del sistema, con su protocolo y su propósito.

Desde	Hacia	Cómo	Para qué
Cámaras 1 y 2	Controlador	Wi-Fi local · WebSocket :8765 y HTTP :8766	órdenes, latidos y fotos (con md5)
App (controlador)	Django /api/v1	HTTPS + JWT + X-Device-Id	login, catálogos, sincronizar, pedir tickets
App (controlador)	Cloudinary	HTTPS con ticket firmado por Django	subir el JPEG de cada foto (solo eso)
App («Ubicar plaga»)	Django · Esri · Google Maps	HTTPS	alertas, teselas satelitales y «Cómo llegar»
Navegador	Django	HTTPS + cookie de sesión + CSRF	todas las pantallas web
Navegador	Cloudinary (CDN) · Esri	URL firmada · HTTPS	ver fotos y miniaturas · teselas del mapa
Django (web y worker)	Supabase	PostgreSQL con TLS, puerto 5432	leer y guardar todos los datos
Worker IA	Cloudinary · WhatsApp	API firmada · HTTPS + token	bajar el original · enviar avisos
Cualquier otro	Supabase	—	bloqueado (RLS + REVOKE)

 **EXPLICACIÓN TÉCNICA**

Solo hay **tres** salidas a internet desde la app: Django, la subida a Cloudinary y, desde la v0.5.0, las teselas de Esri y el enlace de Google Maps (regla R-18 ampliada por ADR 0009). La app nunca tiene claves de Supabase, Cloudinary ni WhatsApp: los permisos los da Django en cada pedido. Supabase solo acepta a Django; falta además limitarlo a la IP del VPS (pendiente de seguridad).

 **HERRAMIENTA USADA Y POR QUÉ**

 **Principio de mínimo privilegio** — **Para qué:** que cada pieza tenga solo el acceso que necesita. **Por qué:** si se filtra algo de la app o del navegador, no abre la base ni las fotos.

El recorrido de una petición

Qué pasa por dentro desde que alguien toca un botón hasta que la base responde.



🕒 Rápido

La bandeja responde en ~39 ms con 6 consultas; el caso en ~46 ms. Si un pedido pasa de 1,5 s, la consola avisa que fue lento.

📶 Rastreado

Cada pedido recibe un **traceld**; si algo falla, ese código lleva a la línea exacta del error en la consola.

🛡️ Protegido en capas

HTTPS, sesión o token, permiso de la vista y regla del servicio: si una capa falla, la siguiente frena.

🔧 EXPLICACIÓN TÉCNICA

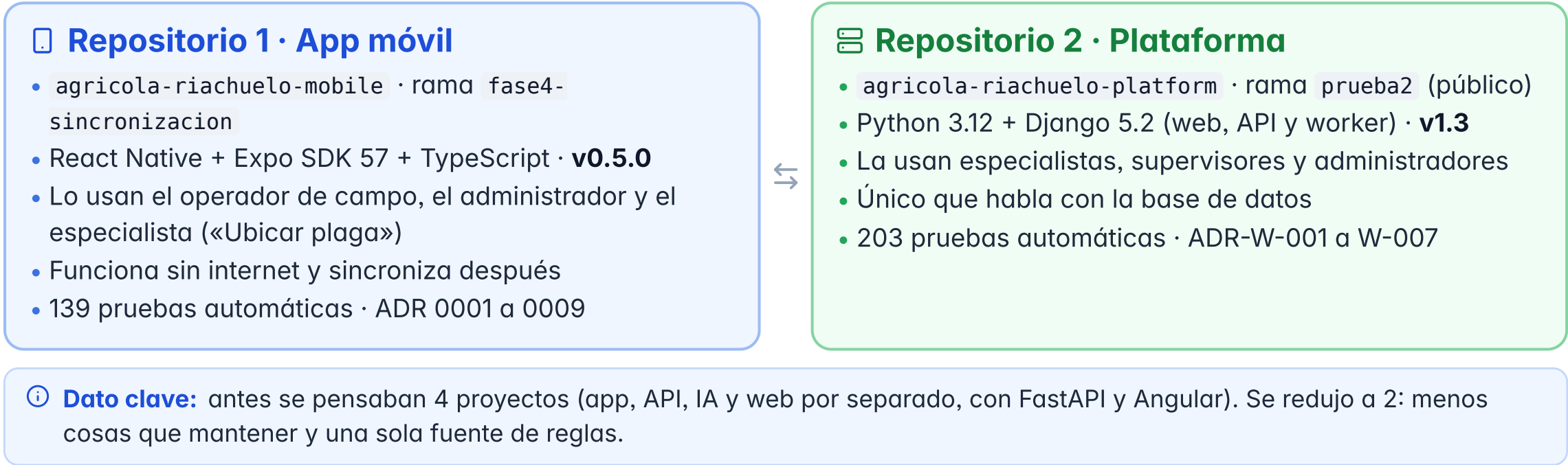
El middleware de diagnóstico (`diagnostico/`) asigna el `traceId` y escribe una línea por pedido (`LOG_REQUESTS=true` , aviso si supera `LOG_SLOW_MS=1500`). Las vistas web llevan `@web_view("permiso")` ; las de la API usan autenticación JWT + `X-Device-Id` y permisos como `FieldWork` o `PestReaders` . Toda escritura de varias tablas va dentro de `transaction.atomic()` (regla W-06).

🔗 HERRAMIENTA USADA Y POR QUÉ

🔧 gunicorn — **Para qué:** servir Django en producción. **Por qué:** servidor WSGI estable y simple; con 2 procesos × 4 hilos entra en 700 MB.

Dos repositorios, un sistema

El código está separado en dos proyectos que se comunican solo por la API.



Las reglas de la arquitectura

Son pocas, pero protegen la seguridad y la consistencia de los datos.



Nadie toca la base excepto Django

Ni la app ni el navegador conocen la contraseña de Supabase. Todo pasa por la API o la web.



Los secretos viven solo en el servidor

Claves de Cloudinary, Supabase y WhatsApp están en un archivo del VPS, no en la app ni en GitHub.



YOLO no es un servicio aparte

El análisis lo hace un proceso del mismo backend (el worker) que escribe directo en la base.



Nadie espera a la IA

La app termina de sincronizar aunque la IA todavía no haya analizado la foto.



La foto no pasa por Django

Django solo firma el permiso y confirma; el archivo va del celular a Cloudinary.



La IA sugiere, la persona decide

Solo hay aviso si el especialista confirma, o si la IA supera un umbral que el administrador activó a propósito.



EXPLICACIÓN TÉCNICA

Estas reglas salen de los maestros de la app (R-18, R-21, §28) y de la web (W-09, RN-W03, RN-W07) y de las decisiones ADR. El código las hace cumplir: la app solo arma URLs en `src/api/httpClient.ts` y solo habla con Cloudinary en `src/api/cloudinaryUpload.ts` ; el servidor rechaza fotos cuya firma de Cloudinary no coincide (`UPLOAD_SIGNATURE_INVALID`); y la base bloquea a todo usuario que no sea el de Django.

Qué cambió en la web v1.3 y la app v0.5.0

Lo nuevo del 08/10/2026, en un vistazo. Cada punto se explica en detalle más adelante.

📱 Web v1.3 (ADR-W-007)

- **Usuarios → Editar:** solo el administrador cambia nombre, correo, celular, código y contraseña
- **Confirmado por IA:** si el modelo tiene umbral y una caja lo supera, avisa por WhatsApp sin esperar
- **Posible plaga:** nueva decisión del especialista (tecla P), sin WhatsApp, visible en la app
- **Mapa satelital** (Esri) con editor: contornos, inicio y fin de hileras, puntos, medir
- **API** GET /mobile/pest-reports para «Ubicar plaga»
- El especialista puede iniciar sesión en la app (sin trabajo de campo)

📱 App v0.5.0 (ADR 0009 · CFG-8)

- **«Ubicar plaga»:** mapa satelital o lista, filtros y orden por cercanía
- Distancia y rumbo desde tu GPS («35 m al noreste ↗»); «Estás en el lugar» bajo 15 m
- **Cómo llegar:** abre Google Maps con ruta a pie
- Detalle con miniatura, lugar en palabras y precisión de la ubicación
- Copia guardada para verla sin internet; se actualiza cada 2 min
- Nuevo módulo nativo react-native-webview → APK nuevo (versionCode 10)

⚠ **Pendiente:** código escrito y probado (web 203 pruebas; app 139 pruebas, simulado 9/9 e integración 30/30). Falta subirlo a GitHub, desplegar la web en el VPS y compilar el APK 0.5.0 para los 3 celulares.

⚙️ EXPLICACIÓN TÉCNICA

Migraciones nuevas de la base: `campo/0004` (tabla `points_of_interest`), `ia/0003` (campo `auto_confirm_threshold` y estados de caja), `revision/0003` (estados `CONFIRMADO_POR_IA` y `POSIBLE_PLAGA`) y `notificaciones/0003` (tipo de aviso `CASO_CONFIRMADO_IA`). Variable nueva opcional: `WHATSAPP_TEMPLATE_IA`. Orden de despliegue: primero la web v1.3, después el APK; con la web v1.2 el especialista recibiría `ROLE_NOT_ALLOWED`.

Las decisiones registradas (ADR)

Cada cambio importante de diseño quedó escrito con su motivo.

ADR web	Decisión
ADR-W-001	Cloudinary simulado en desarrollo
ADR-W-002	Actividad en vivo por sondeo HTMX (sin websockets)
ADR-W-003	Subida de fotos multipart v1 por compatibilidad
ADR-W-004	Integridad en la base: CHECK y trigger de auditoría
ADR-W-005	Alta de cuentas y tipos de cuenta
ADR-W-006	Gestión de catálogos desde la web
ADR-W-007	Alertas de IA, mapa satelital, «Ubicar plaga» y edición de cuentas

ADR app	Decisión
0001	Expo SDK 57 y Expo Router
0002	Modos MANUAL / AUTOMÁTICO y estabilidad
0003	Red local de la mochila
0004	Ciclos y cobertura
0005	La función del celular se elige después del login
0006	Fase 4: sincronización
0007	Códigos de catálogo no únicos entre hileras
0008	Cerrar sesión con monitoreo abierto
0009	«Ubicar plaga» y el especialista en la app

 **EXPLICACIÓN TÉCNICA**
Un **ADR** (registro de decisión de arquitectura) es un archivo corto en `docs/adr/` con el contexto, la decisión y sus consecuencias. Sirve para que nadie «deshaga» una decisión sin saber por qué se tomó, y para que una IA o un integrante nuevo entiendan el sistema. Si un ADR contradice al maestro, manda el ADR (es posterior).

 **HERRAMIENTA USADA Y POR QUÉ**
 **ADR en Markdown** — **Para qué:** documentar decisiones junto al código. **Por qué:** viajan con Git: cada versión del código trae sus decisiones.

4



Flujo completo

El recorrido de una foto: desde que se toma en la hilera hasta el aviso por WhatsApp y la visita a la planta.

El viaje de una foto (1/3): en el campo

Del disparo hasta que la foto queda guardada en el controlador, sin necesitar internet.

1



El controlador da la orden

En la pasada, el operador toca «Capturar» (o el modo automático dispara cada 1 a 5 s). El controlador crea una **secuencia** con su GPS.

2



Las 2 cámaras disparan

Las dos cámaras (en un soporte de hombros) miran el **mismo lado** de la hilera y toman su foto casi a la vez: hasta **12 MP**, JPEG 85.

3



Se revisa la calidad

En el mismo celular se mide la luz y la nitidez en ≤ 2 s. Si sale oscura o movida, **se repite solo esa cámara**.

4



Viaja al controlador

La foto va por el Wi-Fi local al controlador con su huella **md5** y se guarda en su base SQLite con su `captureId`.

```
# lo que el controlador guarda de cada foto (tablas captures, capture_sequences y quality_results)
captureId      "7f3c2a9e-..."      ← identificador único (UUID)      cameraRole  CAMERA_1      ← qué cámara la tomó
sessionId · passId · sequenceId  ← a qué recorrido pertenece  lateralCode  LATERAL_A      ← lado de la hilera
lot · row · segment · marker      ← dónde (va en la secuencia)  quality      UTILIZABLE      ← resultado de calidad Q0
lat · lon · gpsAccuracyM          ← GPS del controlador, si es válido  capturedAt    2026-10-08T15:42:10Z
```

Dato clave: las fotos no llevan GPS dentro (el EXIF va apagado): la ubicación vive en la **secuencia** del controlador, junto con lote, hilera, lateral, segmento y marcador.

 **EXPLICACIÓN TÉCNICA**

La orden viaja por WebSocket (puerto 8765) y cada cámara debe responder en 8 s (`captureResponseTimeoutMs`); no se emite otra orden mientras la anterior no termine (RN-11). La foto viaja por HTTP (puerto 8766, `POST /local/captures`) con los datos en la cabecera `X-Capture-Meta` . Una secuencia queda COMPLETE solo si las dos cámaras tienen foto aceptada y recibida (RN-06). Calidad Q0: exposición sobre una copia de 256 px y nitidez con la varianza del Laplaciano en 3 recortes de 512 px.

 **HERRAMIENTA USADA Y POR QUÉ**

 **expo-camera** — **Para qué:** tomar la foto con la cámara nativa. **Por qué:** es el módulo oficial de Expo y permite elegir resolución y calidad JPEG.

El viaje de una foto (2/3): del celular a la nube

Cuando hay internet, el controlador envía todo en orden y las fotos van directo a Cloudinary.

5



Hay internet: sincroniza

Con Wi-Fi (automático) o con «Sincronizar ahora», el controlador envía en orden: sesión, pasadas, secuencias, fotos, incidencias y cierre.

6



Pide un ticket

Por cada foto pide a Django un permiso firmado (**ticket**) que vale para esa sola foto y por 1 hora.

7



Sube directo a Cloudinary

El JPEG va del celular a Cloudinary con ese ticket. La foto **no pasa por nuestro servidor**.

8



Confirma a Django

La app avisa «ya subí». Django verifica la firma y guarda la captura y su tarea de IA **en una sola operación**.



 **Dato clave:** si se corta la conexión, nada se pierde: la cola reintenta (1, 5, 15 y 60 minutos) y el servidor no duplica, porque reconoce cada `captureId` .

 **EXPLICACIÓN TÉCNICA**

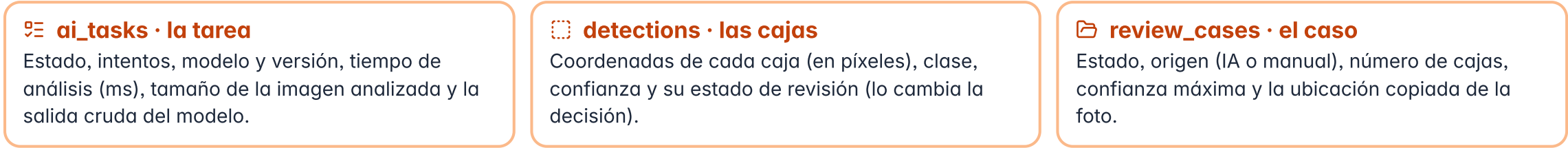
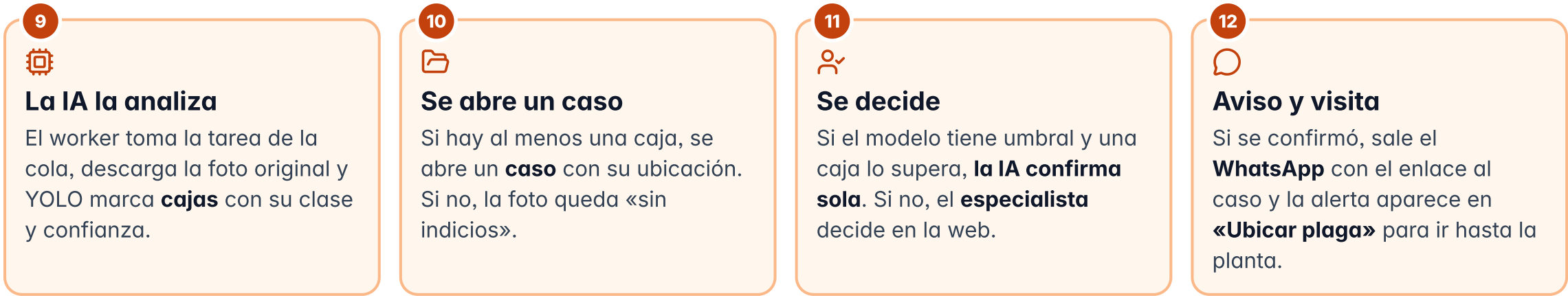
POST /api/v1/captures/{id}/upload-ticket → subida a Cloudinary con los campos firmados (`public_id` , `type=authenticated` , `overwrite=false` , `timestamp` , `signature`) → POST /api/v1/captures/upload con el JSON que devolvió Cloudinary. Si la app se cierra entre la subida y la confirmación, al volver solo confirma (tabla local `remote_uploads`). Una foto está SINCRONIZADA solo cuando Django responde OK (RN-23). Solo las fotos UTILIZABLE o PENDIENTE_REVISION_TECNICA reciben tarea de IA (RN-W11).

 **HERRAMIENTA USADA Y POR QUÉ**

 **expo-file-system (uploadAsync)** — **Para qué:** enviar el archivo a Cloudinary. **Por qué:** sube el archivo directo desde el disco, sin cargarlo en memoria ni usar un SDK de la nube (R-21).

El viaje de una foto (3/3): del servidor a la decisión

La IA ordena el trabajo; la decisión y el aviso cierran el ciclo.



Importante: la IA «sugiere indicios», no diagnostica. La web nunca dice «plaga detectada», «infestación» ni «severidad» (regla W-11), y los colores indican el estado de revisión, no la gravedad (W-12).

EXPLICACIÓN TÉCNICA

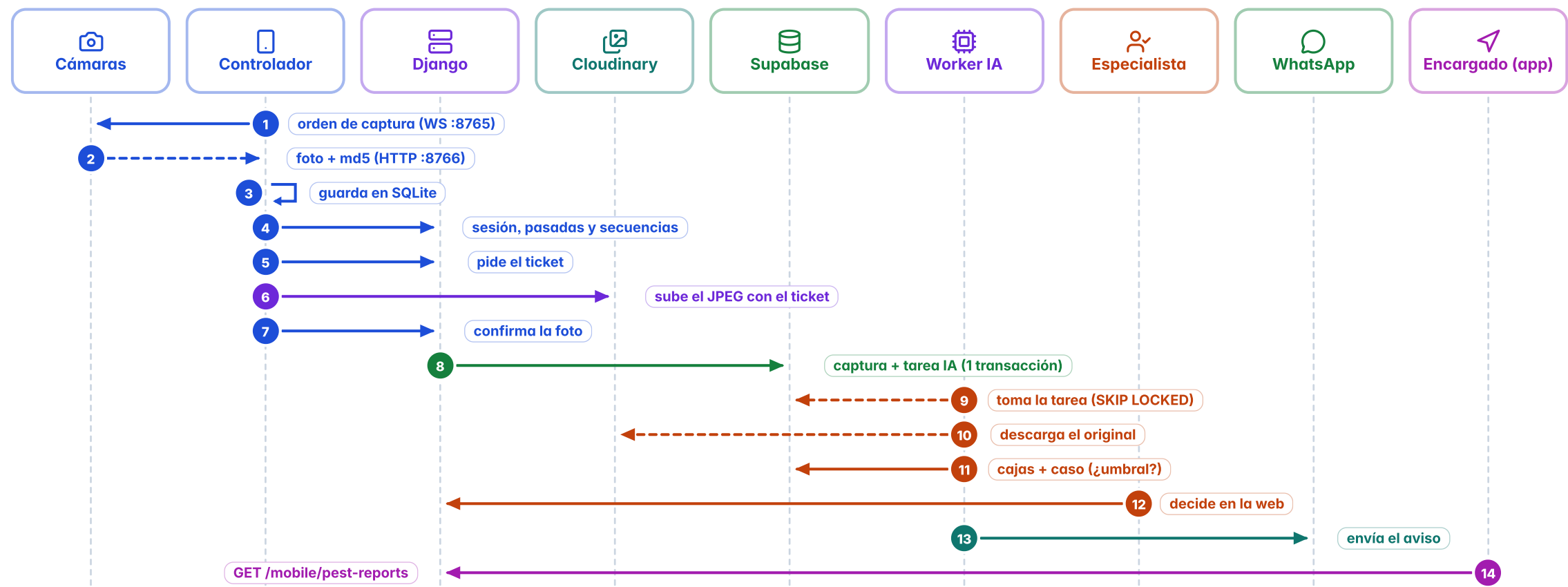
La tarea pasa de `PENDIENTE_DE_ANALISIS` a `EN_ANALISIS` y termina en `INDICIO_SUGERIDO_POR_IA`, `SIN_INDICIOS_IA` o `ERROR_DE_ANALISIS`. El caso se abre en la misma transacción que guarda las cajas (`open_case_from_analysis`), y los avisos se crean en la misma transacción que la decisión. El worker los envía después con su propia cola y reintentos: un error de envío nunca cambia la decisión (RN-W03).

HERRAMIENTA USADA Y POR QUÉ

ONNX Runtime — **Para qué:** ejecutar YOLO en el servidor. **Por qué:** corre en CPU (el VPS no tiene GPU) y no necesita instalar todo el entorno de entrenamiento.

La secuencia completa, actor por actor

Quién le envía qué a quién, en orden. Las flechas punteadas son respuestas o lecturas.



 **EXPLICACIÓN TÉCNICA**

Los pasos 1–3 ocurren sin internet. Del 4 al 7 la app usa la API; el 8 lo hace Django dentro de una transacción. Del 9 al 11 trabaja el worker, que nunca atiende visitas de internet. El 12 es una pantalla web; el 13 vuelve a ser el worker (cola `notifications`) y el 14 es la app pidiendo las alertas.

¿Y si no hay internet en el campo?

La app está pensada como offline-first: primero guarda, después envía.

 **Sin señal**

La app sigue funcionando: guarda sesiones y fotos en el celular y muestra cuántas faltan enviar.

 **Vuelve la señal**

Envía todo lo pendiente en orden. Si se corta a la mitad, lo vuelve a intentar más tarde.

 **Sin duplicados**

Cada foto tiene su `captureId` . Si llega dos veces, el servidor la reconoce y no la repite (idempotencia).

 **Login sin internet**

Tras el primer ingreso con internet, el usuario puede entrar sin señal hasta **7 días**.

 **La mochila no necesita internet**

Las cámaras y el controlador se hablan por el Wi-Fi local (router o zona Wi-Fi), aunque no haya datos móviles.

 **«Ubicar plaga» sin internet**

Muestra la última copia guardada y calcula distancia y rumbo con el GPS. La imagen satelital y Google Maps sí necesitan datos.

 **EXPLICACIÓN TÉCNICA**

Cola local `sync_queue` con prioridades (10 SESSION → 20 PASS → 30 SEQUENCE_BATCH → 40 CAPTURE → 50 INCIDENT_BATCH → 60 SESSION_CLOSE), reintentos a 1, 5, 15 y 60 min (`sync.retryDelaysMs`), lotes de 200 y una tarea a la vez. La sincronización automática corre solo con Wi-Fi, con la app abierta y sin sesión de monitoreo abierta, cada 5 min. Login sin internet: verificador PBKDF2, máximo 7 días, 5 intentos y bloqueo de 15 min (RN-03, RN-20).

 **HERRAMIENTA USADA Y POR QUÉ**

 **expo-network** — **Para qué:** saber si hay internet y si es Wi-Fi. **Por qué:** evita gastar datos móviles subiendo fotos de 3–4 MB.

Qué pasa cuando se decide

Cada estado del caso tiene un responsable y decide si hay aviso y si aparece en la app.

Estado del caso	¿Quién lo pone?	¿WhatsApp?	¿En «Ubicar plaga»?	Qué significa
Pendiente de revisión	El worker, al abrir el caso	No	Sí (en revisión)	La IA vio un indicio; falta decidir.
Confirmado por IA v1.3	El worker, si el modelo tiene umbral y una caja lo supera	Sí	Sí	Alta confianza; el especialista igual puede corregirlo.
Confirmado por especialista	El especialista: «Confirmar» (C)	Sí, a los destinatarios del lote	Sí	Es plaga. La observación es obligatoria.
Posible plaga v1.3	El especialista (P)	No	Sí	Puede ser plaga: que el encargado vaya a verla.
Evidencia insuficiente	El especialista (I)	No	No	La foto no permite decidir; dice qué falta.
Descartado	El especialista (D)	No	No	No corresponde a una plaga.

 **Dato clave:** toda decisión se guarda en la auditoría: quién decidió, cuándo y qué cambió. Esa tabla no se puede editar ni borrar. Corregir una decisión crea otra fila con su motivo; la anterior se conserva.

 **EXPLICACIÓN TÉCNICA**

DECIDIBLES

 = PENDIENTE_REVISION y CONFIRMADO_POR_IA (los demás se «corrigen»).

CON_AVISO

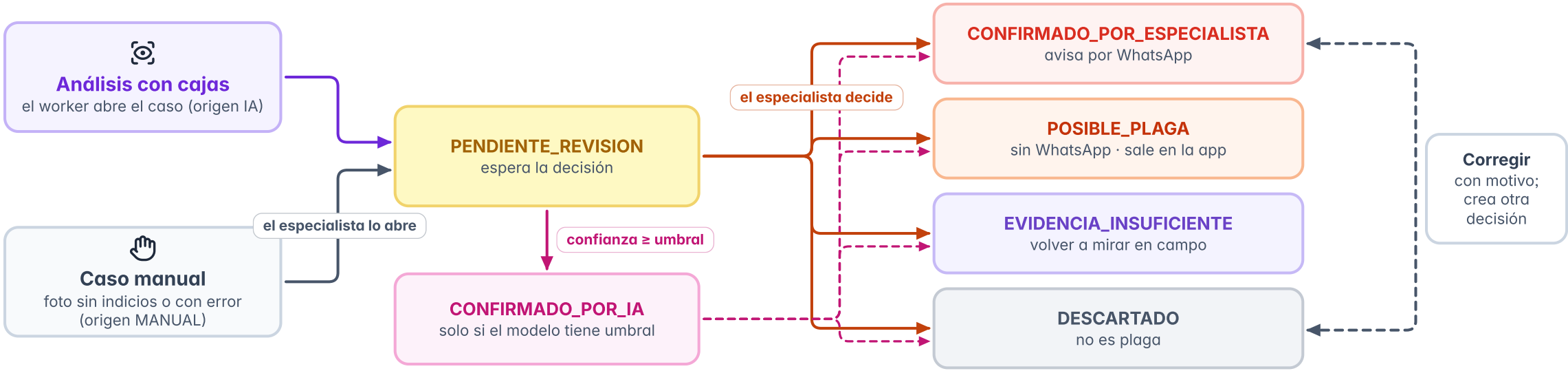
 = CONFIRMADO_POR_IA y CONFIRMADO_POR_ESPECIALISTA.

VISIBLES_EN_APP

 = los cuatro primeros. Confirmar un caso ya confirmado por la IA no duplica avisos; descartarlo o dejarlo como posible anula los pendientes (lo ya enviado no se retira, Q-W07). Observación: obligatoria al confirmar y al marcar evidencia insuficiente, máximo 2 000 caracteres, sin tratamientos ni dosis (RN-W04).

Los estados de un caso

Un caso nace pendiente (o confirmado por la IA) y el especialista lo lleva a su estado final.



EXPLICACIÓN TÉCNICA

Un caso por foto; una sola decisión vigente (índice único parcial en `human_reviews`). Solo se «decide» un caso en `DECIDIBLES`; después solo se «corrige», con motivo y comprobando que la decisión vigente sea la que el especialista tenía en pantalla (RN-W05, bloqueo `select_for_update`). Un análisis nuevo con cajas actualiza un caso pendiente; uno sin cajas no lo cierra; ninguno cambia una decisión tomada (RN-W06). Un caso manual solo se abre sobre una foto aceptada y con análisis `SIN_INDICIOS_IA` o `ERROR_DE_ANALISIS` (RN-W09).

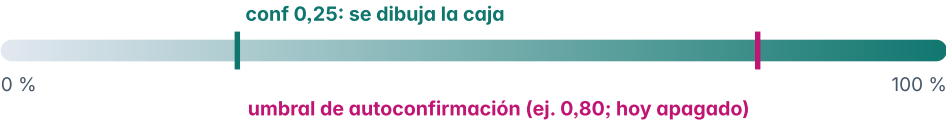
HERRAMIENTA USADA Y POR QUÉ

Transacciones de PostgreSQL — **Para qué:** que decisión, cajas, avisos y auditoría se guarden juntos. **Por qué:** si algo falla a la mitad, no queda nada a medias.

La IA puede confirmar sola (con umbral)

Novedad de la v1.3: si un modelo está bien validado, un caso muy claro avisa de inmediato, sin esperar.

🕒 Cómo decide el worker



0,93 **Confirmado por IA → WhatsApp**
confianza más alta del caso, con umbral 0,80

confianza más alta del caso, con umbral 0,80

0,62 **Pendiente de revisión**
confianza más alta del caso, con umbral 0,80

confianza más alta del caso, con umbral 0,80

0,18 Sin caja (bajo 0,25)
confianza más alta del caso, con umbral 0,80

confianza más alta del caso, con umbral 0,80

 Nace apagado

El campo `auto_confirm_threshold` del modelo está vacío: ningún caso se confirma solo hasta que el administrador lo active.

 Se activa por modelo

En `/gestion/` → Modelos de IA, para un modelo concreto (por ejemplo 0,80).

 Solo con un modelo validado

Primero `verificar_onnx.py` con fotos del fundo: debe detectar ≥ 85 % de las fotos con plaga.

↩ Se puede corregir

El especialista sigue decidiendo los «Confirmado por IA»; lo ya enviado no se retira.

⚠ **Importante:** con el modelo actual (de integración, entrenado con fotos de internet, mAP50 $\approx$ 0,50) el umbral debe seguir apagado: generaría falsas alarmas en el WhatsApp.

EXPLICACIÓN TÉCNICA

`_auto_confirm_if_confident` (`revision/services.py`): si hay umbral, el caso está pendiente y su `max_confidence` lo alcanza, marca esas cajas como `CONFIRMADO_POR_IA`, pone el caso en `CONFIRMADO_POR_IA` sin usuario (`decided_by` nulo), encola avisos de tipo `CASO_CONFIRMADO_IA` (plantilla `WHATSAPP_TEMPLATE_IA`) y audita `CASO_CONFIRMADO_POR_IA` con la confianza, el umbral y la versión del modelo.

El aviso por WhatsApp

Lo recibe quien tiene que actuar, con el enlace al caso en la web.

Riachuelo · Monitoreo

Caso confirmado

Lote: **SWG 5** — Lote 5 (Triángulo)

Hilera: **31** · Lateral A

Segmento SWG5-H31-S1 · marcador SWG5-H31-INI

08/10/2026 10:42

[monitoreo.agricolariachuelo.org/casos/...](#)

10:43 ✓✓

Ejemplo ilustrativo: el texto final lo fija la plantilla aprobada por Meta.

-  ¿A quién?

A los **destinatarios** activos, con consentimiento registrado y con el lote del caso en su alcance (o sin alcance = todos los lotes). RN-W10.
-  ¿Cuándo?

Cuando el caso queda **Confirmado por especialista** o **Confirmado por IA**. Nunca por pendiente, posible, descartado o evidencia insuficiente.
-  ¿Cómo?

El worker los envía con una cola propia: hasta 5 intentos, esperando 2, 4, 8 y 16 min.
-  Estados del aviso

PENDIENTE_ENVIO → ENVIADO o ERROR_ENVIO. Si el caso se corrige antes de enviar, queda NO_APLICA.

⚠ **Pendiente:** hoy el envío está en **modo consola**: el aviso se escribe en el registro del worker. Falta la cuenta de Meta (token, número y la plantilla `caso_confirmado` aprobada) y cargar los destinatarios reales.

 **EXPLICACIÓN TÉCNICA**

Plantilla `caso_confirmado` en español con 6 parámetros: lote, hilera, lateral, «segmento X · marcador Y», fecha y hora, y enlace al caso (`PUBLIC_BASE_URL` + `/casos/<id>/`). El enlace pide iniciar sesión y nunca apunta a la foto (DW-26). Para enviar de verdad:
`WHATSAPP_CLIENT=notificaciones.whatsapp.CloudApiClient` + `WHATSAPP_TOKEN` , `WHATSAPP_PHONE_ID` y `WHATSAPP_GRAPH_VERSION` en `.env.piloto` , y `bash actualizar.sh` .

 **HERRAMIENTA USADA Y POR QUÉ**

 **WhatsApp Cloud API (Meta)** — **Para qué:** enviar el aviso al celular del jefe de fundo. **Por qué:** es la vía oficial; WhatsApp es lo que el personal ya usa todos los días.

De dónde sale la ubicación de un caso

Una regla fija decide qué coordenadas se usan, en este orden.



- 📍 Por qué no basta el GPS**
Un celular de gama de entrada se equivoca entre 5 y 30 m (en pruebas, 16 a 28 m). Las hileras están a pocos metros.
- 📋 Por eso se guarda todo**
Lote, hilera, lateral (lado), segmento (plantas X a Y) y marcador: con eso se llega a la planta aunque el GPS falle.
- 🔒 Se fija al abrir el caso**
La ubicación se copia al caso y ya no cambia, aunque después se edite el catálogo.

🔧 EXPLICACIÓN TÉCNICA

`case_location` en `revision/services.py` (DW-13, RN-W12): contexto de la repetición si existe; si no, la secuencia; coordenadas del GPS (`location_source = GPS`), si no las del marcador (`MARCADOR`), si no ninguna (`NINGUNA`). En el celular nunca se inventan coordenadas: si la última lectura tiene más de 10 s, la secuencia queda sin GPS (RN-18). Hoy los marcadores del piloto aún no tienen coordenadas: se cargan desde el editor del mapa (v1.3).

«Ubicar plaga»: del aviso a la planta

El último tramo del flujo, nuevo en la app v0.5.0: llevar a una persona hasta el lugar.

1



Se confirma o se sospecha

La IA (con umbral) o el especialista confirman, o el especialista marca «posible plaga».

2



La app trae las alertas

Pide `pest-reports` al abrir, cada 2 min con internet y con «Actualizar». Guarda una copia.

3



Mapa o lista

Cada alerta con su color de estado, el lugar en palabras y «35 m al noreste ↗» desde tu GPS.

4



Cómo llegar

Abre Google Maps con la ruta **a pie** hasta las coordenadas del caso.

5



En el lugar

Bajo 15 m dice «Estás en el lugar». Lote, hilera, lado y plantas llevan a la planta exacta.



Ilustración · imagen satelital Esri

```
# «Cómo llegar» abre esta dirección (sin clave de API):
https://www.google.com/maps/dir/?api=1
&destination=-14.027806,-75.699222
&travelmode=walking

# línea de la alerta en la app:
Chanchito blanco · SWG 1 · H06 · Lateral A · plantas 1-25
35 m al noreste ↗
```

 **Nuevo:** esto cierra el indicador del curso **«tiempo para localizar un hallazgo»**: antes había que recordar o preguntar; ahora la app guía hasta la planta.

 **EXPLICACIÓN TÉCNICA**

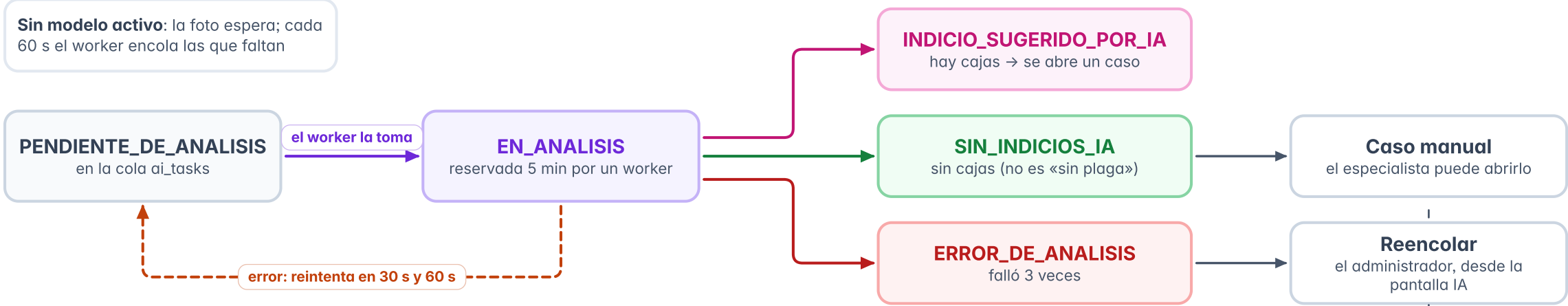
Las alertas visibles son `CONFIRMADO_POR_IA` , `CONFIRMADO_POR_ESPECIALISTA` , `POSIBLE_PLAGA` y `PENDIENTE_REVISION` de los últimos 30 días (máximo 90). El orden de la lista: primero lo confirmado, luego posible y en revisión; dentro de cada grupo, lo más cerca de ti (sin GPS, lo más reciente). Una alerta sin coordenadas (`NINGUNA`) se muestra igual, con lote, hilera, lado y plantas, pero sin «Cómo llegar».

 **HERRAMIENTA USADA Y POR QUÉ**

 **Google Maps (URL universal)** — **Para qué:** guiar a pie. **Por qué:** `google.com/maps/dir/?api=1&destination=lat,lon&travelmode=walking` funciona sin clave ni pago y abre la app o el navegador.

Las tareas de la IA y sus estados

Cada foto aceptada crea una tarea; así se ve su vida dentro de la cola.



EXPLICACIÓN TÉCNICA

ia/services.py : la tarea se toma con `SELECT ... FOR UPDATE SKIP LOCKED` (dos workers nunca toman la misma), se «arrienda» 5 min (`locked_until`) y suma un intento; si el worker muere, vuelve a estar libre al vencer el arriendo. Tras 3 intentos queda `ERROR_DE_ANALISIS` (que no es un negativo). Hay una tarea por foto y por modelo (`UNIQUE`); al cambiar de modelo, el worker encola las fotos que no tienen análisis de ese modelo (`enqueue_missing` , cada 60 s).

HERRAMIENTA USADA Y POR QUÉ

⇒ **Cola en una tabla (ai_tasks)** — **Para qué:** repartir el trabajo de la IA. **Por qué:** no hace falta RabbitMQ ni Redis: la misma base garantiza orden, reintentos y que nada se pierda.

5



Tecnologías y herramientas

Cada pieza del stack: qué es, para qué la usamos y por qué la elegimos frente a otras opciones.

El mapa del stack

Una vista rápida antes de ver cada pieza, con el motivo de cada elección.

Capa	Tecnología	Para qué	Por qué esta
App móvil	React Native + Expo SDK 57 · TypeScript · SQLite	Capturar en campo, funcionar sin internet, ubicar plagas	Un solo código para Android e iPhone; acceso simple a cámara y GPS
Backend	Python 3.12 · Django 5.2 LTS · Django REST Framework	Reglas, API, web y procesos	Trae usuarios, seguridad, admin y ORM listos; soporte largo (LTS)
Web	Plantillas Django · HTMX 2.0 · Leaflet 1.9.4	Revisión, mapa satelital y plano	Sin compilar JavaScript ni npm; liviano y rápido
Base de datos	PostgreSQL en Supabase	Guardar todo de forma segura	Relacional, con CHECK, transacciones y triggers; administrada
Fotos	Cloudinary	Almacenar y mostrar imágenes	Tamaños automáticos, CDN y fotos privadas con firma
IA	YOLO26n (Ultralytics) exportado a ONNX	Sugerir indicios en las fotos	Rápido y liviano; corre en CPU con ONNX Runtime
Avisos	WhatsApp Cloud API (Meta)	Notificar casos confirmados	Es el canal que el personal ya usa
Mapas	Esri World Imagery · Google Maps (URL)	Imágenes satelitales y rutas a pie	Sin clave ni pago; con atribución visible
Servidor	VPS Ubuntu · Docker · Traefik · gunicorn	Publicar todo con HTTPS	Control total a bajo costo; un comando para actualizar

 **EXPLICACIÓN TÉCNICA**

El criterio fue siempre el mismo: **pocas piezas, bien conocidas y gratuitas o baratas** para un piloto. Se descartaron RabbitMQ (la cola es una tabla), FastAPI (Django ya trae API con DRF), Angular o React en la web (HTMX alcanza), Supabase Auth/Storage (Django y Cloudinary ya lo cubren) y SDK de mapas con clave (Leaflet + Esri + URL de Google Maps no cuestan nada).

Python y Django

Python es el lenguaje del backend. Django es un framework: un «kit» que ya trae resuelto lo común.

📦 Qué es Django

Un framework web de Python «con baterías incluidas»: usuarios, seguridad, formularios y panel de administración listos.

🗄️ ORM

Permite trabajar con la base de datos usando Python, sin escribir SQL a mano. Cada clase del código es una tabla.

⚙️ Django Admin

Un panel automático en `/gestion/` para la configuración avanzada: modelos de IA, perfiles de calidad y consultas.

🛡️ Seguridad de fábrica

Protección contra ataques comunes (CSRF, inyección SQL, clickjacking), contraseñas con hash y cookies seguras.

🌟 **Para entenderlo:** Python es el idioma; Django es una casa prefabricada con cocina, baño y cerraduras ya instaladas: nosotros solo ponemos los muebles (las reglas del negocio).

🔧 EXPLICACIÓN TÉCNICA

Django **5.2.17 LTS** (soporte de seguridad por varios años) sobre Python 3.12 (imagen `python:3.12-slim-bookworm`). Comandos con `manage.py`: `migrate`, `createsuperuser`, `test`, `diagnostico`, `worker_ia`, `respaldo`, `mantenimiento`. Idioma `es-pe` y zona `America/Lima` (se guarda en UTC). No se sube a Django 6 mientras SimpleJWT no lo soporte.

🔗 HERRAMIENTA USADA Y POR QUÉ

📦 **Django 5.2 LTS** — **Para qué:** construir web, API y worker con un solo framework. **Por qué:** maduro, seguro y con documentación en español; el equipo lo puede aprender rápido.

</> Así se ve en nuestro código

```
# revision/models.py — cada clase es una tabla
class ReviewStatus(models.TextChoices):
    PENDIENTE_REVISION = "PENDIENTE_...", ...
    DESCARTADO = "DESCARTADO", "Descartado"
    CONFIRMADO_POR_IA = "CONFIRMADO_...", ...
    POSIBLE_PLAGA = "POSSIBLE_PLAGA", ...
# los dos últimos son de la v1.3
```

```
# config/restricciones.py convierte cada lista de
# estados en una regla CHECK de la base
```

Django REST Framework y JWT

Es la parte de Django que atiende a la app móvil.

 **DRF**

Extensión de Django para construir APIs: recibe JSON, valida los datos y responde JSON.

 **JWT (token)**

Al iniciar sesión, la app recibe un «pase» firmado. Lo envía en cada pedido para demostrar quién es.

 **Dos tokens**

Access: dura 15 minutos. **Refresh:** dura 14 días y sirve para pedir un access nuevo sin volver a escribir la contraseña.

 **Documentación automática**

La API se describe sola en `/api/v1/schema` (formato OpenAPI), útil para el equipo de la app.

 **Para entenderlo:**

el JWT es como la pulsera de un evento: te la dan en la entrada (login), la muestras en cada puerta y caduca a cierta hora.

 EXPLICACIÓN TÉCNICA <code>django</code><code>restframework</code> 3.18.1, <code>django</code><code>restframework-simplejwt</code> 5.5.1 (rotación del refresh y lista negra en las tablas <code>token_blacklist_*</code>) y <code>drf-spectacular</code> 0.30.0 (OpenAPI). Límites de intentos (throttles): login 10 por minuto, registro 20 por hora, renovar 60 por minuto; al pasarse responde <code>429 T00_MANY_ATTEMPTS</code> con <code>Retry-After</code>. La cabecera <code>X-Device-Id</code> ata el token a un celular registrado, que el administrador puede revocar.	 HERRAMIENTA USADA Y POR QUÉ  Django REST Framework — Para qué: la API de la app. Por qué: reutiliza los mismos modelos y servicios de la web; no hace falta otro servidor (FastAPI).
---	---

</> Así se ve en nuestro código

```
# api/v1/views.py – la vista nueva de la v1.3
class PestReportsView(APIView):
    permission_classes = [IsAuthenticated, PestReaders]

    def get(self, request):
        dias = max(1, min(int(request.query_params.get("days", 30)), 90))
        # ... casos con ubicación y miniatura ...
        return Response({"reports": reports,
                        "farm": farm, "days": dias,
                        "serverTime": iso(timezone.now())})
```

La web: plantillas, HTMX y Leaflet

La web se arma en el servidor y se actualiza por partes, sin un framework pesado de JavaScript.

📁 Plantillas Django

Archivos HTML con «huecos» que Django rellena con datos antes de enviarlos al navegador.

⚡ HTMX

Pequeña librería que actualiza solo un pedazo de la página (la bandeja cada 15 s, la actividad cada 20 s) sin recargar todo.

🗺 Leaflet + Esri

Dibuja el mapa del fondo sobre **imagen satelital** con los casos de colores según su estado, y el editor de lotes, hileras y puntos (v1.3).

⚙ Detalles de uso

Sonidos, atajos de teclado (C, P, D, I), avisos emergentes y pantallas de carga, hechos con JavaScript propio, sin CDN.

</> Así se ve en nuestro código

```
<!-- partials/bandeja_tabla.html: se recarga sola -->
<div id="bandeja"
  hx-get="{{ request.get_full_path }}"
  hx-swap="outerHTML"
  hx-trigger="every 15s
    [document.visibilityState === 'visible']">

# sin una línea de JavaScript propio: HTMX pide la
# misma página cada 15 s (solo si la pestaña se ve)
# y reemplaza la tabla con el HTML nuevo
```

🔧 EXPLICACIÓN TÉCNICA

HTMX **2.0.11** y Leaflet **1.9.4** se copiaron a `web/static/web/vendor/` (sin CDN, regla W-01); el JavaScript propio está en `app.js`, `caso.js` y `mapa.js` (sin JavaScript en línea, W-17). Los estáticos los sirve WhiteNoise comprimidos y con huella (`CompressedManifest`); la fuente Inter, los iconos (sprite `iconos.svg`) y los sonidos también son locales. Sin React, Angular, Vue, Tailwind compilado ni npm.

🔗 HERRAMIENTA USADA Y POR QUÉ

⚡ **HTMX** — **Para qué:** actualizar partes de la página. **Por qué:** da sensación de app moderna con unas pocas etiquetas HTML, sin construir una SPA.

PostgreSQL y Supabase

PostgreSQL es la base de datos; Supabase es la empresa que nos la da administrada en la nube.

📦 PostgreSQL

Base de datos relacional: guarda la información en tablas que se relacionan entre sí (un lote tiene hileras, una hilera tiene segmentos...).

☁ Supabase

La aloja, hace copias y la mantiene. Usamos **solo la base de datos**: no usamos su API pública, Auth, Storage ni Realtime.

🌐 Región São Paulo

El proyecto del piloto (`riachuelo-piloto`) está en Sudamérica para responder rápido al servidor.

🔒 Conexión segura

Django se conecta por el «Session pooler», puerto 5432, con cifrado TLS (`DB_SSLMODE=require`).

☰ Qué usamos de Supabase

- ✔ PostgreSQL gestionado (copias, parches, monitoreo)
- ✔ Session pooler, puerto 5432, con TLS
- ✗ Data API (PostgREST) y GraphQL
- ✗ Supabase Auth: el login es de Django
- ✗ Storage: las fotos van a Cloudinary
- ✗ Realtime: la web usa sondeo HTMX
- ✗ Queues: la cola es la tabla `ai_tasks`
- ✗ Claves `anon` y `service_role`

🔧 EXPLICACIÓN TÉCNICA

El transaction pooler (6543) no se usa porque no admite sentencias preparadas. La cadena tiene la forma `postgresql://postgres.REF:CLAVE@aws-0-sa-east-1.pooler.supabase.com:5432/postgres` y la contraseña lleva solo letras y números (24+). Las claves `anon` y `service_role` de Supabase no se usan en ningún lado. Plan Free: 500 MB (alcanza para unas 300 000 fotos de datos); el pase a Pro está pendiente.

🔗 HERRAMIENTA USADA Y POR QUÉ

📖 **Supabase (solo PostgreSQL)** — **Para qué:** tener la base en la nube sin administrarla. **Por qué:** PostgreSQL real, gratis para el piloto y con panel para revisar; se puede migrar a otro PostgreSQL.

Cloudinary

Servicio especializado en guardar y entregar imágenes.

Por qué no guardar fotos en la base

Las fotos pesan de 3 a 4 MB. La base solo guarda su identificador; el archivo vive en Cloudinary.

Fotos privadas

Se suben como «authenticated»: solo se ven con un enlace firmado que genera Django al mostrar la pantalla.

Tamaños automáticos

Cloudinary genera la miniatura (400 px) y la versión de revisión (1600 px) y las sirve rápido por CDN.

Orden

Cada foto se guarda en `riachuelo/piloto/sesión/pasada/captura` ; el nombre es su `captureId`, así no se duplica.

</> Así se ve en nuestro código

```
# así queda una foto en Cloudinary
public_id  riachuelo/piloto/<sesión>/
           <pasada>/7f3c2a9e-... ← captureId
type       authenticated ← sin firma no se ve
overwrite  false         ← no se pisa

# forma de la URL firmada que arma Django:
res.cloudinary.com/wd9meyw0/image/
  authenticated/s--FIRMA--/
t_miniatura
/
v<versión>/riachuelo/piloto/...
```

EXPLICACIÓN TÉCNICA

Transformaciones con nombre: `miniatura = c_limit,w_400,q_auto` y `revision = c_limit,w_1600,q_auto` ; la original solo cuando el especialista pulsa «Original» (W-10). La base guarda `cloudinary_public_id` , versión, bytes, formato y tamaño, nunca una URL. Plan Free: 25 créditos al mes (unas 1 000 fotos ≈ 10 créditos) y máximo 10 MB por foto. Recomendado: «Strict transformations» activado y sin presets de subida sin firma.

HERRAMIENTA USADA Y POR QUÉ

 **Cloudinary** — **Para qué:** guardar y entregar las fotos. **Por qué:** resuelve CDN, tamaños y privacidad sin programarlo; el VPS no carga con los archivos.

YOLO y ONNX: la IA

YOLO es un modelo de visión por computadora que encuentra objetos en una imagen.

📦 Qué devuelve

Para cada foto: cajas alrededor de lo sospechoso, la clase (qué cree que es) y su confianza (0 a 100 %).

📄 ONNX

Formato estándar para ejecutar el modelo entrenado en el servidor, sin GPU y sin instalar todo el entorno de entrenamiento.

⚙️ Umbrales

Solo se dibuja una caja si la confianza supera 0,25; un umbral aparte (apagado) permite que la IA confirme sola.

✅ Estado actual

Activo en el servidor: **yolo-chanchito-melaza 2026.10-int**, un modelo de integración entrenado con fotos de internet.

</> Así se ve en nuestro código

```
# /gestion/ → Modelos de IA (model_configs)
name          yolo-chanchito-melaza
version       2026.10-int
weights_uri   modelos/modelo.onnx  (9,8 MB)
classes       ["chanchito_blanco",
               "melaza_fumagina"]

conf_threshold 0.25
iou_threshold  0.45
imgsz          640
tiling         {"tile": 1280, "overlap": 0.2}
auto_confirm_threshold vacío (apagado)
active
true
← solo uno a la vez
```

🔧 **Pendiente:** el modelo «de verdad» (v1) se entrena con fotos propias del fundo: captura del 10 al 19 de octubre, etiquetado hasta el 25 y activación el 2 de noviembre si detecta al menos 85 % de las fotos con plaga.

🔧 EXPLICACIÓN TÉCNICA

Detectores en `ia/inference/` : `simulado` (solo desarrollo), `onnx` (el del piloto: ONNX Runtime en CPU, letterbox a 640, NMS y tiling) y `yolo` (Ultralytics directo, no instalado por su licencia AGPL-3.0).
`IA_DETECTOR=onnx` y `MODEL_PATH=modelos/modelo.onnx` ; el archivo vive en `/srv/riachuelo/deploy/modelos/` (montado de solo lectura) y nunca va a Git.

🔧 HERRAMIENTA USADA Y POR QUÉ

🔧 **onnxruntime + numpy** — **Para qué:** ejecutar el modelo en el VPS.
Por qué: liviano, sin GPU y sin la licencia AGPL de Ultralytics en el servidor.

WhatsApp Cloud API

La forma oficial de Meta para que un sistema envíe mensajes de WhatsApp.

🔔 Cuándo se envía

Cuando el caso queda **Confirmado por especialista** o **Confirmado por IA** (solo si el modelo tiene umbral). Nunca por una simple sugerencia.

👤 A quién

A los destinatarios del lote (jefe de fundo, supervisor de zona u otro) que dieron su consentimiento.

📄 Plantilla aprobada

Meta exige una plantilla aprobada (caso_confirmado) con lote, hilera, lateral, ubicación, fecha y enlace al caso.

⌛ Estado actual

Hoy el mensaje se escribe en el registro del worker (modo consola). Falta el token, el número y la plantilla.

📄 Así se ve en nuestro código

```
# deploy/.env.piloto (sin valores secretos)
WHATSAPP_CLIENT=notificaciones.whatsapp.
    ConsoleClient ← hoy
#
    CloudApiClient ← envío real
WHATSAPP_TOKEN=(vacío)
WHATSAPP_PHONE_ID=(vacío)
WHATSAPP_GRAPH_VERSION=(vacío)
WHATSAPP_TEMPLATE=caso_confirmado
WHATSAPP_TEMPLATE_LANG=es
WHATSAPP_TEMPLATE_IA=(opcional, v1.3)
PUBLIC_BASE_URL=https://monitoreo.
    agricolariachuelo.org
```

🔧 EXPLICACIÓN TÉCNICA

Dos clientes intercambiables en notificaciones/whatsapp.py : ConsoleClient (escribe en la consola) y CloudApiClient (llama a la Graph API de Meta con WHATSAPP_TOKEN , WHATSAPP_PHONE_ID y WHATSAPP_GRAPH_VERSION). Se elige con WHATSAPP_CLIENT . Cada aviso es una fila de notifications (una por decisión y destinatario) con estado, intentos y error; la pantalla Avisos de la web los muestra.

🔗 HERRAMIENTA USADA Y POR QUÉ

🔗 WhatsApp Cloud API — Para qué: avisar al instante. Por qué: oficial (no se arriesga el número), con plantillas aprobadas y estado de entrega.

Docker, Traefik y gunicorn

Las herramientas que mantienen el backend funcionando en el servidor.

 **Docker**

Empaqueta la plataforma en una «imagen». De ella salen 2 contenedores: **web** y **worker**.

 **gunicorn**

El programa que atiende las visitas a Django dentro del contenedor web: 2 procesos con 4 hilos cada uno.

 **Traefik**

El portero del VPS: recibe el tráfico en los puertos 80/443, pone el HTTPS y lo pasa al contenedor web.

 **Límites**

Web: máx. 700 MB y 0,8 CPU.
Worker: máx. 1,2 GB y 0,7 CPU.
Nunca acaparan el servidor.

</> Así se ve en nuestro código

```
# deploy/docker-compose.yml (extracto)
x-comun: &comun
  image: riachuelo-plataforma:latest
  env_file: [.env.piloto]
  read_only:
    true

  cap_drop: [ALL]
  volumes: [./modelos:/app/modelos:ro]
services:
  web:    { mem_limit: 700m, cpus: 0.8 }
  worker: { mem_limit: 1200m, cpus: 0.7,
            command: [python, manage.py,
                      worker_ia] }
```

 **EXPLICACIÓN TÉCNICA**

Una sola imagen (`Dockerfile`) para los dos servicios: instala `requirements.txt` y, con `INSTALAR_IA=1` , `requirements-ia.txt` ; ejecuta `collectstatic` sin secretos y corre como usuario `riachuelo` (uid 10001). El web arranca con `migrate` y luego `gunicorn ... --workers 2 --threads 4 --timeout 60` ; el worker espera a que el web esté «healthy». Traefik ya existía en el VPS: la plataforma solo se conecta a él con la red `riachuelo-proxy` y etiquetas, sin modificarlo.

 **HERRAMIENTA USADA Y POR QUÉ**

 **Traefik** — **Para qué:** HTTPS y enrutamiento. **Por qué:** ya estaba instalado; emite y renueva certificados solo con etiquetas en el contenedor.

React Native y Expo: la app

Tecnología para hacer apps de celular con un solo código para Android e iPhone.

**React Native**
Se programa en TypeScript (JavaScript con tipos) y se convierte en una app nativa.

**Expo**
Herramientas que simplifican cámara, GPS, almacenamiento seguro y la generación del instalador (APK).

**SQLite**
Una base de datos pequeña dentro del celular, para trabajar sin internet.

**Red local**
Los 3 celulares se hablan por el Wi-Fi de la mochila: WebSocket para órdenes y HTTP para las fotos.

**WebView + Leaflet** v0.5.0
Una «ventanita web» dentro de la app dibuja el mapa satelital de «Ubicar plaga».

**EAS Build**
El servicio de Expo que compila el APK en la nube, sin instalar Android Studio.

</> Así se ve en nuestro código

```
// app.json (extracto)
"name": "Riachuelo Monitoreo",
"version": "0.5.0",
"android": {
  "package": "pe.riachuelo.monitoreo",
  "versionCode": 10 },
"ios": { "buildNumber": "10" }

// cada versión sube versionCode: los 3
// celulares deben tener la misma (RN-17)
```

**EXPLICACIÓN TÉCNICA**
Expo SDK 57 · React Native 0.86 (nueva arquitectura) · React 19.2 · TypeScript estricto · Expo Router (navegación por carpetas). `react-native-tcp-socket` 6.4.3 levanta el servidor del controlador (no existe en Expo Go: se carga con `require()` diferido). `react-native-webview` 13.16.1 es nativo: por eso la v0.5.0 exige un APK nuevo. Paquete Android `pe.riachuelo.monitoreo` ; cuenta EAS `oscar2020` .

**HERRAMIENTA USADA Y POR QUÉ**
 **React Native + Expo** — **Para qué:** una sola app para Android (y luego iPhone). **Por qué:** el mismo código sirve para los dos sistemas y Expo resuelve los permisos nativos.

Mapas: Leaflet, Esri y Google Maps

Herramientas nuevas de la v1.3 y la v0.5.0 para ver el fondo y llegar a cada alerta.

Herramienta	Dónde se usa	Para qué	Por qué / condición
Leaflet 1.9.4	Web (<code>mapa.js</code>) y app (dentro de un WebView)	Dibujar el mapa, capas, marcadores y herramientas	Libre y liviano; la misma copia en web y app; sin CDN
Esri World Imagery	Fondo satelital de la web y la app	Ver el fondo «como Google Earth»	Sin clave; exige mostrar la atribución
Esri World Street Map · OpenStreetMap	Fondo alternativo (calles)	Ubicarse por caminos y nombres	Gratis con atribución; OSM con su política de uso
Google Maps (URL universal)	Botón «Cómo llegar» de la app y del caso web	Ruta a pie hasta la alerta	No necesita SDK, clave ni facturación
GeoJSON	Contornos de lotes, hileras y puntos	Guardar y enviar figuras del mapa	Estándar; lo leen Leaflet y PostgreSQL (JSON)

 **Importante:** las imágenes satelitales necesitan internet. Sin datos, la app sigue mostrando la lista, los contornos y la distancia; Google Maps solo guía sin datos si el mapa de la zona se descargó antes.

 **EXPLICACIÓN TÉCNICA**
Se descartó `expo-maps` /Google Maps SDK dentro de la app: exige un proyecto de Google Cloud, una clave restringida al paquete y a la firma del APK, y facturación activa. Con Leaflet embebido como texto (`src/pests/vendor/leaflet.ts`) la app no depende de un CDN y el APK solo crece ~0,2 MB. En la web, el editor del mapa guarda con `POST /mapa/editar/` (JSON + CSRF).

 **HERRAMIENTA USADA Y POR QUÉ**
 **Leaflet** — **Para qué:** mapas interactivos. **Por qué:** la opción libre más usada; funciona igual en la web y en un WebView.

Las librerías de la app

Cada módulo instalado en la app y el trabajo que hace.

Librería	Para qué
expo-camera	tomar fotos (hasta 12 MP, JPEG 0,85)
expo-location	GPS del controlador y de «Ubicar plaga»
expo-sqlite	base local con transacciones
expo-file-system	guardar fotos y subirlas a Cloudinary
expo-secure-store	tokens y verificador sin internet
expo-crypto	UUID y huellas (md5/PBKDF2)
expo-sensors	saber si el celular está quieto (AUTOMÁTICO)
expo-network	¿hay internet? ¿es Wi-Fi?
expo-battery · expo-keep-awake	aviso de batería y pantalla encendida
expo-audio · expo-haptics	sonidos y vibración de confirmación

Librería	Para qué
react-native-tcp-socket	servidor TCP/HTTP del controlador (8765/8766)
react-native-webview nuevo	mapa satelital con Leaflet embebido
expo-router	navegación: cada archivo de app/ es una pantalla
zustand	estado compartido de la app (sesión, sincronización)
zod	validar mensajes del protocolo y la API
expo-image · image-manipulator · jpeg-js	ver fotos y medir la calidad
qrcode-generator · react-native-svg	dibujar el QR de vinculación
expo-sharing	compartir el diagnóstico
jest · TypeScript	139 pruebas y tipos estrictos

 **EXPLICACIÓN TÉCNICA**

Todas se instalan con `npx expo install`, que elige la versión compatible con el SDK 57 (R-01). Antes de sumar una librería se justifica (R-20) y se prefiere un módulo oficial de Expo. Prohibido instalar SDK de Supabase o Cloudinary en la app (R-21): la subida es un multipart normal con los campos del ticket.

 **HERRAMIENTA USADA Y POR QUÉ**

 **npx expo install** — **Para qué:** agregar módulos. **Por qué:** evita versiones incompatibles que rompen el build nativo.

Las librerías del servidor

Cada paquete de Python que usa la plataforma, con su versión fija.

Paquete	Para qué
Django 5.2.17	framework web (LTS)
djangoestframework 3.18.1	API REST de la app
djangoestframework-simplejwt 5.5.1	tokens JWT con rotación y lista negra
drf-spectacular 0.30.0	esquema OpenAPI en /api/v1/schema
django-htmx 1.29.0	detectar pedidos HTMX en las vistas
whitenoise 6.12.0	servir CSS, JS e íconos comprimidos
cloudinary 1.46.2	firmar tickets y URLs, descargar fotos

Paquete	Para qué
psycopg[binary] 3.3.6	conectar con PostgreSQL
gunicorn 26.2.0	servidor de producción (Linux)
django-environ 0.14.0	leer el archivo .env
requests 2.34.2	llamar a WhatsApp y descargar
Pillow 12.3.0	orientación EXIF y tamaño de la foto
numpy ≥ 2.0 · onnxruntime ≥ 1.20	ejecutar YOLO (requirements-ia)

 **Dato clave:** las versiones están fijas en requirements.txt : el VPS instala exactamente lo mismo que se probó en la PC. Ultralytics no se instala en el servidor (licencia AGPL-3.0): solo se usa para entrenar.

 **EXPLICACIÓN TÉCNICA**

Regla W-21: cada dependencia nueva se justifica en docs/REPORTE_AVANCE.md . La imagen Docker instala requirements.txt siempre y requirements-ia.txt con INSTALAR_IA=1 (por defecto en el piloto). En la PC, iniciar.bat -IA instala onnxruntime y numpy solo si se quiere probar el detector real.

Las herramientas del día a día

Lo que usa el equipo para programar, probar, entrenar y operar el sistema.

Herramienta	Para qué la usa el equipo	Por qué
VS Code	Editar código, ver diferencias, resolver conflictos de Git	Gratis, con terminal y Git integrados
Git + GitHub	Guardar historial y compartir los dos repositorios	El VPS se actualiza con <code>git pull</code>
iniciar.bat (PowerShell)	Levantar la plataforma en la PC con datos demo	Un doble clic crea entorno, base y worker
Expo Go · build de desarrollo	Probar la app en el celular mientras se programa	Sin compilar un APK cada vez
EAS Build (Expo)	Compilar el APK del piloto en la nube	No hace falta Android Studio
Google Colab (GPU T4)	Entrenar YOLO	GPU gratis por sesiones; datos en Google Drive
Roboflow	Conseguir y revisar datos de entrenamiento	Datasets públicos de chanchito (Mealybug)
Paneles de Supabase, Cloudinary y Hostinger	Revisar base, fotos, DNS y VPS	Lo que no se toca desde el código
Asistentes de IA (con AGENTS.md)	Programar y documentar siguiendo las reglas	Cada repo trae sus reglas para agentes

 **EXPLICACIÓN TÉCNICA**

Ninguna herramienta del día a día guarda secretos del piloto: las claves se pegan solo en `/srv/riachuelo/deploy/.env.piloto` (VPS) o en el `.env` local, y nunca en un chat, un commit o una captura. Los agentes de IA leen `AGENTS.md` y los maestros antes de tocar código y deben correr las pruebas antes de entregar.

6



App móvil

Qué hace la app, cómo trabaja en el campo, cómo guarda y envía los datos, y la nueva pantalla «Ubicar plaga».

Cómo trabaja la mochila

Tres celulares cooperan durante el recorrido de cada hilera.

**Controlador**

Lo lleva el operador en la mano. Inicia la sesión y la pasada, marca los marcadores, guarda el GPS y ordena cuándo disparar.

**Cámara 1 y Cámara 2**

Van en el soporte de hombros, mirando el mismo lado de la hilera. Toman la foto cuando el controlador lo ordena.

**Vinculación por QR**

Las cámaras se unen al controlador escaneando un código QR (IP, puertos y un código de un solo uso); luego se hablan por el Wi-Fi local.

**Calidad técnica**

Cada foto se revisa (desenfoque, luz). Si sale mal, se repite en el momento solo esa cámara.



**EXPLICACIÓN TÉCNICA**

La función del celular (`CONTROLADOR` , `CAMERA_1` , `CAMERA_2`) es del celular, no del usuario, y se elige después de cada login (ADR 0005). En una sesión cada rol de cámara lo ocupa un solo celular; uno que se reconecta conserva su rol, y el operador puede liberar el rol de una cámara caída para reemplazarla (RN-16). En el piloto las cámaras deben tener la misma versión de app que el controlador (RN-17).

**HERRAMIENTA USADA Y POR QUÉ**

 **Código QR** — **Para qué:** emparejar sin escribir IP ni contraseñas.
Por qué: es rápido con guantes y al sol; el código de emparejamiento es de un solo uso.

La red local de la mochila

El controlador hace de servidor: un canal para órdenes y otro para las fotos, sin internet.



- 💖 **Latido cada 2 s**

Si no llega en 3 s la cámara está «inestable»; a los 6 s, «perdida», y se reconecta sola (1, 2, 4, 8 s).
- 🕒 **Orden con respuesta**

Cada cámara responde CAPTURE_OK, QUALITY_ERROR o ERROR en 8 s; si no, la secuencia queda incompleta.
- 📶 **El router no debe aislar clientes**

Los 3 celulares deben verse entre sí en la misma red.

🔧 **EXPLICACIÓN TÉCNICA**

Protocolo local **v1** con mensajes validados por zod (`src/protocol/`), máximo 64 KB por mensaje; cada mensaje se procesa una sola vez (`processed_messages` , 24 h). La foto viaja con `POST /local/captures` : JPEG binario, datos en la cabecera `X-Capture-Meta` y verificación md5; una foto a la vez por cámara, espera de 30 s y reintentos a 2, 4, 8 y 16 s. Medido en un Galaxy A03: el controlador recibe unos 0,75 MB/s en total.

🔧 **HERRAMIENTA USADA Y POR QUÉ**

🔗 **react-native-tcp-socket** — **Para qué:** que el controlador sea un pequeño servidor. **Por qué:** Expo no trae servidores TCP; esta librería sí, y funciona en Android e iPhone.

Una jornada de monitoreo (1/2): preparar

Antes de caminar: red, ingreso, función, catálogos, sesión, vinculación y prueba.

1



Misma red e ingreso

Los 3 celulares en el mismo Wi-Fi. Cada uno inicia sesión (el primer ingreso de cada celular necesita internet).

2



Elegir función

Controlador, Cámara 1 o Cámara 2 (PANT-08) y dar permisos: ubicación al controlador, cámara a las cámaras.

3



Catálogos → Actualizar

El controlador baja lotes, hileras, segmentos y marcadores del servidor.

4



Nueva sesión

Modo MANUAL o AUTOMÁTICO (intervalo de 1, 2, 3 o 5 s).

5



Vincular cámaras

El controlador muestra un QR; cada cámara lo escanea y queda como Cámara 1 o 2.

6



Prueba corta

Cada cámara toma una foto, la evalúa y la envía; aprueba si todo llega en menos de 20 s.

 **Dato clave:** sin prueba corta aprobada no se puede iniciar la primera pasada: así se descubre antes de caminar si una cámara, la red o la calidad fallan (RN-10).

 **EXPLICACIÓN TÉCNICA**

Las fotos de la prueba corta no son evidencia: no pertenecen a una pasada, nunca se sincronizan y se borran al cerrar la sesión (RN-21). Para iniciar una pasada además se exige: ambas cámaras `CONECTADA`, lote, hilera, lado y marcador definidos, y espacio libre ≥ 2 GB en los tres celulares (con menos de 3 GB avisa; con menos de 1 GB pausa). La batería solo avisa bajo 15 % (CFG-7): conectar el power bank.

 **HERRAMIENTA USADA Y POR QUÉ**

 **GET /mobile/bootstrap** — **Para qué:** bajar los catálogos. **Por qué:** trae `catalogVersion` : la app sabe si su copia está al día y avisa si tiene más de 72 h.

Una jornada de monitoreo (2/2): recorrer y cerrar

En la hilera: pasada, capturas, repeticiones, cierre y sincronización.

7

Nueva pasada

Lote → hilera → lado (A o B) → segmento → marcador de inicio.

8

Capturar

Con el botón o en automático. Cada orden crea una secuencia con el GPS del controlador y una foto por cámara.

9

Repetir y marcar

Se repite solo la foto rechazada; el marcador se cambia (en automático, en pausa).

10

Terminar la pasada

Luego el lado B, en sentido contrario, u otra hilera.

11

Cerrar la sesión

PANT-19. Cada celular vuelve a la pantalla de elegir función.

12

Sincronizar

Con internet: «Sincronización» (PANT-20) o automática con Wi-Fi.

Importante: el controlador debe quedarse con la app en pantalla durante la pasada: si pasa a segundo plano, Android la suspende y las cámaras se desconectan cada ~10 s (regla R-19: todo en primer plano).

EXPLICACIÓN TÉCNICA

Reglas de la pasada: solo una abierta por sesión (READY, ACTIVE o PAUSED); el lado B exige el lado A cerrado de esa hilera en la misma sesión; «Repetir lateral» crea otra pasada con incidencia obligatoria (RN-12). Una hilera queda **cubierta** con los dos lados cerrados (RN-13). Desde la v0.4.5 se puede cerrar la sesión de usuario con un monitoreo abierto: la pasada se pausa y todo queda guardado (ADR 0008).

HERRAMIENTA USADA Y POR QUÉ

expo-keep-awake — **Para qué:** que la pantalla no se apague. **Por qué:** la captura y la coordinación ocurren con la app al frente y la pantalla encendida.

Captura y calidad de la foto

La calidad se revisa en el mismo celular, al momento, para no volver al campo por una foto mala.

 **Tamaño de la foto**
La mayor resolución disponible de hasta **12 MP** (4:3). Sin este tope Android usaba 50 MP o más.

 **JPEG calidad 0,85**
Con calidad 1, una foto de 12 MP pesaba ~9 MB (límite de Cloudinary Free: 10 MB). Nunca se recomprime después (RN-24).

 **Sin GPS dentro**
Se toma con `exif: false` : la ubicación va en la secuencia del controlador.

Resultado	Qué significa	Qué pasa
UTILIZABLE	Técnicamente útil: va a la IA	No significa «sin plaga» (RN-14)
REPETIR_EXPOSICION	Muy oscura o quemada	Se repite solo esa cámara
REPETIR_NITIDEZ	Movida o desenfocada	Se repite solo esa cámara
PENDIENTE_REVISION_TECNICA	No se pudo decidir a tiempo	Se acepta y también va a la IA

 **EXPLICACIÓN TÉCNICA**
Perfil de calidad **Q0**, calculado en el celular en ≤ 2 s: **exposición** sobre una copia de 256 px (media entre 50 y 205, como máximo 40 % de zonas oscuras y 20 % saturadas) y **nitidez** con la varianza del Laplaciano en 3 recortes de 512 px (mínimo 100). El perfil se publica desde el servidor (`quality_profiles`) y baja con el bootstrap.

 **HERRAMIENTA USADA Y POR QUÉ**
 **expo-image-manipulator + jpeg-js** — **Para qué:** medir luz y nitidez en el celular. **Por qué:** permite decidir al instante, sin internet, si la foto sirve o hay que repetirla.

Modos, marcadores y repeticiones

Cómo se toma cada foto y qué hacer cuando algo sale mal en la hilera.

 **MANUAL**

El operador toca «Capturar» en cada punto que quiere fotografiar.

 **AUTOMÁTICO**

Dispara solo cada 1, 2, 3 o 5 s (por defecto 2 s), y solo cuando el celular está quieto.

 **Marcadores**

Señalan el avance en la hilera (inicio, intermedios y fin). Se cambian a mano; en AUTOMÁTICO, en pausa.

 **Repetir una foto**

Solo la cámara afectada, con un captureId nuevo que reemplaza al anterior; la otra foto válida se conserva.

 **Repetir un lado**

«Repetir lateral» crea otra pasada del mismo lado con una incidencia que explica el motivo.

 **Incidencias**

Batería, GPS, desconexión, calidad, transferencia u otra: quedan registradas y viajan al servidor.

 **EXPLICACIÓN TÉCNICA**

Estabilidad (AUTOMÁTICO): ventana de 600 ms leyendo acelerómetro y giroscopio cada 50 ms; dispara si el giro es $\leq 0,12$ rad/s y la variación de aceleración $\leq 0,025$ g durante 350 ms, con espera máxima de 1,2 s. Una repetición lleva `replacesCaptureId` y el contexto vigente al repetir (`retakeContext` : segmento, marcador y GPS), nunca el anterior (RN-07). Tipos de incidencia: CALIDAD, DESCONEXION, BATERIA, TEMPERATURA, ESPACIO, GPS, TRANSFERENCIA, SINCRONIZACION, SOPORTE, OPERADOR y OTRO.

 **HERRAMIENTA USADA Y POR QUÉ**

 **expo-sensors** — **Para qué:** saber si el celular está quieto. **Por qué:** evita fotos movidas en el modo automático sin que el operador se detenga cada vez.

Quién puede hacer qué en la app

La app la usan tres roles; desde la v0.5.0 el especialista entra solo para ubicar plagas.

Rol	¿Entra a la app?	¿Elige función?	¿Monitorea y sincroniza?	¿Ubicar plaga?
Operador de campo	Sí	Sí	Sí	Sí · desde Controlador y Ajustes
Administrador	Sí	Sí	Sí	Sí · desde Controlador y Ajustes
Especialista fitosanitario v0.5.0	Sí	No (va directo a «Ubicar plaga»)	No (403 ROLE_NOT_ALLOWED)	Sí · es su pantalla principal
Supervisor	No (solo web)	—	—	—

 **Funciones del celular (PANT-08)**

Controlador, Cámara 1 o Cámara 2. Se elige después de **cada** login; no se puede cambiar con una sesión de monitoreo abierta ni con envíos pendientes (RN-15).

 **El especialista en la app**

Ve las alertas para ir al lugar, con Ajustes y Cerrar sesión arriba. Los datos de monitoreo que haya en ese celular quedan guardados para el operador.

 **EXPLICACIÓN TÉCNICA**

`MOBILE_ALLOWED_ROLES` incluye operador, administrador y especialista (login con y sin internet); `FIELD_ROLES` solo operador y administrador. La función `canDoFieldWork()` (`src/domain/types.ts`) protege las pantallas `(setup)` , `gallery` , `controller` y `camera` , la vinculación (`USER_NOT_ALLOWED`) y la sincronización. En el servidor, las rutas de campo exigen el permiso `FieldWork` .

Estado de la app por fases

La app se construyó por fases. Las fases 0 a 4 funcionan; falta el iPhone y la prueba completa en campo.

Fase	Contenido	Estado
0	Base del proyecto, configuración, estructura por capas, pruebas	Listo
1	Login con y sin internet, registro, contraseñas, función del celular, permisos	Listo
2	Vinculación por QR, protocolo local (WebSocket + HTTP), prueba corta, simulador de 2 cámaras	Listo probada con 3 Android
3	Sesión, pasadas, marcadores, captura MANUAL/AUTOMÁTICO, calidad, repetición, cierre	Listo en prueba de campo
4	Sincronización con la plataforma y subida directa a Cloudinary (ticket v2.0)	Listo 29/29 contra Django local; falta confirmarla en el piloto
5	iPhone	Pendiente
6	Campo y versión congelada del piloto	En curso
Nueva	«Ubicar plaga»: mapa satelital, distancia y «Cómo llegar» (v0.5.0)	Listo falta el APK

 **Importante:** la app ya no usa el backend simulado: desde la v0.4.0 apunta al piloto real (`EXP0_PUBLIC_USE MOCK_API=0`). Las cuentas `@demo.pe` solo existen en el simulado y en la demo local de la plataforma.

 **EXPLICACIÓN TÉCNICA**

Pruebas: `npm run validate` = typecheck + lint + jest (139 pruebas). En `tools/verificacion/` hay pruebas que ejecutan el código real de `src/` en Node: `simulado.ts` (9/9), `redlocal.ts` (red local real, 9/9), `actualizacion.ts` (migraciones con datos), `integracion.ts` (contra la plataforma en la laptop, 30/30 con la v1.3) y `piloto.ts` (prueba corta contra el piloto, pide `PERMITIR_PILOT0=1`).

Historial de versiones

Cada versión nació de algo que se vio en el campo.

Versión	Problema visto en campo	Cambio
0.4.0	—	Fase 4: sincronización y ticket directo a Cloudinary
0.4.1	Catálogos «sin descargar» y error al mostrar el QR	Distingue servidor vacío de no descargado; arreglo de carga del módulo TCP
0.4.2	Prueba corta reprobada en 0 s (ERROR_CAMARA)	La cámara dejaba de estar «lista» al redibujar; referencia estable
0.4.3	Prueba corta de 18,5 s y cámaras «sin conexión»	Fotos ≤ 12 MP, latido fuera de la cola, motivo visible, límite 20 s
0.4.4	Fotos de 9 MB; una foto llegaba dos veces	Cierre ordenado de la conexión; JPEG 85 (CFG-6)
0.4.5	Sin segmentos ni marcadores; no se podía salir con monitoreo abierto	Segmento «hilera completa»; ADR 0008
0.4.6	Pedido del equipo	Batería: solo aviso bajo 15 % (CFG-7)
0.5.0	Pedido del equipo: ir hasta la plaga	«Ubicar plaga», especialista en la app, CFG-8, WebView

 **EXPLICACIÓN TÉCNICA**

Cada APK sube `version` , `versionCode` y `buildNumber` en `app.json` (0.5.0 = versionCode 10). Lo aprendido en campo el 07/10 quedó en `docs/CAMBIOS.md` . Valores medidos: con JPEG 100 una foto de 12 MP pesaba 8,9 MB; GPS en interiores de 16 a 28 m de error; login sin internet en el Galaxy A03 ~9,7 s (pendiente calibrar).

 **HERRAMIENTA USADA Y POR QUÉ**

 `docs/CAMBIOS.md` — **Para qué:** registrar cada versión. **Por qué:** explica qué falló en campo y cómo se resolvió, para no repetir errores.

Cómo se conectó la app al piloto

Los pasos que en la versión anterior de este documento estaban pendientes, hoy ya están hechos.

1



Apuntar al servidor ✓

`EXP0_PUBLIC_API_URL=https://monitoreo.agricolariachuelo.org` y `EXP0_PUBLIC_USE MOCK_API=0` en el perfil «piloto» de `eas.json`.

2



Fase 4: sincronización ✓

Enviar sesiones, pasadas, secuencias, incidencias y fotos a `/api/v1`, con reintentos.

3



Ticket v2.0 ✓

La foto sube directo a Cloudinary con el permiso firmado; el multipart v1 queda solo de respaldo.

4



Generar el APK ✓

Expo copia esas variables al compilar: se crea un instalador y se instala en los 3 celulares.

⚠ **Pendiente:** compilar el APK 0.5.0 (incluye un módulo nativo nuevo) e instalarlo en los 3 celulares, y después poner `API_SUBIDA_MULTIPART=false` en el servidor, porque ninguna app usa ya la subida v1.

 **EXPLICACIÓN TÉCNICA**

Las variables `EXP0_PUBLIC_*` quedan dentro del APK y no son secretas (R-10): por eso la app nunca lleva claves. Perfiles de `eas.json`: `piloto` (servidor real), `preview` (backend simulado) y `development` (dev client). Comandos: `npm run build:apk:piloto`, `npm run build:apk` y `npm run build:apk:dev`.

 **HERRAMIENTA USADA Y POR QUÉ**

 **EAS Build** — **Para qué:** compilar el APK. **Por qué:** compila en la nube con la firma del proyecto y entrega un enlace o QR para descargarlo.

Cómo está organizada la app

Dos carpetas principales: las pantallas y la lógica.

agricola-riachuelo-mobile/	
└─ app/	pantallas (cada archivo = una ruta)
└─ │ (auth)/ (setup)/	login, registro, recuperar · función y permisos
└─ │ controller/	sesión, pasada, marcador, sincronizar
└─ │ camera/	escanear QR, cámara en sesión, prueba
└─ │ pests/	«Ubicar plaga»: mapa, lista, detalle (v0.5.0)
└─ │ └─ gallery · settings · index	fotos guardadas, ajustes, arranque
└─ src/	lógica, separada por temas
└─ │ api/ auth/ sync/	pedidos a la plataforma, sesión, sincronización
└─ │ controller/ camera/ device/	runtime de cada función y del celular
└─ │ local-network/ protocol/	mochila (Wi-Fi local) y mensajes
└─ │ storage/ domain/	SQLite y reglas puras
└─ │ pests/ ui/ config/	mapa embebido · pantallas · CFG-8
└─ tools/verificacion/	pruebas contra la plataforma real
└─ docs/adr/ · CAMBIOS.md	decisiones y versiones
└─ app.json · eas.json	nombre, permisos, versión y builds

Lo esencial

- `app/` usa **Expo Router**: la carpeta define la navegación.
- `src/api/` tiene los pedidos reales; `mock/` , el backend simulado.
- Solo `src/api/httpClient.ts` arma URLs y solo `cloudinaryUpload.ts` habla con Cloudinary.
- Parámetros en `src/config/default.ts` ; textos en `src/ui/strings.ts` y `messages.ts` .
- Cada archivo empieza con un comentario «QUÉ HACE».

 **EXPLICACIÓN TÉCNICA**

Capas (R-05): las pantallas no tocan SQLite, archivos ni red; llaman a servicios. Los módulos nativos que no existen en Expo Go se cargan con `require()` diferido e `isRunningInExpoGo()` (ver `src/local-network/factory.ts`). Nunca se editan `android/` ni `ios/` a mano: se generan desde `app.json` .

Las carpetas de src/, una por una

Dónde vive cada parte de la lógica de la app.

Carpeta de src/	Qué contiene
api/	Cliente HTTP, contrato (DTO), subida a Cloudinary, pestApi.ts y backend simulado (mock/)
auth/	Login con y sin internet, tokens, verificador PBKDF2, revocación
camera/	Agente de cámara: captura, cola de envío al controlador, retención de archivos
controller/	Runtime del controlador: sesiones, pasadas, secuencias, consolidación de fotos, prueba corta, catálogos, vinculación
device/	Cámara, GPS, batería, red, estabilidad, calidad, galería y permisos
diagnostics/	Registro de eventos (30 días), exportar diagnóstico, incidencias
domain/	Tipos, reglas puras, máquinas de estado, geo.ts (distancia y rumbo) y pests.ts
local-network/ · protocol/	Servidor TCP/WebSocket/HTTP propios, cliente, simulador · mensajes y esquemas zod
storage/	SQLite, migraciones 001–004, repositorios y archivos
sync/	Motor de sincronización: syncService , syncPlanner , syncPayloads , captureUploader , retry
pests/ v0.5.0	pestService (copia y actualización), mapHtml (página del mapa), openMaps (Google Maps), Leaflet embebido
ui/ · config/	Componentes, tema, textos, sonidos · parámetros CFG-8 y entorno

 **EXPLICACIÓN TÉCNICA**

syncPlanner.ts y syncPayloads.ts son funciones puras (fáciles de probar) que deciden el orden y arman los datos del contrato; captureUploader.ts y retry.ts son copias exactas del maestro (§15.7.3 y C.3). Todo lo que usa la pantalla «Ubicar plaga» pasa por pestService.ts , que guarda la última respuesta en app_meta para verla sin internet.

Las pantallas de la app

Todas las pantallas, agrupadas por para qué sirven.

Grupo	Pantallas (carpeta app/)
Sin sesión	PANT-02 login · PANT-03 registro · PANT-04 pendiente · PANT-05 recuperar · PANT-06 cambiar contraseña
Configuración	PANT-01 arranque · PANT-08 elegir función · PANT-07 permisos · PANT-09 Ajustes
Controlador	PANT-10 panel · 11 catálogos · 12 nueva sesión · 13 vincular (QR) · 14 prueba corta · 15 nueva pasada · 16 pasada activa · 17 marcador · 18 cerrar pasada · 19 cerrar sesión · 20 sincronización · 21 incidencias
Cámaras	PANT-30 escanear QR · PANT-31 cámara en sesión · PANT-32 modo prueba
Comunes	«Fotos guardadas» (galería) · Ajustes: exportar diagnóstico y cambiar función
«Ubicar plaga» v0.5.0	PANT-40 mapa o lista de alertas · PANT-41 detalle de una alerta

 **Dato clave:** cada pantalla tiene un código PANT-xx del maestro de la app: así las pruebas, los textos y los reportes de errores se refieren a la misma pantalla sin ambigüedad.

 **EXPLICACIÓN TÉCNICA**

Estructura de rutas de Expo Router: `(auth)` y `(setup)` son grupos (los paréntesis no aparecen en la ruta); `_layout.tsx` define la navegación y las guardas (por ejemplo, un especialista que intenta abrir `/controller` vuelve a `/pests`). `app/pests/[id].tsx` es una ruta con parámetro: el identificador del caso.

Los datos dentro del celular

Una base SQLite por celular, más archivos y un almacén seguro.

 **Monitoreo**
monitoring_sessions , session_devices ,
monitoring_passes , marker_changes ,
capture_sequences , monitoring_cycles ,
uncovered_areas

 **Fotos**
captures , quality_results ,
retake_requests , camera_context ,
transfer_queue

 **Envíos**
sync_queue , remote_uploads ,
processed_messages

 **Catálogos y cuenta**
cat_lots , cat_rows , cat_segments ,
cat_markers , users_cache , app_meta

 **Diagnóstico**
incidents , event_log (30 días)

 **Fuera de SQLite**
Tokens y verificador en **SecureStore**; fotos en
el almacenamiento privado de la app.

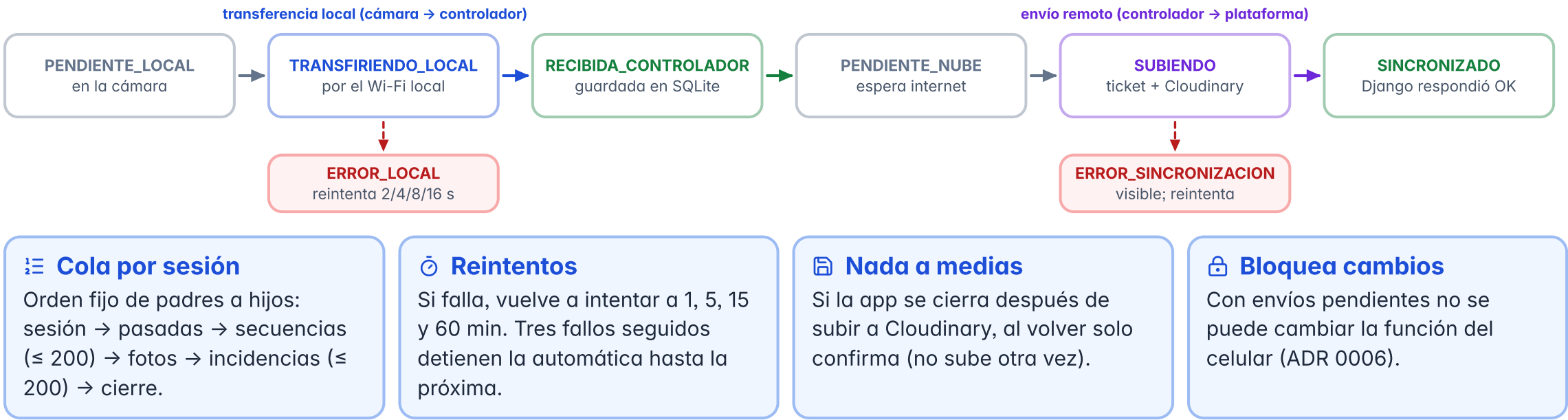
 **Importante:** nada se borra de una sesión abierta. Las cámaras borran una foto solo cuando el controlador confirma que ya la sincronizó; el controlador lo conserva todo y libera espacio solo a mano (RN-09).

 **EXPLICACIÓN TÉCNICA**
Migraciones SQLite 001_initial , 002_cobertura , 003_remote_uploads y 004_catalog_codes : nunca se modifica una publicada, se agrega la siguiente. «Ubicar plaga» no necesitó migración: guarda su copia en app_meta.pest_reports_json . Toda escritura de varias tablas usa withExclusiveTransactionAsync (R-14). Esquema SQLite versión 4.

 **HERRAMIENTA USADA Y POR QUÉ**
 **expo-secure-store** — **Para qué:** guardar tokens y el verificador.
Por qué: usa el almacén cifrado del sistema (Keystore/Keychain); nunca van a SQLite ni a los registros.

La sincronización por dentro

Los estados de una foto desde la cámara hasta la plataforma, y las reglas del motor.



Ingreso, tokens y uso sin internet

Cómo la app sabe quién eres, incluso en medio del campo.

➔ Primer ingreso: con internet

El servidor valida correo, contraseña, estado y rol, y registra el celular (X-Device-Id). La app guarda los tokens y un verificador para el modo sin internet.

📶 Después: sin internet

Solo el último usuario que entró con internet en ese celular, hasta 7 días, con 5 intentos y bloqueo de 15 min.

🔑 Contraseña temporal

Si el administrador la asignó, la app obliga a cambiarla (con internet) antes de usar nada más (RN-04).

🚫 Revocación

Si la cuenta se bloquea o el administrador revoca el celular, la app cierra la sesión en el siguiente pedido (403).

⚠ **Pendiente:** calibrar el verificador: en el Galaxy A03 el login sin internet tarda ~9,7 s (40 000 iteraciones de PBKDF2); el objetivo es ≤ 1,5 s.

🔧 EXPLICACIÓN TÉCNICA

Tokens SimpleJWT: access 15 min y refresh 14 días con rotación; la app renueva 60 s antes del vencimiento, una sola renovación a la vez, y usa la hora del servidor para no fallar si el reloj del celular está mal (avisa si difiere más de 2 min). Nunca se registran contraseñas, tokens ni el verificador (R-10). Registro desde la app: la cuenta queda `PENDIENTE_APROBACION` hasta que el administrador la aprueba en la web.

🔧 HERRAMIENTA USADA Y POR QUÉ

🔑 **PBKDF2 (expo-crypto)** — **Para qué:** entrar sin internet de forma segura. **Por qué:** el celular guarda una huella de la contraseña, no la contraseña.

«Ubicar plaga» (1/3): lo que ve el encargado

La nueva pantalla de la v0.5.0: todas las alertas del fundo, en un mapa satelital o en una lista.



📖 Mapa satelital o lista

Arriba se elige la vista. El mapa muestra lotes, hileras, puntos del fundo, las alertas y tu posición (punto azul).

🔍 Filtros

Todas, Confirmadas (IA o especialista) o Posibles (posible plaga y en revisión).

📍 Distancia y dirección

«35 m al noreste ↗» desde tu GPS. Bajo 15 m: «Estás en el lugar».

🎨 Mismo color que la web

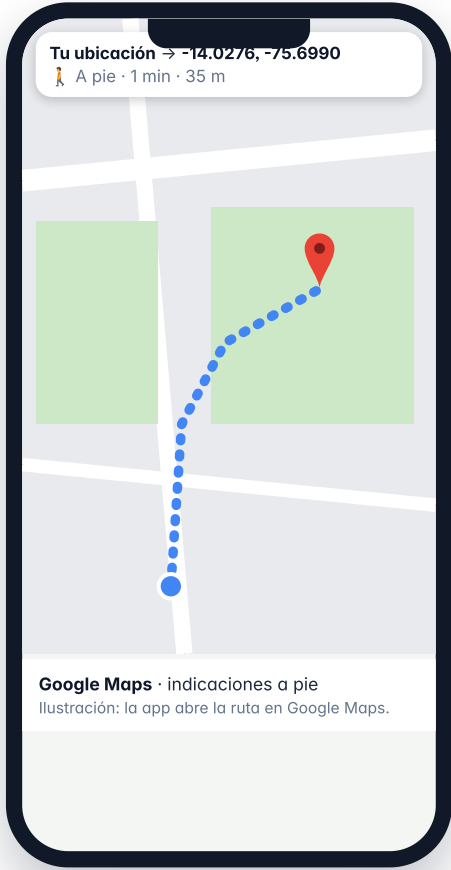
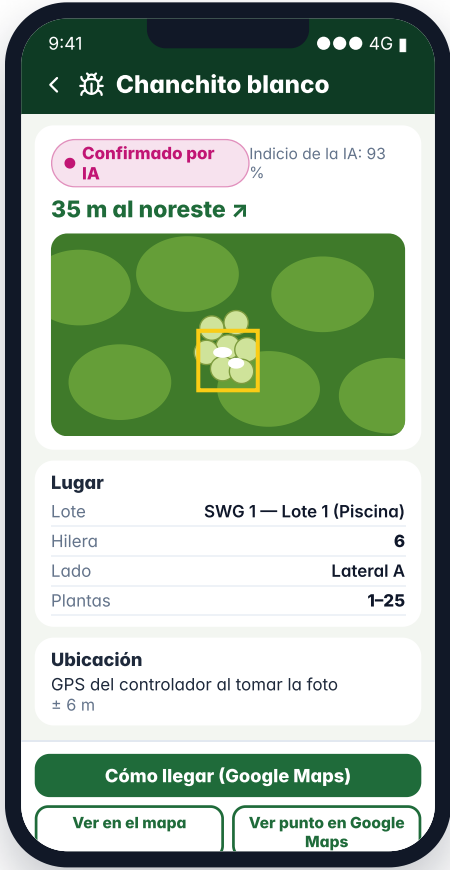
Fucsia: confirmado por IA · rojo: por especialista · naranja: posible · amarillo: en revisión.

📖 EXPLICACIÓN TÉCNICA

Ilustración de la pantalla PANT-40 (`app/pests/index.tsx`) con datos de ejemplo. Orden de la lista: primero lo confirmado; dentro de cada grupo, lo más cerca. Se actualiza al abrir, cada 2 min con internet y con «Actualizar»; sin internet muestra la última copia guardada con su hora.

«Ubicar plaga» (2/3): el detalle y «Cómo llegar»

Todo lo necesario para encontrar la planta, y un botón para que Google Maps guíe a pie.



-  **Foto del caso**
Miniatura firmada por la plataforma (se ve con internet) y el indicio de la IA en %.
-  **Lugar en palabras**
Lote, hilera, lado, plantas y marcador: sirve aunque el GPS sea impreciso.
-  **De dónde sale la ubicación**
GPS del controlador con su precisión, o «aproximada» (marcador de la hilera).
-  **Cómo llegar**
Abre Google Maps con la ruta a pie. También: «Ver en el mapa» y «Ver punto en Google Maps».

 **EXPLICACIÓN TÉCNICA**

PANT-41 (`app/pests/[id].tsx`). «Cómo llegar» abre `https://www.google.com/maps/dir/?api=1&destination=lat,lon&travelmode=walking` con `Linking.openURL` : sin SDK ni clave.

Si la alerta ya no está en la lista (por ejemplo, el especialista la descartó), la pantalla lo dice y ofrece volver.

«Ubicar plaga» (3/3): por dentro

Las piezas de código de la nueva pantalla y sus parámetros.



Parámetro CFG-8	Valor	Para qué
pests.windowDays	30 días (1 a 90)	cuántos días hacia atrás se piden
pests.autoRefreshMs	2 min (≥ 30 s)	cada cuánto se actualiza con internet
pests.requestTimeoutMs	30 s	espera máxima del pedido
pests.locationIntervalMs	2 s	cada cuánto se lee tu GPS
pests.arrivedRadiusM	15 m (3 a 50, calibrar)	debajo de esto: «Estás en el lugar»

 **EXPLICACIÓN TÉCNICA**

El mapa es una página HTML armada por `mapHtml.ts` con Leaflet **1.9.4** como texto (`src/pests/vendor/leaflet.ts` , la misma copia que la web; sin CDN). La app le manda los datos con `injectJavaScript` y la página avisa los toques con `postMessage` . Un 403 de cuenta o celular revoca la sesión como en el resto de la app. Excepción explícita y solo de lectura a la regla §28.10 (la API no daba resultados de IA a la app).

 **HERRAMIENTA USADA Y POR QUÉ**

 **react-native-webview** — **Para qué:** mostrar el mapa Leaflet dentro de la app. **Por qué:** reutiliza la misma librería de la web y no exige clave de Google; crece solo ~0,2 MB.

La configuración de la app (CFG-8)

Todos los números que gobiernan la app, en un solo lugar.

Grupo	Valores (src/config/defaults.ts · CFG-8)
Red local	control 8765 · fotos 8766 · latido 2 s · inestable 3 s · perdido 6 s · respuesta de captura 8 s
Captura	intervalo 2 s (1, 2, 3, 5) · máx. 12 MP · JPEG 0,85 · prueba corta 20 s
Transferencia	una foto a la vez por cámara · espera 30 s · reintentos 2/4/8/16 s
Sincronización	modo TICKET · foto máx. 10 MB · subida 120 s · lotes de 200 · reintentos 1/5/15/60 min · auto cada 5 min con Wi-Fi
GPS	lectura cada 1 s · válida si tiene ≤ 10 s · aviso si la precisión es peor que 15 m
Dispositivo	batería: aviso bajo 15 % · espacio: aviso bajo 3 GB, no inicia bajo 2 GB, pausa bajo 1 GB
Login sin internet	7 días · 5 intentos · bloqueo 15 min · PBKDF2 40 000 iteraciones
Ubicar plaga	30 días · actualiza cada 2 min · GPS cada 2 s · «en el lugar» bajo 15 m
Retención	cámaras: borran tras sincronizar · controlador: conserva todo

🔧 EXPLICACIÓN TÉCNICA

Regla R-08: ningún umbral, puerto o tiempo va como número suelto en el código; todo sale de `src/config/defaults.ts`. `checkConfigCoherence()` revisa que los valores tengan sentido entre sí (por ejemplo, que «perdido» sea mayor que «inestable», o que la actualización de alertas sea mayor que la espera del pedido); una prueba falla si alguna regla no se cumple. Cada cambio de valores sube la versión (CFG-7 → CFG-8).

Reglas de construcción de la app (R-01 a R-22)

Cómo se escribe el código de la app para que no se rompa en el campo.

Regla	En pocas palabras
R-01	Expo SDK fijo; instalar con npx expo install
R-02	TypeScript estricto, sin any
R-03	Solo librerías Android + iOS
R-04	Lo nativo detrás de interfaces y require() diferido
R-05	Capas: pantallas → servicios
R-06	No inventar requisitos ni endpoints
R-07	Contratos exactos (tipos, SQL, protocolo, API)
R-08	Parámetros solo en src/config
R-09	Pruebas: typecheck, lint y jest
R-10	Nunca registrar contraseñas ni tokens
R-11	No borrar evidencia

Regla	En pocas palabras
R-12	Textos en español en strings.ts y messages.ts
R-13	Fechas ISO UTC; IDs con randomUUID
R-14	Transacciones para varias tablas
R-15	Uso en campo: botones ≥ 48 dp, letra ≥ 16
R-16	Reporte al terminar cada tarea
R-17	Sin diagnóstico, tratamientos ni dosis
R-18	Solo Django y la subida a Cloudinary (+ mapas, ADR 0009)
R-19	Todo en primer plano
R-20	Pocas dependencias, justificadas
R-21	Sin SDK de Supabase ni de Cloudinary
R-22	Ticket de un solo uso, pedido justo antes de subir

 **EXPLICACIÓN TÉCNICA**

Estas reglas vienen del maestro de la app (sección 1) y están resumidas en `AGENTS.md`. Algunas cambiaron por decisión registrada: el SDK pasó de 54 a **57** (ADR 0001), R-18 suma las teselas de Esri y Google Maps (ADR 0009) y la regla de cerrar sesión con monitoreo abierto cambió con el ADR 0008.

Reglas de negocio de la app (RN-01 a RN-25)

Lo que la app garantiza durante el trabajo de campo.

Regla	En pocas palabras
RN-01	Nada funciona sin una sesión de usuario válida
RN-02	Solo cuentas ACTIVO con rol permitido
RN-03	Login sin internet: último usuario, 7 días, sin bloqueo
RN-04	Contraseña temporal: solo cambiarla
RN-05	Una foto exige sesión, pasada, lateral, secuencia, IDs y hora
RN-06	Secuencia COMPLETE: las 2 fotos aceptadas y recibidas
RN-07	Repetir solo la cámara afectada, con el contexto actual
RN-08	Las cámaras no crean secuencias
RN-09	No borrar sin confirmación; nada de una sesión abierta
RN-10	Iniciar: cámaras conectadas, prueba corta, espacio
RN-11	Una orden a la vez
RN-12	Lado B solo con lado A cerrado; una pasada abierta
RN-13	Hilera cubierta = lados A y B cerrados

Regla	En pocas palabras
RN-14	UTILIZABLE ≠ sin plaga
RN-15	Cambiar función solo sin monitoreo ni envíos pendientes
RN-16	Un celular por rol de cámara
RN-17	Misma versión de protocolo (y de app en piloto)
RN-18	No inventar coordenadas
RN-19	Cerrar sesión no borra datos (ver ADR 0008)
RN-20	Bloqueo tras intentos fallidos sin internet
RN-21	La prueba corta no es evidencia
RN-22	Una foto pertenece siempre a su sesión
RN-23	SINCRONIZADO solo con respuesta de Django
RN-24	Nunca recomprimir una foto
RN-25	Ticket con margen; si no, pedir otro

 **EXPLICACIÓN TÉCNICA**

Las reglas de negocio (RN) describen el comportamiento que el operador ve en el campo, y cada una tiene pruebas automáticas. Ejemplos de cómo se aplican:

`deriveSequenceStatus` (RN-06), `decideRoleChange` (RN-15, ADR 0005), el chequeo de antigüedad del GPS `gps.maxAgeMs` (RN-18) y `sync.ticketMinRemainingMs` (RN-25).

Diagnóstico y problemas conocidos

Cómo se investiga un problema de campo y qué limitaciones se conocen hoy.

 **Exportar diagnóstico**

Ajustes → «Exportar diagnóstico» crea `diagnostico_<celular>_<fecha>.json` (versiones, configuración, sesiones, colas y eventos de 30 días; sin contraseñas ni tokens) y abre «Compartir».

 **Eventos útiles**

NET/LINK_STATE , TRANSFER/RECEIVED (bytes y ms),
SESSION/SHORT_TEST , CAPTURE/PICTURE_SIZE ,
GPS/LOW_ACCURACY , AUTH/PBKDF2_MS , SYNC/* .

 **Controlador en segundo plano**

Las cámaras se desconectan cada ~10 s: mantener la app al frente.

 **Login sin internet lento**

~9,7 s en el Galaxy A03; falta calibrar PBKDF2.

 **Recepción ~0,75 MB/s**

Si limita, el controlador podría descargar cada foto con el cliente HTTP nativo.

 **GPS impreciso**

16–28 m en interiores: por eso hilera, lado y plantas.

 **EXPLICACIÓN TÉCNICA**

El diagnóstico se comparte por WhatsApp o correo y permite ver, sin estar en el campo, qué pasó con cada transferencia y cada intento de sincronización. Documentos de apoyo: `docs/CAMBIOS.md` (versiones), `docs/evidencias/fase4_integracion.md` (pruebas) y `docs/referencia/CONTEXT0_ERRORRES_Y_CONFIGURACION.txt` .

 **HERRAMIENTA USADA Y POR QUÉ**

 **expo-sharing** — **Para qué:** enviar el archivo de diagnóstico. **Por qué:** usa el menú «Compartir» del celular: WhatsApp, correo o Drive.

7



Código de la plataforma web

Cómo están organizadas las carpetas, cuáles son los archivos que todo integrante debe conocer y cómo se ven las pantallas, incluidas las nuevas de la v1.3.

La carpeta del proyecto

Repositorio `agricola-riachuelo-platform`: cada carpeta tiene una responsabilidad.

agricola-riachuelo-platform/	
└ config/	configuración general de Django
└ api/	API para la app móvil (/api/v1)
└ web/	pantallas web: vistas, HTML, CSS, JS
└ campo/ cuentas/ monitoreo/ evidencias/	
└ ia/ revision/ notificaciones/ auditoria/	«apps» del negocio (una por tema)
└ diagnostico/	revisión de configuración y errores
└ simulador/	solo en desarrollo (fotos de prueba)
└ deploy/	instalación en el servidor (VPS)
└ tools/ia/	entrenamiento YOLO: notebook y verificación
└ scripts/ docs/	arranque local · documentación y ADR
└ manage.py	comando principal de Django
└ requirements.txt · requirements-ia.txt	librerías con versión fija
└ Dockerfile · iniciar.bat · AGENTS.md	imagen, arranque en Windows, reglas

Cómo leerla

- **Entrada:** `api/` (celulares) y `web/` (navegador).
- **Negocio:** 8 apps con las reglas y las tablas.
- **Soporte:** diagnóstico, simulador, despliegue y herramientas de IA.
- **Para agentes de IA:** `AGENTS.md` y `CLAUDE.md` con las reglas que más se rompen.

🔧 EXPLICACIÓN TÉCNICA

Repositorio `agricola-riachuelo-platform` (público), rama `prueba2`. En el VPS vive en `/srv/riachuelo` (permisos 700). `.gitignore` deja fuera `.env`, `db.sqlite3`, `media/`, `staticfiles/`, `.venv/`, pesos de IA y respaldos. El archivo `riachuelo-plataforma.tar.gz` es del primer despliegue (antes de usar git en el VPS) y ya no se usa.

🔗 HERRAMIENTA USADA Y POR QUÉ

📁 **Una app de Django por tema** — **Para qué:** ordenar el código. **Por qué:** cada tema (cuentas, campo, IA...) tiene sus tablas y reglas juntas; se encuentra todo rápido.

Anatomía de una «app» de Django

Las 8 apps del negocio tienen la misma forma. Si entiendes una, entiendes todas.

monitoreo/	
└ models.py	QUÉ se guarda: cada clase = una tabla
└ services.py	CÓMO se hace: las reglas del negocio
└ admin.py	cómo se ve en el panel /gestion/
└ migrations/	historial de cambios de las tablas
└ apps.py	registro de la app en Django
└ tests/	pruebas automáticas

La idea clave

- models.py describe los datos.
- services.py concentra las reglas: la API y la web lo llaman, así las reglas están en un solo lugar.
- Las vistas solo reciben el pedido y llaman al servicio.

🔗 **Para entenderlo:** models.py es el formulario en blanco, services.py es el manual de procedimientos y migrations/ es el registro de cada vez que se cambió el formulario.

 **EXPLICACIÓN TÉCNICA**

Reglas W-02 y W-03: plantilla → vista (web/views.py) → servicio (<app>/services.py) o consulta de solo lectura (web/queries.py) → modelo. Las reglas de revisión viven en revision/services.py , las de cuentas en cuentas/services.py , las de avisos en notificaciones/services.py y las de la cola en ia/services.py . Django Admin no ofrece acciones que salten esos servicios.

Las apps del negocio (1/2)

Del lugar físico y las personas hasta la foto.

App	De qué se encarga	Tablas
campo	El fundo: lotes, hileras, segmentos y marcadores; perfiles de calidad; v1.3 contornos y puntos con nombre	<code>field_lots</code> , <code>field_rows</code> , <code>field_segments</code> , <code>markers</code> , <code>quality_profiles</code> , <code>points_of_interest</code>
cuentas	Usuarios, roles, celulares registrados, pedidos de contraseña	<code>users</code> , <code>user_roles</code> , <code>devices</code> , <code>password_reset_requests</code>
monitoreo	Sesiones de recorrido, cámaras de la sesión, pasadas por lado, cambios de marcador, secuencias e incidencias	<code>monitoring_sessions</code> , <code>session_devices</code> , <code>monitoring_passes</code> , <code>marker_changes</code> , <code>capture_sequences</code> , <code>incidents</code>
evidencias	Las fotos (capturas), su calidad y la conexión con Cloudinary	<code>captures</code> , <code>quality_results</code>

 **EXPLICACIÓN TÉCNICA**

`campo/services.py` tiene toda la gestión de catálogos (v1.2) y, desde la v1.3, el editor del mapa (contorno del lote, inicio y fin de hilera, puntos). `cuentas/services.py` tiene `validate_roles` , `account_kind` , `create_account` , `update_account` y `set_password_by_admin` . `evidencias/services.py` confirma la subida (verifica la firma de Cloudinary) y encola el análisis en la misma transacción; `evidencias/media.py` firma las URL.

Las apps del negocio (2/2)

Del análisis a la decisión, el aviso y su registro.

App	De qué se encarga	Tablas
ia	Modelos de IA, cola de análisis, detecciones, detectores (onnx, yolo, simulado) y el worker	model_configs , ai_tasks , detections
revision	Casos para revisar, ubicación del caso, decisiones del especialista, autoconfirmación v1.3	review_cases , human_reviews
notificaciones	Destinatarios por lote, avisos de WhatsApp y su cola de envío	notification_recipients , notification_recipient_lots , notifications
auditoria	Registro inmutable de todo lo importante; seguridad de la base (RLS y REVOKE)	audit_events

 **EXPLICACIÓN TÉCNICA**

ia/management/commands/worker_ia.py es el worker; ia/inference/ tiene onnx.py (con analizar_imagen()), yolo.py y simulado.py . revision/services.py : open_case_from_analysis , case_location , decide_case , correct_decision , open_manual_case . auditoria/seguridad_bd.py aplica RLS y REVOKE después de cada migrate .

Las puertas de entrada: api/ y web/

Dos formas de llegar al mismo backend: una para celulares, otra para personas en el navegador.

api/	
└ v1/urls.py	la lista de direcciones de la API
└ v1/views.py	qué hace cada dirección
└ v1/serializers.py	valida y traduce el JSON
└ authentication.py	JWT + X-Device-Id
└ permissions.py	FieldWork · PestReaders (v1.3)
└ errors.py	formato único de errores
web/	
└ urls.py · views.py	páginas y acciones de la web
└ queries.py · forms.py	consultas de pantallas · formularios
└ permissions.py	quién puede ver qué (por rol)
└ messages.py	todos los textos para el usuario
└ templates/web/	el HTML de cada pantalla
└ static/web/	CSS, JavaScript, íconos, sonidos

Recuerda

- Ninguna de las dos tiene reglas propias: llaman a los `services.py` .
- La API responde JSON; la web responde HTML.
- La API autentica con token; la web con cookie de sesión y CSRF.

 **EXPLICACIÓN TÉCNICA**

`api/authentication.py` valida el JWT y la cabecera `X-Device-Id` (celular registrado y no revocado); `api/permissions.py` define `FieldWork` (operador y administrador: sincronización) y `PestReaders` (también el especialista: alertas). En la web, `web/management/commands/` tiene `sembrar_demo` (solo desarrollo) y `medir_rendimiento` .

config/ y los archivos de la raíz

La configuración que afecta a todo el proyecto.

Archivo	Para qué sirve
config/settings.py	La configuración central: base de datos, seguridad, Cloudinary, WhatsApp, IA. Lee las variables del .env
config/urls.py	El «índice» de direcciones: /api/v1/ , /gestion/ , /diagnostico/ y la web
config/restricciones.py	Agrega reglas CHECK a la base (estados válidos, rangos de GPS)
manage.py	La puerta de los comandos: migrate , runserver , createsuperuser , diagnostico ...
.env / .env.example	Variables y secretos de cada entorno. .env.example es la plantilla sin secretos
requirements.txt	Lista de librerías de Python con versión fija (más requirements-ia.txt para YOLO)
Dockerfile · .dockerignore	La imagen única de web y worker; deja fuera .env , .venv y datos

 **EXPLICACIÓN TÉCNICA**

settings.py cambia según APP_ENV : en dev acepta SQLite, Cloudinary simulado e IA simulada; en piloto exige DJANGO_SECRET_KEY , DATABASE_URL de PostgreSQL, Cloudinary, HTTPS y dominio configurado, y no arranca si DEBUG está activo. Idioma es-pe , zona America/Lima , sesiones de 8 h, HSTS y cookies seguras.

Soporte: diagnóstico, simulador, despliegue y herramientas

Carpetas que no son del negocio, pero hacen la vida más fácil.

Carpeta / comando	Para qué sirve
diagnostico/ → <code>diagnostico</code>	Revisa todo (BD, RLS, CHECK, Cloudinary, IA, cola, cuentas) y dice qué corregir
diagnostico/ → <code>comprobar_bd</code>	Prueba la conexión y explica el error en lenguaje claro
diagnostico/ → <code>respaldo</code> · <code>mantenimiento</code>	Copia de la base (pg_dump) y limpieza de sesiones, tokens y caché
diagnostico/ (errores)	Los errores del navegador llegan a la consola del servidor con un traceld
simulador/	Solo en desarrollo: imita Cloudinary y genera fotos de prueba
web/ → <code>sembrar_demo</code>	Carga datos de demostración (prohibido en el piloto)
deploy/	Instalar, actualizar y retirar la plataforma del VPS
tools/ia/ <code>nuevo</code>	Notebook de Colab, <code>cortar_mosaicos.py</code> , <code>verificar_onnx.py</code> y guía
ia/ → <code>exportar_dataset</code>	Exporta fotos (o casos revisados) para etiquetar y entrenar

 **EXPLICACIÓN TÉCNICA**

`exportar_dataset --modo fotos` saca fotos del fundo para etiquetarlas; `--modo revisados` usa las decisiones de los especialistas como etiquetas (cajas confirmadas o descartadas): así el sistema produce sus propios datos de entrenamiento para el modelo v2. `cortar_mosaicos.py` corta fotos de 12 MP en recortes de 1280 px y `verificar_onnx.py` comprueba un modelo con fotos reales antes de activarlo.

Los archivos que todo integrante debe conocer

Si solo tienes una hora para entender el proyecto, empieza por estos.

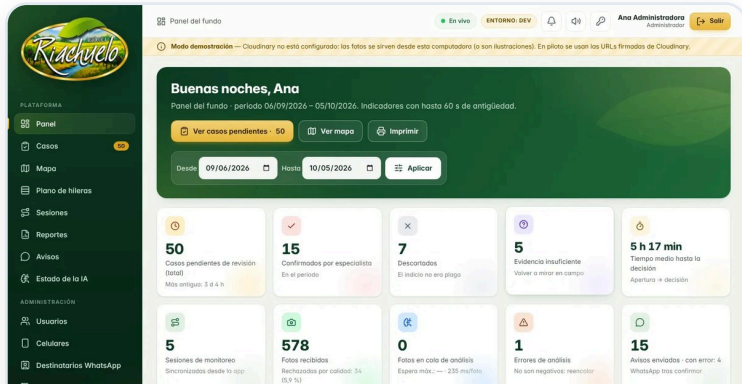
#	Archivo	Por qué importa
1	README.md · AGENTS.md	Cómo instalar y ejecutar · reglas para agentes
2	config/settings.py	Toda la configuración
3	*/models.py	Qué datos existen (las tablas)
4	*/services.py	Las reglas del negocio
5	api/v1/urls.py	Qué ofrece la API a la app
6	web/urls.py · web/permissions.py	Qué pantallas tiene la web y quién las ve
7	ia/management/commands/worker_ia.py	Cómo se analizan las fotos
8	revision/services.py	Cómo nace, se decide y se ubica un caso
9	docs/ARQUITECTURA_DATOS.md	Diseño y seguridad de la base
10	docs/INTEGRACION_APP.md	Contrato con la app móvil
11	docs/adr/	Las decisiones de arquitectura (hasta ADR-W-007)
12	deploy/LEEME.md	Cómo vive en el servidor

 **EXPLICACIÓN TÉCNICA**

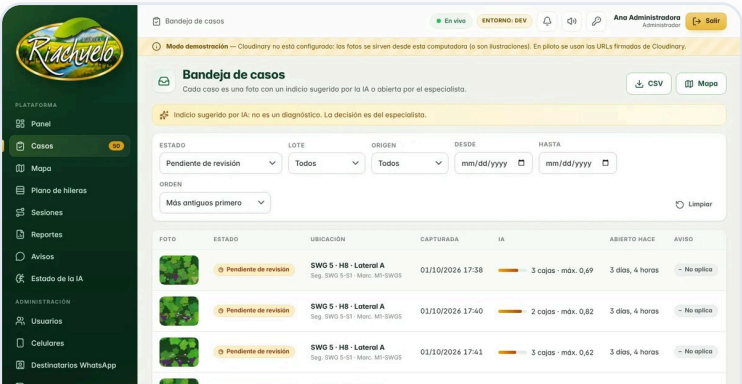
Orden sugerido para alguien nuevo: README → AGENTS.md → web/permissions.py (quién hace qué) → revision/models.py y services.py (el corazón del negocio) → api/v1/urls.py (lo que usa la app) → worker_ia.py → ADR. Los textos que ve la persona están en web/messages.py , nunca sueltos en las plantillas.

Las pantallas de la web

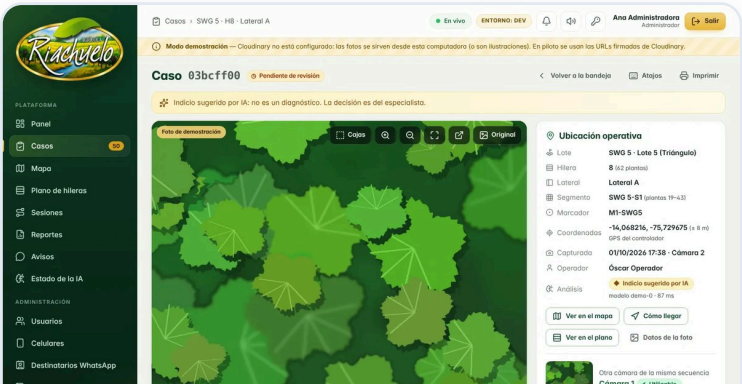
Lo que ven especialistas, supervisores y administradores.



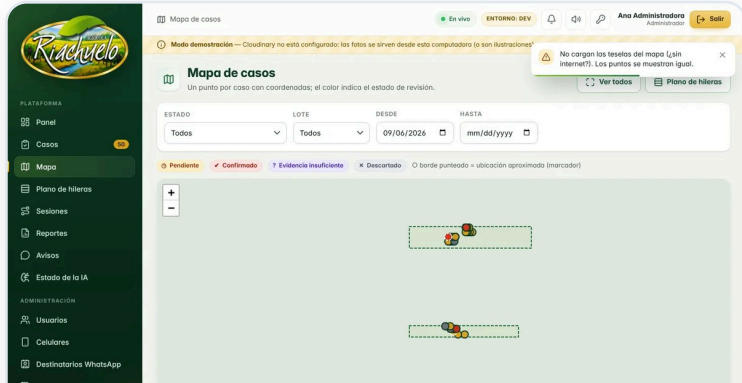
Panel: indicadores del día



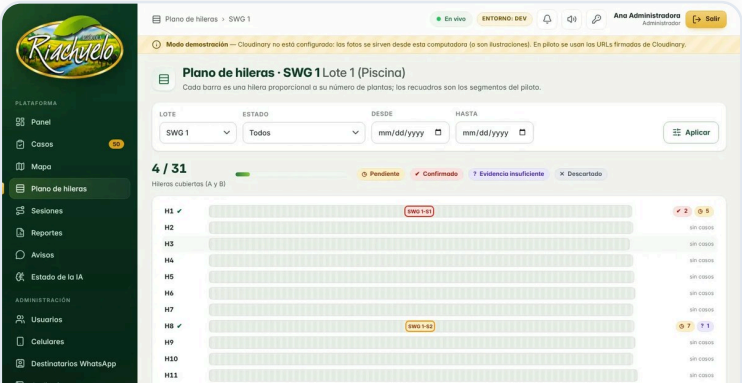
Bandeja: casos por revisar



Caso: foto con cajas y decisión



Mapa del fondo por estado



Plano de hileras



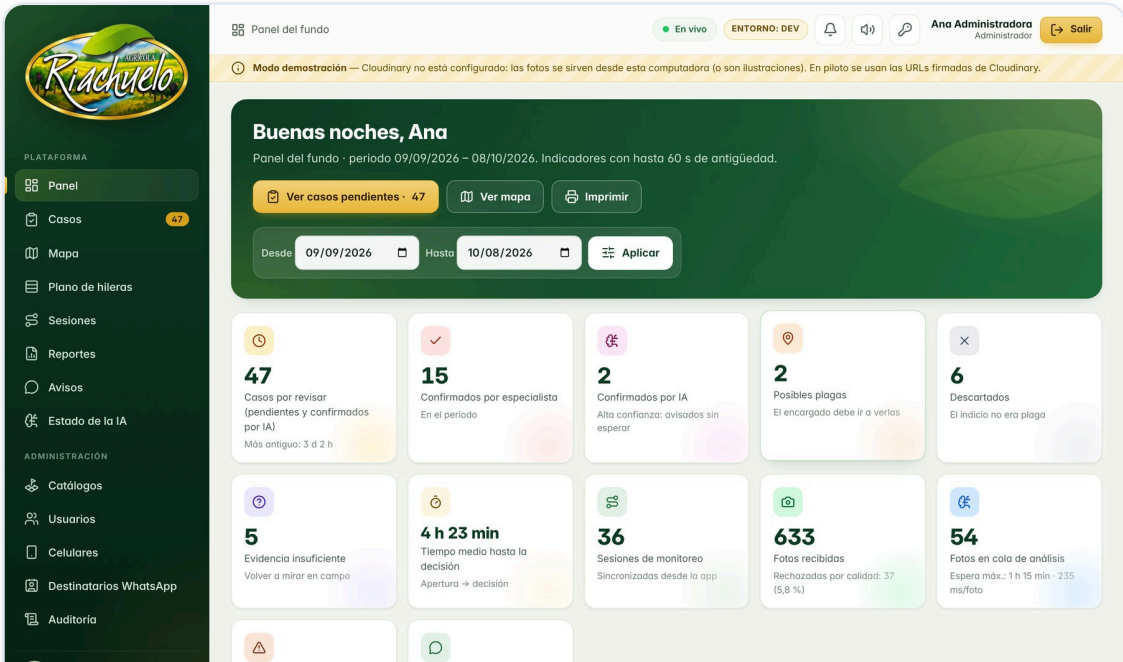
También se adapta al celular

EXPLICACIÓN TÉCNICA

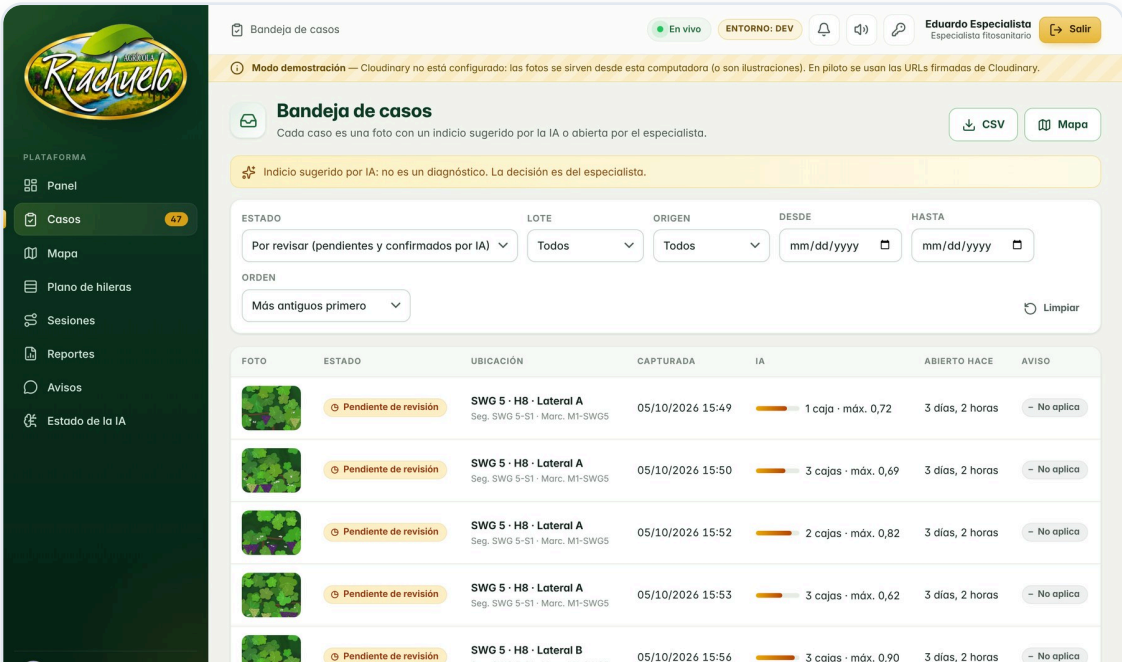
Capturas de la v1.0–v1.2 con datos de demostración. El diseño es responsivo (celular, tableta y escritorio), con avisos «toast», sonidos cuando llegan casos nuevos, página 403 propia y reporte automático de errores de JavaScript. La bandeja y la actividad se refrescan solas (15 y 20 s) solo si la pestaña está visible.

El panel y la bandeja, hoy

Las dos pantallas de entrada, ya con los estados de la v1.3.



Panel del fundo (v1.3) · indicadores con los estados nuevos



Bandeja de casos (v1.3) · abre en «Por revisar»

Panel

Casos por revisar, confirmados por especialista y **por IA, posibles plagas**, descartados, evidencia insuficiente, tiempo medio hasta la decisión, sesiones, fotos, cola de IA y avisos.

Bandeja

Por defecto muestra **«Por revisar»** = pendientes + confirmados por IA, los más antiguos primero, con filtros por estado, lote, origen y fechas, y exportación CSV.

EXPLICACIÓN TÉCNICA

Los indicadores del panel se guardan en caché 60 s (tabla `web_cache`): por eso responde en ~5 ms. La bandeja hace **exactamente 6 consultas** a la base, lo comprueba una prueba (`assertNumQueries(6)`) y tarda ~39 ms. Capturas del entorno de desarrollo (banner «Modo demostración»).

El caso: «Confirmado por IA» y la decisión

La pantalla donde el especialista mira la foto y decide. Las capturas son de la v1.3.

PLATAFORMA

Panel

Casos47

Mapa

Plano de hileras

Sesiones

Reportes

Avisos

Estado de la IA

Casos > SWG 1 · H22 · Lateral B

En vivoENTORNO: DEV

Eduardo EspecialistaEspecialista fitosanitario

Sair

Modo demostración — Cloudinary no está configurado: las fotos se sirven desde esta computadora (o son ilustraciones). En piloto se usan las URLs firmadas de Cloudinary.

Caso 6d6e1ff7Confirmado por IA

Volver a la bandejaAtajosImprimir

Indicio sugerido por IA: no es un diagnóstico. La decisión es del especialista.

Foto de demostraciónCajasOriginal

3 · 0,57

Ubicación operativa

LoteSWG 1 · Lote 1 (Piscina)

Hilera22 (270 plantas)

LateralLateral B

SegmentoSWG 1-S4 (plantas 123–147)

MarcadorM4-SWG1

Coordenadas-14,060594, -75,728349 (± 3 m)
GPS del controlador

Capturada07/10/2026 16:32 · Cámara 2

OperadorÓscar Operator

AnálisisIndicio sugerido por IA
modelo demo-0 · 124 ms

Ver en el mapa

Cómo llegar

Ver en el plano

Datos de la foto

Confirmado por la IA con alta confianza (94 %; umbral 80 %). Ya se avisó por WhatsApp y aparece en la app. Revisalo: puedes confirmarlo, dejarlo como posible plaga o descartarlo.

Aviso por WhatsApp: No aplica

Decisión del especialista

Confirmado por especialista

Es plaga: se avisa por WhatsApp y aparece en la app

C

Possible plaga

Puede ser plaga: aparece en la app para que el encargado vaya a verla (sin WhatsApp)

P

Descartado

No corresponde a una plaga: desaparece de la app

D

Evidencia insuficiente

La foto no permite decidir: volver a mirar en campo

I

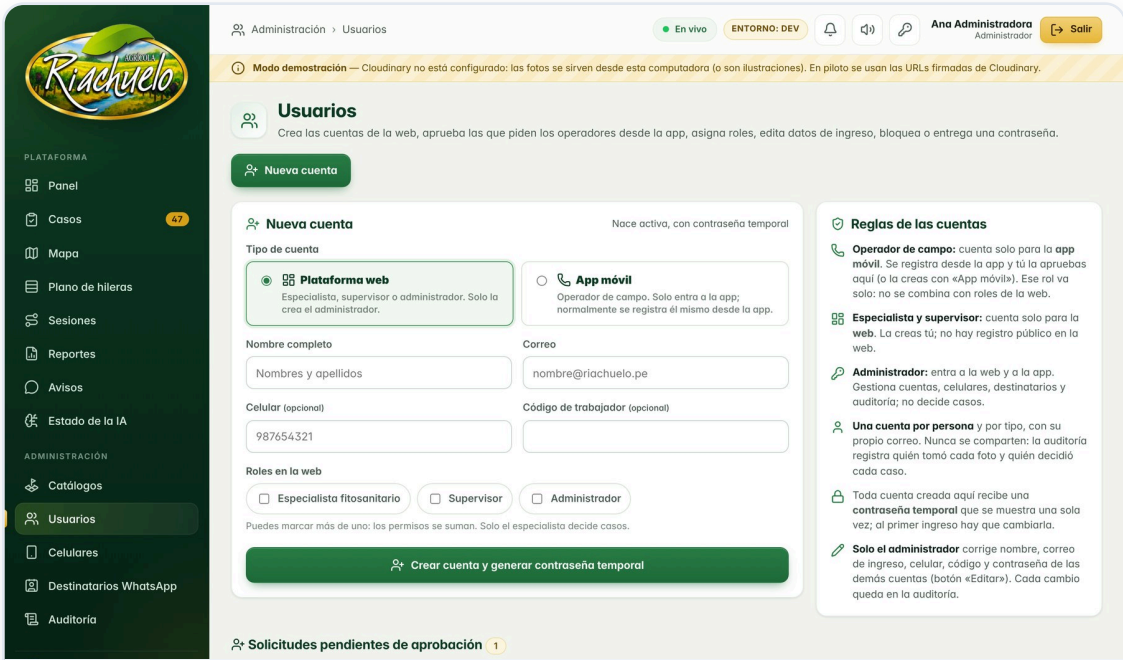
Panel de decisión · C · P · D · I

EXPLICACIÓN TÉCNICA

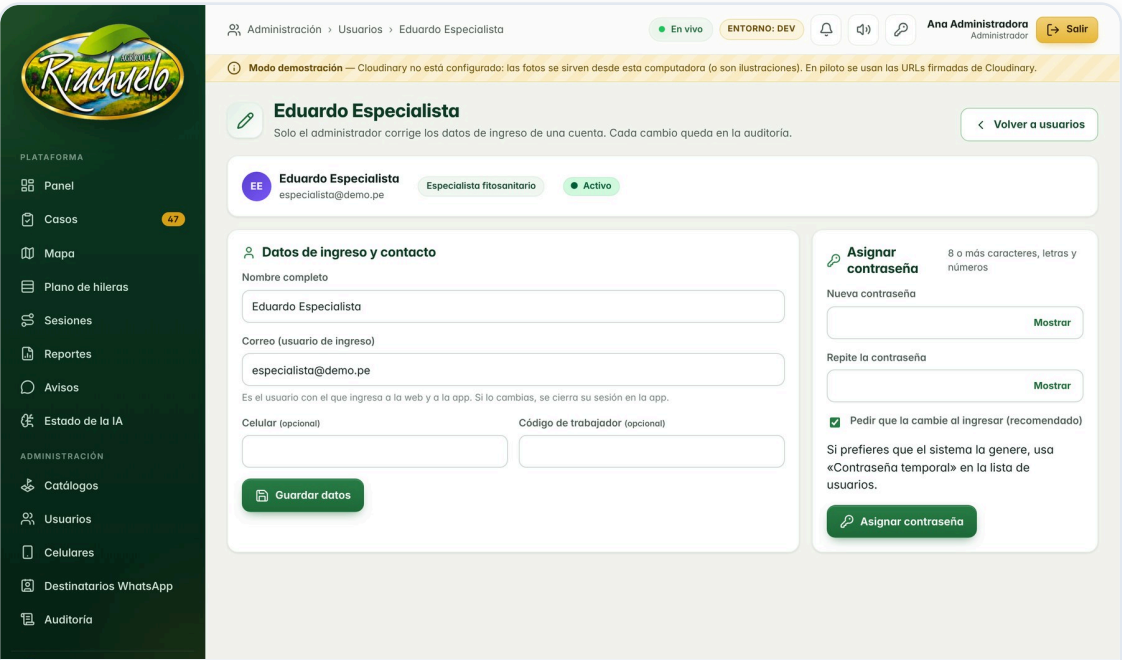
El visor dibuja las cajas sobre la versión `revision` (1600 px) con un SVG del tamaño de la imagen analizada; se acerca y se mueve en el navegador, y la original solo se carga con «Original». La ubicación operativa muestra lote, hilera, lateral, segmento (plantas), marcador, coordenadas con precisión, cámara, operador y análisis (modelo y ms), con «Ver en el mapa», «Cómo llegar» y «Ver en el plano». Atajos: **C** confirmar, **P** posible plaga, **D** descartar, **I** evidencia insuficiente, **Ctrl+Enter** guardar.

Usuarios: crear, aprobar y editar

Solo el administrador gestiona cuentas; desde la v1.3 también corrige los datos de ingreso.



Usuarios · «Nueva cuenta» y reglas de las cuentas



Editar cuenta (v1.3) · solo el administrador

+ Nueva cuenta

«Plataforma web» (especialista, supervisor, administrador) o «App móvil» (operador). Nace activa con una **contraseña temporal** que se muestra una sola vez.

✎ Editar v1.3

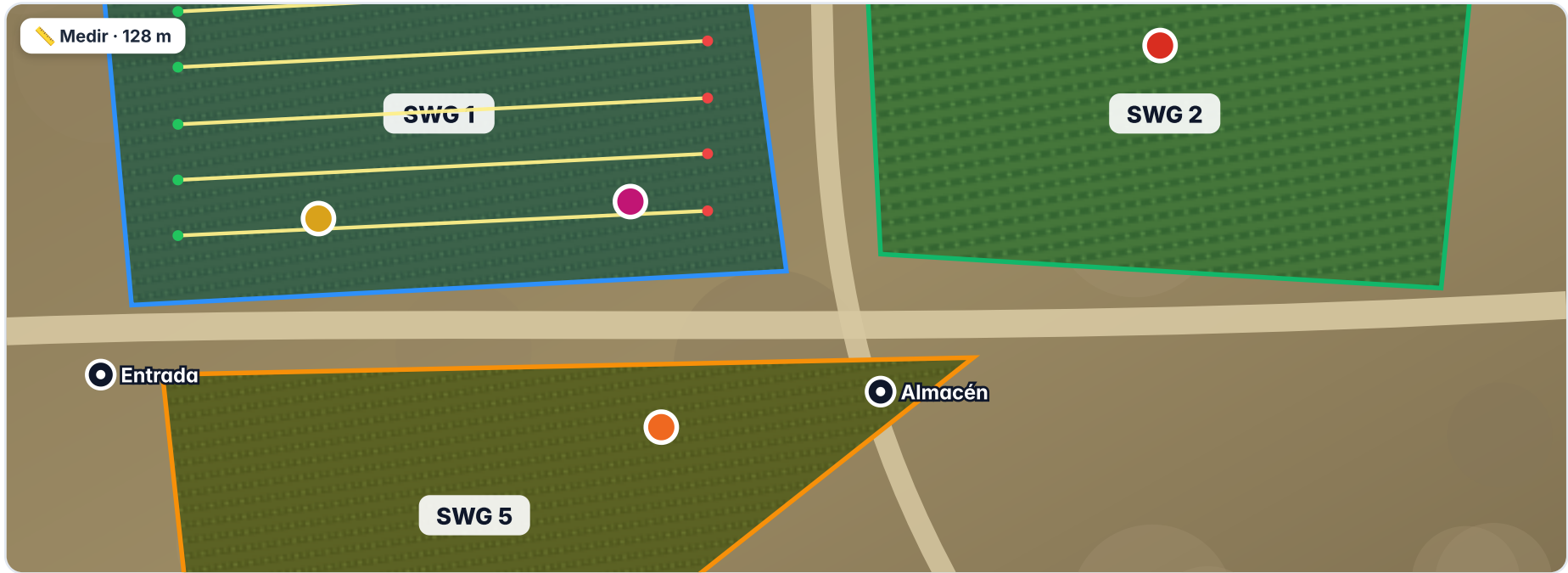
Nombre, correo de ingreso, celular, código y asignar contraseña (con «pedir que la cambie al ingresar»). Cambiar correo o contraseña cierra la sesión de la app.

🔧 EXPLICACIÓN TÉCNICA

Ruta `/administracion/usuarios/<id>/editar/` , permiso `usuarios.gestionar` ; servicios `update_account` y `set_password_by_admin` , auditados como `CUENTA_EDITADA` y `CONTRASENA_ASIGNADA` (nunca con la contraseña). En la misma pantalla se aprueban las solicitudes que llegan desde la app.

El mapa satelital y su editor

Nuevo en la v1.3: ver el fondo desde el satélite y dibujar lotes, hileras y puntos. (Ilustración del mapa.)



 Medir · 128 m

SWG 1

SWG 2

Entrada

Almacén

SWG 5

 **Contorno de lote**

Clic en cada esquina sobre la imagen satelital y «Guardar».

 **Inicio y fin de hileras**

Primer clic = inicio, segundo = fin; pasa sola a la siguiente.

 **Punto con nombre**

Entrada, almacén, pozo, reunión, oficina u otro.

Elige una herramienta o navega por el mapa.

Medir

 Medir distancia

Haz clic en dos o más puntos; doble clic o «Terminar» para cerrar.

Lote A: — ▼ Lote B: — ▼

 Distancia entre lotes

☒ Sombrear lotes

☒ Mostrar hileras

☒ Mostrar puntos del fondo

Contorno de lote

Lote: SWG 1 · Lote 1 (Piscina) ✓ ▼

 Dibujar

Herramientas del mapa

 **EXPLICACIÓN TÉCNICA**

Fondo Esri World Imagery (o calles), centro en el fondo. Lotes sombreados con área y perímetro, hileras como línea entre sus marcadores de inicio y fin (los crea si faltan), puntos de `points_of_interest`, medir distancias y la distancia más corta entre dos lotes. Pueden editar el administrador y el supervisor (`catalogos.gestionar`); se guarda con `POST /mapa/editar/` (JSON + CSRF) y queda en la auditoría (`LOTE_CONTORNO` , `MARCADOR_UBICADO` , `PUNTO_CREADO` ...).

El frontend de la web, archivo por archivo

Todo se sirve desde el propio servidor: sin CDN externos.

Archivo	Qué hace
templates/web/base.html	El «marco» común: menú, cabecera, avisos. Las demás pantallas se insertan dentro
templates/web/*.html	Una plantilla por pantalla: bandeja, caso, mapa, plano, reportes, usuarios, usuario_editar.html (v1.3)...
templates/web/partials/	Pedazos que HTMX actualiza solos (tabla de la bandeja, actividad en vivo, panel de decisión)
static/web/app.js	Comportamiento general: avisos, sonidos, barra de carga, confirmaciones, errores a la consola
static/web/caso.js · mapa.js	Zoom de la foto, cajas y atajos de decisión · mapa con Leaflet y su editor (v1.3)
static/web/app.css · vendor/	Estilos propios (responsivos) · copias locales de HTMX 2.0.11 y Leaflet 1.9.4
static/web/iconos.svg · fuentes/ · sonidos/	Íconos Lucide en un sprite · fuente Inter · sonidos de aviso

⚠ **Importante:** todo se sirve desde el propio servidor: no se usan CDN externos (W-01) ni JavaScript dentro de las plantillas (W-17). Una prueba automática (web/tests/test_extras.py) falla si aparece.

 **EXPLICACIÓN TÉCNICA**

Los datos pasan del servidor al JavaScript por atributos `data-*` o `json_script` , nunca con `<script>` en línea: así la web está lista para una política de seguridad de contenido (CSP) estricta. Los números que van a atributos o CSS se escriben sin localizar (`|stringformat`), porque `es-pe` usa coma decimal (W-16).

Las rutas de la web y su permiso

Cada dirección de la web, qué muestra y qué permiso exige.

Ruta	Pantalla	Permiso
/ingresar/ · /salir/ · /cuenta/contrasena/	Ingreso, salida y cambio de contraseña	sesión
/ · /actividad/	Panel · actividad en vivo (cada 20 s)	dashboard.ver
/casos/ · /casos/<id>/	Bandeja · caso	bandeja.ver · caso.ver
.../decidir/ · .../corregir/	Decidir · corregir con motivo	caso.decidir · caso.corregir
/capturas/<id>/ · .../abrir-caso/	Foto · abrir caso manual	caso.ver · caso.abrir_manual
/mapa/ · datos/ · capas/ · editar/	Mapa, GeoJSON, capas del fondo y editor (v1.3)	mapa.ver · catalogos.gestionar
/plano/ · /sesiones/ · /reportes/ · casos.csv	Plano, sesiones, reportes y CSV	mapa.ver · sesiones.ver · reportes.exportar
/notificaciones/	Avisos enviados o pendientes	notificaciones.ver
/administracion/usuarios/ · nueva/ · <id>/editar/	Cuentas	usuarios.gestionar
/administracion/catalogos/ · dispositivos/ · destinatarios/	Catálogos, celulares, destinatarios	catalogos.gestionar · dispositivos... · destinatarios...
/ia/ · /ia/tareas/<id>/reencolar/ · /auditoria/	IA, reencolar y auditoría	ia.ver · ia.reencolar · auditoria.ver
/gestion/	Django Admin (superusuario)	admin de Django

 **EXPLICACIÓN TÉCNICA**

Rutas en web/urls.py ; la v1.3 agregó /mapa/capas/ , /mapa/editar/ y /administracion/usuarios/<id>/editar/ . Una prueba recorre todas las rutas y falla si alguna no tiene @web_view (salvo ingresar, salir y cambiar contraseña). Django Admin ya no crea usuarios ni borra catálogos.

Catálogos del fondo desde la web

Lotes, hileras, segmentos y marcadores: la base para que la app pueda trabajar.

 **Lotes**

Crear y editar. El identificador se genera del código («SWG 1» → SWG1) y no cambia.

 **Hileras en bloque**

Desde–hasta y plantas por hilera (máx. 300 por vez), con segmento completo y marcadores de inicio y fin.

 **Segmentos**

Crear, editar o «Dividir en segmentos»: en N partes iguales o cada K plantas, con vista previa.

 **Marcadores**

Inicio, fin o intermedios, con coordenadas opcionales (6 decimales) y descripción.

 **«Eliminar» = desactivar**

Nunca se borra nada y se puede reactivar. La web avisa si hay sesiones en curso que lo usan.

 **Llega a la app**

Con «Catálogos → Actualizar»: el bootstrap ya no envía lo inactivo, pero la sincronización acepta IDs desactivados.

 **EXPLICACIÓN TÉCNICA**

ADR-W-006 (v1.2). IDs del servidor, estables: hilera {lote}-H{nn} (SWG1-H05), segmento {hilera}-S{k} , marcadores de hilera completa -INI y -FIN , otros -M{k} . El 07/10 se crearon por script los 83 segmentos «Hxx completa» con sus marcadores. Todo queda en la auditoría (CATALOGO_CREADO , CATALOGO_EDITADO , CATALOGO_DESACTIVADO , CATALOGO_REACTIVADO , CATALOGO_LOTE_HILERAS).

 **HERRAMIENTA USADA Y POR QUÉ**

 **Administración → Catálogos** — **Para qué:** cargar el fondo sin tocar la base. **Por qué:** antes había que usar /gestion/ o scripts; ahora lo hace el supervisor desde la web.

Reglas de la web (W-01 a W-24)

Cómo se construye la web para que siga siendo simple y segura.

Regla	En pocas palabras
W-01	Stack fijo: Django + HTMX + Leaflet, sin CDN ni npm
W-02	Capas: vista → servicio → modelo
W-03	Una regla, un lugar (services.py)
W-04	No inventar: preguntar o marcar «Supuesto»
W-05	Contratos exactos (modelos, estados, permisos)
W-06	Transacciones y select_for_update
W-07	Permisos en el servidor: @web_view
W-08	Sesión de Django y CSRF siempre
W-09	Sin secretos en el navegador
W-10	URLs firmadas al mostrar; nunca guardadas
W-11	La IA «sugiere indicios», nunca diagnostica
W-12	Colores = estado, con ícono y texto

Regla	En pocas palabras
W-13	Fuera de alcance: tratamientos, dosis, urgencia
W-14	Evidencia inmutable
W-15	Auditoría en la misma transacción
W-16	Español; números sin localizar en atributos
W-17	Sin JavaScript en línea
W-18	HTMX con patrones fijos (P-1 a P-9)
W-19	Consultas acotadas; bandeja = 6 consultas
W-20	Pruebas antes de entregar
W-21	Pocas dependencias, justificadas
W-22	sembrar_demo prohibido en piloto
W-23	Accesibilidad (contraste AA, teclado, alt)
W-24	Reporte de avance al terminar

 **EXPLICACIÓN TÉCNICA**

Vienen del maestro de la web (sección 1) y se resumen en `AGENTS.md` . Las que más se rompen: CDN (W-01), JavaScript en línea (W-17), vista sin `@web_view` (W-07), consultas extra en `base.html` (bandeja de 6 consultas), colores como gravedad (W-12) y números localizados en CSS (W-16). Al terminar: `check` , `makemigrations --check --dry-run` , `test` y el reporte en `docs/REPORTE_AVANCE.md` .

Reglas de negocio de la web (RN-W01 a RN-W14)

Lo que la plataforma garantiza al revisar y decidir.

Regla	Qué garantiza
RN-W01	Solo un especialista ACTIVO decide, corrige o abre casos manuales (el servicio lo vuelve a comprobar)
RN-W02	Una sola decisión vigente; se «decide» lo decidible (pendiente o confirmado por IA) y luego solo se corrige
RN-W03	Avisan Confirmado por especialista y, desde la v1.3, Confirmado por IA ; un error de envío no cambia la decisión
RN-W04	Observación obligatoria al confirmar y en evidencia insuficiente; máx. 2 000; sin tratamientos
RN-W05	Corregir exige motivo y que la decisión vigente sea la que se tenía en pantalla
RN-W06	Un análisis nuevo con cajas actualiza un caso pendiente; ninguno cambia una decisión
RN-W07	La IA es un indicio no vinculante (excepción v1.3: umbral de autoconfirmación, apagado por defecto)
RN-W08	La web no crea, edita ni borra datos de campo, análisis, decisiones ni avisos enviados
RN-W09	Caso manual solo sobre foto aceptada con análisis sin indicios o con error; un caso por foto
RN-W10	Destinatario activo, con consentimiento y con el lote en su alcance
RN-W11	Solo se analizan fotos UTILIZABLE o PENDIENTE_REVISION_TECNICA
RN-W12	La ubicación se fija al abrir el caso (repetición → secuencia; GPS → marcador → ninguna)
RN-W13	Hilera cubierta = lados A y B cerrados en el periodo
RN-W14	Nadie cambia su propia cuenta ni se quita su rol de administrador

 **EXPLICACIÓN TÉCNICA**

Las reglas de negocio de la web están probadas con pruebas automáticas (203 en total en la v1.3). La v1.3 (ADR-W-007) modificó RN-W03 y RN-W07 a propósito: la IA puede avisar sola solo con un umbral activado por modelo; el resto de reglas sigue igual.

8



La API

Cómo conversan la app móvil y el servidor: direcciones, formato de los mensajes, errores, subida de fotos y la ruta nueva para «Ubicar plaga».

Las direcciones de /api/v1

Todas empiezan con `https://monitoreo.agricolariachuelo.org/api/v1`.

Grupo	Direcciones	Para qué
Salud	<code>GET /health</code>	¿El servidor y la base responden? (lo usa también Docker)
Cuenta	<code>/auth/register</code> · <code>login</code> · <code>refresh</code> · <code>logout</code> · <code>me</code>	Registro, ingreso, renovar y cerrar sesión, perfil
Contraseña	<code>/auth/change-password</code> · <code>/auth/password-reset-requests</code>	Cambiar la contraseña · pedir ayuda al administrador (siempre 202)
Arranque	<code>GET /mobile/bootstrap</code>	Catálogos para trabajar offline: lotes, hileras, segmentos, marcadores
Recorrido	<code>POST /sessions</code> · <code>/sessions/{id}/passes</code>	Crear o actualizar sesiones y pasadas
Detalle	<code>/sequences/batch</code> · <code>/incidents/batch</code>	Secuencias de disparo e incidencias, en lotes de 200
Fotos	<code>/captures/{id}/upload-ticket</code> · <code>/captures/upload</code> · <code>GET /captures/{id}</code>	Permiso de subida, confirmación y estado de la foto
Alertas v1.3	<code>GET /mobile/pest-reports?days=30</code>	Casos con ubicación para «Ubicar plaga»
Documentación	<code>GET /schema</code>	Descripción automática de toda la API (OpenAPI)

 **EXPLICACIÓN TÉCNICA**

Todas cuelgan de `https://monitoreo.agricolariachuelo.org/api/v1`. La app llama sin barra final (también se acepta con barra). Todo es JSON con nombres en camelCase; las fechas en ISO-8601 UTC. Las rutas de campo (sesiones, pasadas, secuencias, incidencias, fotos) exigen el permiso `FieldWork` (operador o administrador); `pest-reports` exige `PestReaders` (también el especialista).

 **HERRAMIENTA USADA Y POR QUÉ**

 **drf-spectacular** — **Para qué:** publicar el esquema en `/api/v1/schema`. **Por qué:** la documentación sale del mismo código: nunca queda desactualizada.

Cómo se ven los mensajes

Siempre JSON, con nombres en camelCase. Los errores tienen siempre la misma forma.

✔ **Respuesta correcta (login)**

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "user": { "fullName": "Ana Pérez",
            "roles": ["OPERADOR_CAMPO"] }
}
```

>

✖ **Error (siempre igual)**

```
{
  "code": "VALIDATION_ERROR",
  "message": "Revisa los datos enviados.",
  "fieldErrors": [ { "field": "email",
                     "message": "Correo inválido" } ],
  "traceId": "9f2c41..."
}
```

❗ **Dato clave:** el `traceId` es un código único de cada pedido. Si algo falla, con ese código se encuentra el error exacto en la consola del servidor.

⚙ **EXPLICACIÓN TÉCNICA**

Todos los errores tienen la forma `ApiErrorBody = { code, message, fieldErrors?, traceId }` (`api/errors.py`). El manejador convierte también los errores de DRF y cualquier error inesperado (500 `INTERNAL_ERROR` , con la traza en la consola y el mismo `traceId`). El texto que ve la persona lo decide la app con su tabla de mensajes en español; el servidor solo manda el código.

Agrícola Riachuelo · Plataforma de monitoreo fitosanitario

107 / 183

Los errores, uno por uno

Cada código de error, cuándo aparece y cómo reacciona la app.

Código	HTTP	Cuándo pasa	Qué hace la app
VALIDATION_ERROR	400	Datos inválidos (también 405 y 415)	Muestra qué campo corregir
INVALID_CREDENTIALS	401	Correo o contraseña incorrectos	Pide de nuevo los datos
TOKEN_EXPIRED · REFRESH_INVALID	401	El pase venció · la sesión ya no vale	Renueva · vuelve al login
ACCOUNT_PENDING · _REJECTED · _BLOCKED	403	La cuenta no está activa	Explica el estado
ROLE_NOT_ALLOWED · DEVICE_REVOKED	403	Rol sin permiso · celular revocado	Cierra la sesión
EMAIL_ALREADY_REGISTERED	409	Ese correo ya tiene cuenta	Ofrece ingresar
CAPTURE_CONFLICT · UPLOAD_MISMATCH	409	Mismo captureId con otro contenido	Error definitivo visible
UPLOAD_SIGNATURE_INVALID	422	La firma de Cloudinary no coincide	Error definitivo visible
PAYLOAD_TOO_LARGE	413	La foto supera 10 MB	FOTO_DEMASIADO_GRANDE
SESSION_/PASS_/SEQUENCE_NOT_FOUND	404	Falta el «padre» en el servidor	Reordena la cola y reintenta
TOO_MANY_ATTEMPTS	429	Demasiados intentos	Espera lo que dice Retry-After
INTERNAL_ERROR	500–503	Problema del servidor o de Cloudinary	Reintenta más tarde

⚙️ EXPLICACIÓN TÉCNICA

Los códigos están en `api/errors.py` (`MESSAGES`) y en el maestro de la app (§15.3). Los errores «de red» o 5xx son **temporales**: la cola los reintenta. Los de datos (409, 422, 413) son **definitivos**: se muestran en la pantalla de sincronización para que alguien los revise, sin bloquear el resto de la cola.

Cómo se identifica la app

Token JWT más el identificador del celular, en cada pedido.

 **Login**

Correo + contraseña + datos del celular. Devuelve access (15 min) y refresh (14 días).

 **Renovar**

Con el refresh se pide un par nuevo; el anterior entra a la lista negra (rotación).

 **Logout**

Revoca el refresh: ese celular ya no puede renovar.

 **Celular registrado**

El X-Device-Id debe existir y no estar revocado.

Las cabeceras de un pedido

```
# cada pedido de la app lleva dos cabeceras
POST /api/v1/sessions
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
X-Device-Id: 3b9e...-...-... # el celular
Content-Type: application/json

# renovar: POST /api/v1/auth/refresh
{ "refreshToken": "eyJ..." }
# → par nuevo; el refresh anterior ya no sirve
```

 **EXPLICACIÓN TÉCNICA**

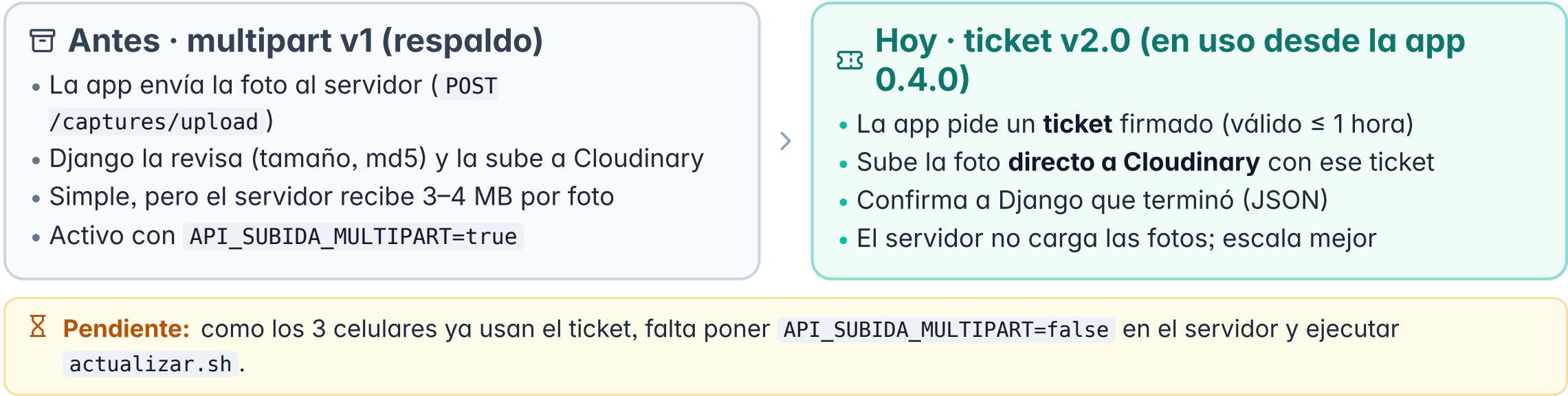
api/authentication.py valida el JWT y además revisa en cada pedido que la cuenta siga ACTIVA, que el rol pueda usar la app y que el celular no esté revocado: si no, responde 403 (ACCOUNT_BLOCKED , ROLE_NOT_ALLOWED o DEVICE_REVOKED) y la app cierra la sesión. Límites: login 10 por minuto, registro 20 por hora, renovar 60 por minuto. Al cambiar la contraseña o el correo desde la web (v1.3), los refresh se invalidan.

 **HERRAMIENTA USADA Y POR QUÉ**

 **SimpleJWT + lista negra** — **Para qué:** tokens firmados y revocables. **Por qué:** un token robado sirve como mucho 15 minutos y el refresh se puede anular.

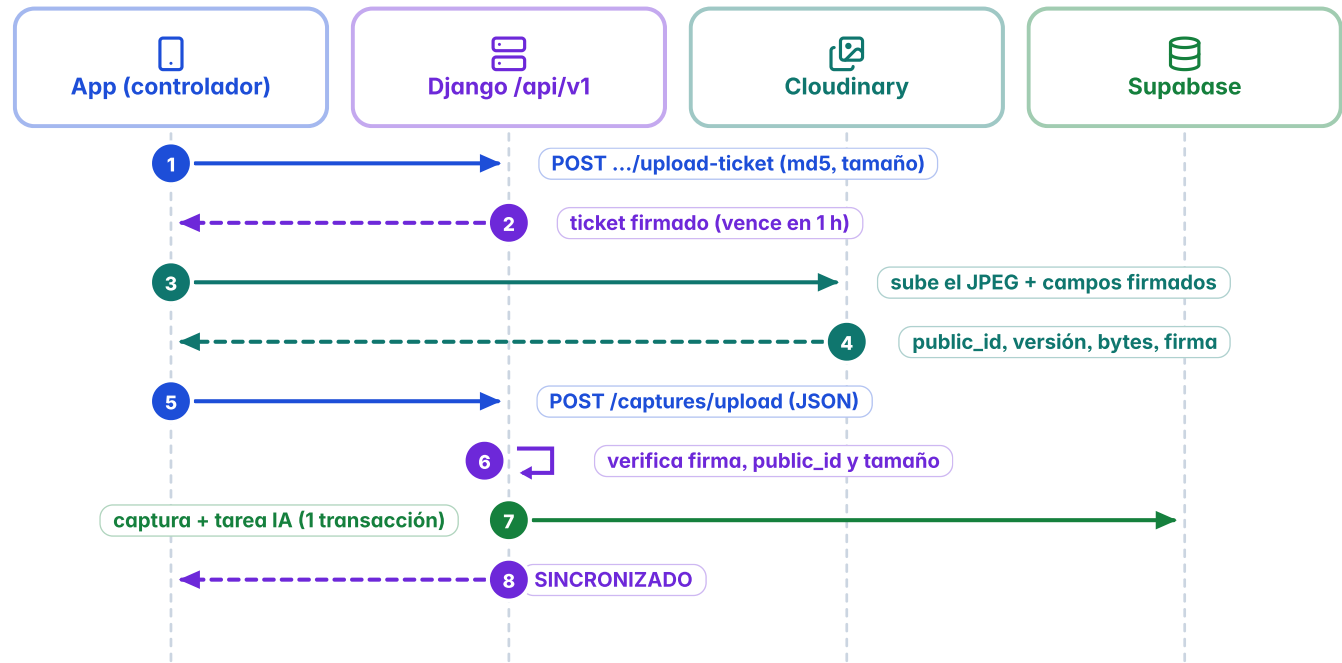
Dos formas de subir una foto

El servidor acepta las dos, para no romper una app antigua mientras se migra.



El ticket de subida, paso a paso

Así una foto llega a Cloudinary sin pasar por el servidor y sin que la app tenga claves.



```
# respuesta del ticket (resumen)
{ "captureId": "7f3c...",
  "alreadyConfirmed": false,
  "upload": {
    "url": "...cloudinary.com/v1_1/
           wd9meyw0/image/upload",
    "fields": { timestamp, public_id,
                type=authenticated, overwrite=false,
                api_key, signature },
    "expiresAt": "...+1 h", "maxBytes": 10485760 } }
```

EXPLICACIÓN TÉCNICA

La firma se calcula en el servidor con el **api_secret** (`evidencias/nube.py` , `_build_upload_ticket`); la app recibe solo el `api_key` (público) y la firma, válidos para esa foto y por 1 hora. Si la foto ya estaba confirmada con el mismo md5 y tamaño, el ticket responde `alreadyConfirmed: true` y no hay que subir nada. La app no guarda tickets: pide uno justo antes de subir (R-22) y descarta el que vence en menos de 5 min (RN-25).

La ruta nueva: GET /mobile/pest-reports

Lo que la app recibe para «Ubicar plaga» (v1.3). Es la única ruta que devuelve resultados de la IA a la app.

```
GET /api/v1/mobile/pest-reports?days=30
Authorization: Bearer ...
X-Device-Id: ...
```

```
{ "reports": [ {
  "caseId": "6d6e1ff7-...",
  "status": "CONFIRMADO_POR_IA", "origin": "IA",
  "label": "chanchito_blanco", "maxConfidence": 0.94,
  "lot": { "id": "SWG1", "code": "SWG 1", ... },
  "row": { "number": 22, "plantCount": 270 },
  "lateralCode": "LATERAL_B",
  "segment": { "startPlant": 123, "endPlant": 147 },
  "lat": -14.060594, "lon": -75.728349,
  "gpsAccuracyM": 3, "locationSource": "GPS",
  "thumbnailUrl": "https://res.cloudinary..." } ],
  "farm": { lots, rows, points, center },
  "days": 30, "serverTime": "..." }
```

 **Qué casos trae**

Confirmados por IA o por especialista, posibles plagas y en revisión de los últimos `days` (1 a 90). Nunca descartados ni evidencia insuficiente.

 **Miniatura firmada**

Una URL de la versión `miniatura` (400 px), firmada en el momento.

 **Capas del fondo**

Contornos de lotes, hileras (inicio y fin), puntos con nombre y el centro del fundo, para dibujar el mapa.

 **Solo lectura**

No cambia nada en el servidor: es una excepción explícita a la regla §28.10.

 **EXPLICACIÓN TÉCNICA**

Vista `PestReportsView` (`api/v1/views.py`), permisos `IsAuthenticated` + `PestReaders` . Las consultas salen de `review_cases` (que ya guarda lote, hilera, lado, segmento, marcador, coordenadas y su fuente) con `select_related` , sin tocar la bandeja ni la sincronización. Ejemplo con datos de la demo local.

Las reglas del contrato

Lo que app y servidor se prometen mutuamente para no romperse.

+ Agregar sí, quitar no

Se pueden agregar campos o rutas; quitar o renombrar rompería los celulares que tienen una versión anterior instalada.

Idempotente por ID

Sesiones, pasadas, secuencias, fotos e incidencias usan el UUID de la app: repetir un envío no duplica.

En lotes

Secuencias e incidencias viajan de a 200 por pedido para ahorrar conexiones en el campo.

⌚ Hora del servidor

Las respuestas traen `serverTime`: la app no depende del reloj del celular.

✉ Sin fotos ni IA hacia la app

Regla §28.10: la API no devuelve URLs de fotos ni resultados de la IA... salvo `pest-reports` (v1.3).

📖 Un solo documento

`docs/INTEGRACION_APP.md` y el maestro de la app (§15 y §28) describen el contrato; `/schema` lo confirma.

🔧 EXPLICACIÓN TÉCNICA

El contrato se prueba desde los dos lados: la plataforma tiene pruebas de API (`api/tests`) y la app corre `tools/verificacion/integracion.ts` contra la plataforma real en la laptop (30/30 con la v1.3), usando el código real de `src/` . Cambiar el contrato exige actualizar el maestro, el ADR y `INTEGRACION_APP.md` .

9



Base de datos

Qué se guarda, cómo se relaciona, cómo se protege, cuánto ocupa y en qué se diferencia de la base del celular.

39 tablas en total

Se crean solas con las migraciones; nadie las diseña a mano en Supabase.

 **27 tablas del negocio**

Las de las 8 apps: lotes, puntos del fundo, usuarios, sesiones, capturas, detecciones, casos, avisos y auditoría.

 **12 tablas técnicas**

Las que Django necesita para funcionar: migraciones aplicadas, sesiones web, permisos, tokens revocados y caché.

 **Poco espacio**

100 000 fotos ocupan ~103 MB en la base, porque las imágenes están en Cloudinary. El plan gratis tiene 500 MB.

 **Sin acceso público**

Solo Django entra, con su usuario y contraseña, por conexión cifrada.

 **Nuevo:** la v1.3 agregó la tabla `points_of_interest` (puntos con nombre del mapa): antes eran 38 tablas, ahora son **39**.

 **EXPLICACIÓN TÉCNICA**

Todas las crea `python manage.py migrate` ; nadie las diseña a mano en el Table Editor ni en el SQL Editor de Supabase. El nombre de cada tabla está fijo en su modelo (`db_table`), así coincide con el informe del curso. La base guarda coordenadas como `double precision` y las fechas en UTC (se muestran en hora de Lima).

 **HERRAMIENTA USADA Y POR QUÉ**

 **PostgreSQL** — **Para qué:** base central del piloto. **Por qué:** tiene CHECK, triggers, índices parciales y transacciones: la base misma defiende los datos.

Las 27 tablas del negocio

Agrupadas por la app de Django que las crea.

App	Tablas (27)	Qué guardan
cuentas	users · user_roles · devices · password_reset_requests	personas, roles, celulares registrados y pedidos de contraseña
campo	field_lots · field_rows · field_segments · markers · quality_profiles · points_of_interest v1.3	el fundo (con contornos GeoJSON), perfil de calidad y puntos con nombre
monitoreo	monitoring_sessions · session_devices · monitoring_passes · marker_changes · capture_sequences · incidents	lo que hizo la mochila en campo
evidencias	captures · quality_results	cada foto (sin URL) y su calidad
ia	model_configs · ai_tasks · detections	modelos, cola y cajas
revision	review_cases · human_reviews	casos y decisiones (nunca se borran)
notificaciones	notification_recipients · notification_recipient_lots · notifications	a quién avisar, de qué lotes, y cada aviso
auditoria	audit_events	quién hizo qué y cuándo (solo inserción)

🔧 EXPLICACIÓN TÉCNICA

Las tablas de monitoreo usan como clave el UUID que generó la app (sesión, pasada, secuencia, captura, incidencia): así los reintentos de sincronización no duplican. Los catálogos usan IDs legibles que genera el servidor (SWG1 , SWG1-H05 , SWG1-H05-S1). users tiene estado (PENDIENTE_APROBACION, ACTIVO, RECHAZADO, BLOQUEADO), must_change_password y la aceptación del aviso de privacidad.

Las 12 tablas técnicas

Las que Django necesita para funcionar. No se tocan a mano.

Tabla técnica	Para qué la usa Django
<code>django_migrations</code>	qué migraciones ya se aplicaron (por eso migrar dos veces no hace nada)
<code>django_session</code>	las sesiones de la web (la cookie solo guarda su identificador)
<code>django_content_type</code> · <code>auth_permission</code>	catálogo interno de modelos y permisos de Django
<code>auth_group</code> · <code>auth_group_permissions</code>	grupos de Django (los roles reales están en <code>user_roles</code>)
<code>users_groups</code> · <code>users_user_permissions</code>	relaciones de Django para el usuario propio
<code>django_admin_log</code>	cambios hechos desde /gestion/
<code>token_blacklist_outstandingtoken</code> · <code>..._blacklistedtoken</code>	refresh emitidos y revocados de la app
<code>web_cache</code>	caché compartida: bloqueo de ingreso e indicadores del panel

 **EXPLICACIÓN TÉCNICA**

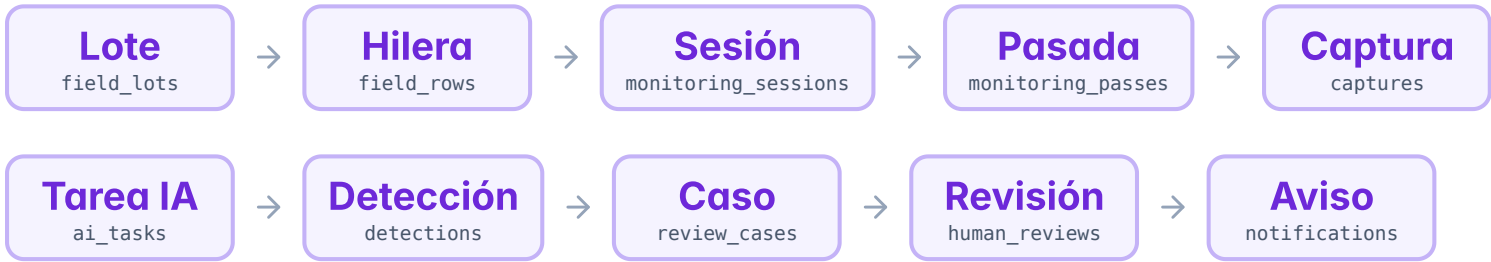
La caché en la base (`web_cache` , DW-23) permite compartir datos entre los dos procesos de gunicorn sin instalar Redis: se usa para el límite de intentos de ingreso y para guardar 60 s los indicadores del panel.
`python manage.py mantenimiento` limpia sesiones vencidas, tokens viejos y caché antigua.

 **HERRAMIENTA USADA Y POR QUÉ**

 **Caché en PostgreSQL** — **Para qué:** compartir datos entre procesos.
Por qué: evita otro servicio (Redis) en un VPS pequeño; el volumen es mínimo.

Cómo se relacionan los datos

La cadena principal: del lugar físico hasta el aviso.



🌱 **Para entenderlo:** es como un árbol genealógico: cada aviso «desciende» de una revisión, que viene de un caso, que viene de una foto tomada en una pasada de una sesión en una hilera de un lote.

🔧 EXPLICACIÓN TÉCNICA

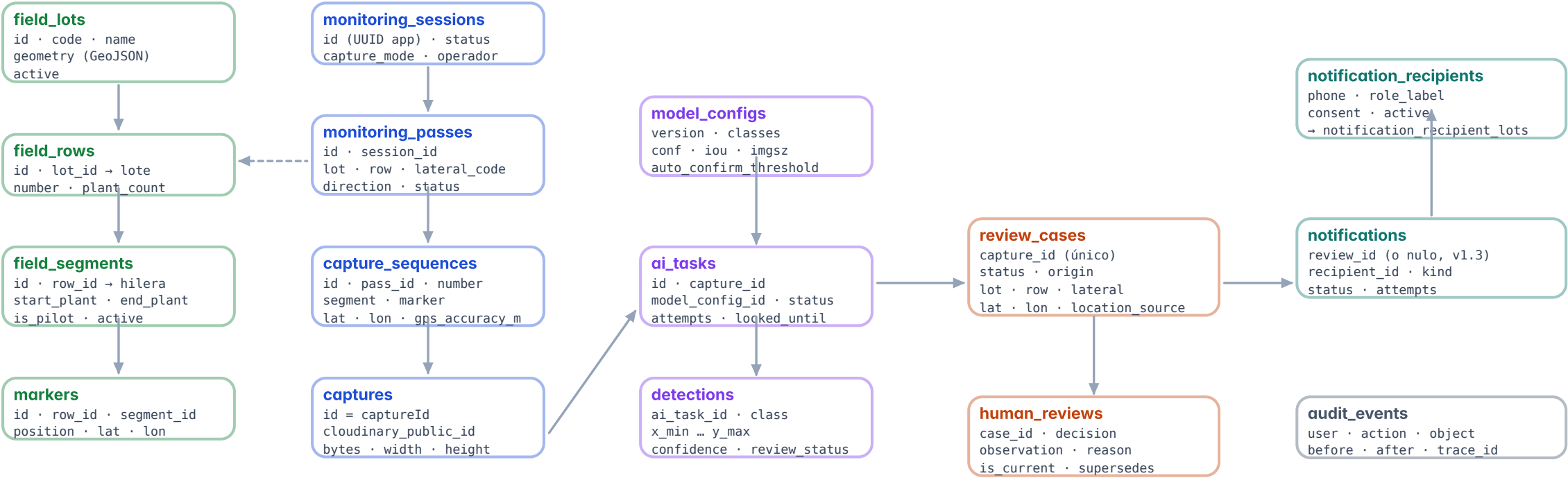
En el medio hay más eslabones: la captura pertenece a una `capture_sequences` (con el GPS, segmento y marcador) y la pasada a un `lateral_code`. El caso copia la ubicación al abrirse (lote, hilera, lado, segmento, marcador, coordenadas y fuente) para no depender de joins en la bandeja, el mapa y la API. Desde la v1.3 un aviso puede no tener revisión (`notifications.review` nulo) cuando la IA confirmó sola.

🔗 HERRAMIENTA USADA Y POR QUÉ

🔗 **Claves foráneas (ForeignKey)** — **Para qué:** unir las tablas. **Por qué:** la base impide referencias rotas; con PROTECT no se puede borrar lo que tiene historia.

El mapa de las tablas principales

Un diagrama entidad–relación simplificado del corazón del sistema.



🔧 EXPLICACIÓN TÉCNICA

Diagrama simplificado (no todas las columnas). Flechas = «pertenece a». Colores por app: verde campo, azul monitoreo y evidencias, morado IA, naranja revisión, verde azulado avisos, gris auditoría. Únicos importantes: un caso por captura, una tarea por (captura, modelo), una secuencia por (pasada, número), un aviso por (decisión, destinatario), una hilera por (lote, número).

Dentro de un caso: qué guarda review_cases

La tabla más consultada del sistema, columna por columna.

 Estado status (6 estados), origin (IA o MANUAL), notification_status (NO_APLICA, PENDIENTE_ENVIO, ENVIADO, ERROR_ENVIO).	 Lo que vio la IA ai_task, detections_count y max_confidence (la caja más segura).
 Dónde session, pass_id, sequence, lot, row, lateral_code, segment, marker, lat, lon, gps_accuracy_m, location_source.	 Cuándo y quién captured_at, opened_at, decided_at, decided_by (nulo si confirmó la IA).

 EXPLICACIÓN TÉCNICA Índices pensados para las pantallas: (status, opened_at) para la bandeja, (lot, row, status) para el plano y captured_at para filtrar por fecha. Copiar la ubicación al caso es una desnormalización a propósito: la bandeja hace exactamente 6 consultas y pest-reports no necesita recorrer secuencias. La decisión vive en human_reviews (una vigente por caso, índice único parcial); una corrección crea otra fila con supersedes.	 HERRAMIENTA USADA Y POR QUÉ  Índices de PostgreSQL — Para qué: que la bandeja y el mapa respondan rápido. Por qué: con 100 000 fotos sintéticas la bandeja sigue en ~39 ms.
--	---

¿Qué es una migración?

Un archivo que describe un cambio en la estructura de la base, en orden.



Se generan desde el código

Al cambiar un `models.py` , Django crea un archivo nuevo en `migrations/` con los cambios.



Se aplican solas

En tu PC con `iniciar.bat` ; en el servidor, al arrancar el contenedor web (`migrate`).



Cambios que agregan

Nuevas tablas o columnas: no tocan los datos existentes. Son seguras.



Cambios que quitan

Borrar o renombrar columnas puede perder datos: probar antes y hacer respaldo.

App	Migraciones (en negrita: v1.3)
cuentas	0001 · 0002 CHECK
campo	0001 · 0002 · 0003 activo · 0004 puntos del fundo
monitoreo · evidencias	0001 · 0002 CHECK
ia	0001 · 0002 · 0003 umbral y estados v1.3
revisión	0001 · 0002 · 0003 estados IA y posible plaga
notificaciones	0001 · 0002 · 0003 aviso confirmado por IA
auditoria	0001 · 0002 · 0003 caché · 0004 solo inserción



EXPLICACIÓN TÉCNICA

Antes de subir: `python manage.py makemigrations --check --dry-run` falla si alguien cambió un modelo sin crear su migración. Las migraciones `0002_restricciones_check` agregan los CHECK de estados; la `0004` de auditoría crea el trigger de solo inserción. En el VPS se aplican solas al arrancar el contenedor web, antes de gunicorn; el worker espera a que el web esté «healthy».

Cómo se protegen los datos

Varias capas, por si una falla.

 **RLS + REVOKE**

Se bloquea a los usuarios públicos de Supabase. La base no se puede leer desde internet aunque alguien tenga la URL del proyecto.

 **51 reglas CHECK**

La base rechaza estados inválidos o coordenadas imposibles, aunque el código tuviera un error.

 **Auditoría inmutable**

Un «trigger» impide editar o borrar el historial de auditoría.

 **Contraseñas con hash**

Nunca se guarda la contraseña; solo una huella (PBKDF2) que no se puede revertir.

</> Así se ve en nuestro código

```
# auditoria/seguridad_bd.py (tras cada migrate)
ALTER TABLE public."users"
  ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON ALL TABLES
  IN SCHEMA public FROM anon, authenticated;
REVOKE USAGE ON SCHEMA public
  FROM anon, authenticated;
ALTER DEFAULT PRIVILEGES ... REVOKE ALL;

# ejemplo de CHECK generado
review_cases_status_valido:
  status IN ('PENDIENTE_REVISION', ...)
```

 **EXPLICACIÓN TÉCNICA**

RLS se activa **sin políticas** en todas las tablas: los roles `anon` y `authenticated` de Supabase no ven ninguna fila, y Django (dueño de las tablas, sin `FORCE`) no queda limitado. El Security Advisor de Supabase muestra «RLS enabled, no policy» (nivel INFO): es intencional, no se crean políticas. Los CHECK de la v1.3 incluyen `model_configs_umbral_auto_valido` , `notifications_kind_valido` y `points_of_interest_kind/lat/lon_valido` .

Integridad: lo que la base garantiza sola

Reglas escritas en la propia base de datos, que se cumplen pase lo que pase en el código.

**UUID de la app**

Sesiones, pasadas, secuencias, capturas e incidencias: reenviar no duplica.

**PROTECT**

No se puede borrar una captura con caso ni un usuario con decisiones.

**UNIQUE**

`cloudinary_public_id` ; (decisión, destinatario); (captura, modelo); (pasada, número); (lote, número de hilera).

**Índices únicos parciales**

Un solo modelo de IA activo; una sola decisión vigente por caso.

**Rangos**

lat $-90 \dots 90$, lon $-180 \dots 180$, precisión ≥ 0 , confianza $0 \dots 1$, cajas con $x_{\min} < x_{\max}$, $start_{\text{plant}} \leq end_{\text{plant}}$.

**Transacciones**

Captura + tarea de IA juntas; decisión + avisos + auditoría juntas.

**EXPLICACIÓN TÉCNICA**

El principio (ADR-W-004): la base es la última línea de defensa. Si mañana alguien edita algo desde un panel o un script con un error, PostgreSQL rechaza el dato inválido. La única escritura permitida sobre `audit_events` es poner `user_id` en NULL si se elimina una cuenta (migración `auditoria/0004`).

Dos bases de datos separadas

Una para probar y otra para trabajar de verdad. Nunca se mezclan.

🧪 Desarrollo

- Proyecto de desarrollo en EE. UU. (us-west-2)
- Datos de demostración (cuentas `@demo.pe` , fotos de prueba)
- La usa tu PC con `iniciar.bat` (o SQLite si no hay `DATABASE_URL`)
- Se puede romper y volver a cargar



🚀 Piloto

- Proyecto `riachuelo-piloto` en São Paulo (sa-east-1)
- Solo datos reales, sin cuentas demo
- La usa únicamente el servidor (VPS)
- Las migraciones llegan con cada actualización

⚠️ **Importante:** nunca se mezclan datos de prueba con evidencia del piloto. `sembrar_demo` se niega a correr con `APP_ENV=piloto` (W-22), y Cloudinary separa las fotos por prefijo (`dev` / `piloto`).

🔧 EXPLICACIÓN TÉCNICA

La conexión del piloto: Supabase → Connect → Session pooler (puerto 5432, IPv4), `DB_SSLMODE=require` , contraseña de 24+ letras y números. El transaction pooler (6543) no se usa porque no admite sentencias preparadas. La cadena completa vive solo en `.env.piloto` del VPS.

Capacidad, respaldo y mantenimiento

Cuánto aguanta la base y cómo se cuida.

**Cuánto ocupa**

Medido con datos sintéticos: 100 000 fotos con análisis y ~19 000 casos ≈ **103 MB** (~1 KB por foto). El plan Free (500 MB) alcanza para ~300 000 fotos.

**Aviso de espacio**

DB_LIMITE_MB=500 : el diagnóstico avisa al llegar al 80 % (8192 si se pasa a Pro).

**Respaldo**

`python manage.py respaldo` genera respaldos/riachuelo-ENTORN0-FECHA.dump (pg_dump). Semanal en Free.

**Restaurar**

`pg_restore --no-owner --no-privileges -d "DATABASE_URL" archivo.dump` y luego `migrate .`

**Mantenimiento**

`python manage.py mantenimiento` semanal: limpia y, en Free, evita la pausa por inactividad.

**El límite real**

Está en Cloudinary: 25 créditos al mes en Free (unas 1 000 fotos ≈ 10 créditos).

**Pendiente:** el respaldo automático desde el VPS no está configurado ni probado (el contenedor es de solo lectura y necesita `pg_dump` y una carpeta con escritura). Pasar el piloto a Supabase Pro daría copias diarias.

**EXPLICACIÓN TÉCNICA**

Retención: no se borra evidencia (R-11) hasta que el equipo decida la política (pregunta abierta Q-05). Para medir rendimiento con volumen: `docs/perf/datos_sinteticos.sql` y `python manage.py medir_rendimiento .`

La base del celular y la base central

Dos bases con trabajos distintos que se complementan.

	Base del celular	Base central
Motor	SQLite (un archivo en el celular)	PostgreSQL gestionado (Supabase)
Quién la usa	Solo la app de ese celular	Solo Django (web y worker)
Para qué	Trabajar sin internet y no perder nada	La verdad oficial: lo que ve la web
Esquema	Migraciones 001–004 (versión 4)	Migraciones de Django por app
Identificadores	Los crea (UUID)	Los respeta (idempotencia)
Fotos	Archivos en la memoria del celular	Solo el public_id; el archivo en Cloudinary
Borrado	Cámaras: tras sincronizar · controlador: a mano	Nunca se borra evidencia
Seguridad	Tokens en SecureStore; nada de contraseñas	RLS + REVOKE, CHECK, auditoría, TLS

 EXPLICACIÓN TÉCNICA

Las dos bases se parecen a propósito (mismos nombres de entidades: sesión, pasada, secuencia, captura, incidencia) para que la sincronización sea casi una copia. La app tiene tablas que el servidor no necesita (`transfer_queue` , `sync_queue` , `processed_messages` , `remote_uploads`) y el servidor tiene lo que la app no ve (IA, casos, avisos, auditoría).

El panel de Supabase: qué está hecho y qué falta

Ajustes manuales que complementan la seguridad que pone el código.

 **Configuración del panel de Supabase**

- ✓ Un proyecto por entorno; el piloto en São Paulo
- ✓ Contraseña larga solo con letras y números
- ✓ Session pooler en DATABASE_URL con DB_SSLMODE=require
- ✓ Sin buckets de Storage ni publicaciones de Realtime
- ✓ Ninguna tabla creada o editada desde el panel
- ✓ Claves anon/service_role sin usar

 **Pendiente o por verificar**

- ✗ Data API desactivada o public fuera de «Exposed schemas» (verificar)
- ✗ «Enforce SSL on incoming connections» (verificar)
- ✗ Network Restrictions: solo la IP del VPS (179.198.103.82)
- ✗ Desactivar «Allow new users to sign up» en Auth (verificar)
- ✗ Plan Pro para el piloto (copias diarias, sin pausa)

 **EXPLICACIÓN TÉCNICA**

Estos ajustes no están en el código: se hacen a mano en el panel de Supabase, por eso se listan aquí. Ninguno cambia el funcionamiento de la plataforma; todos reducen la superficie de ataque. La restricción por IP es la más importante: con ella nadie fuera del VPS puede siquiera intentar conectarse.

10

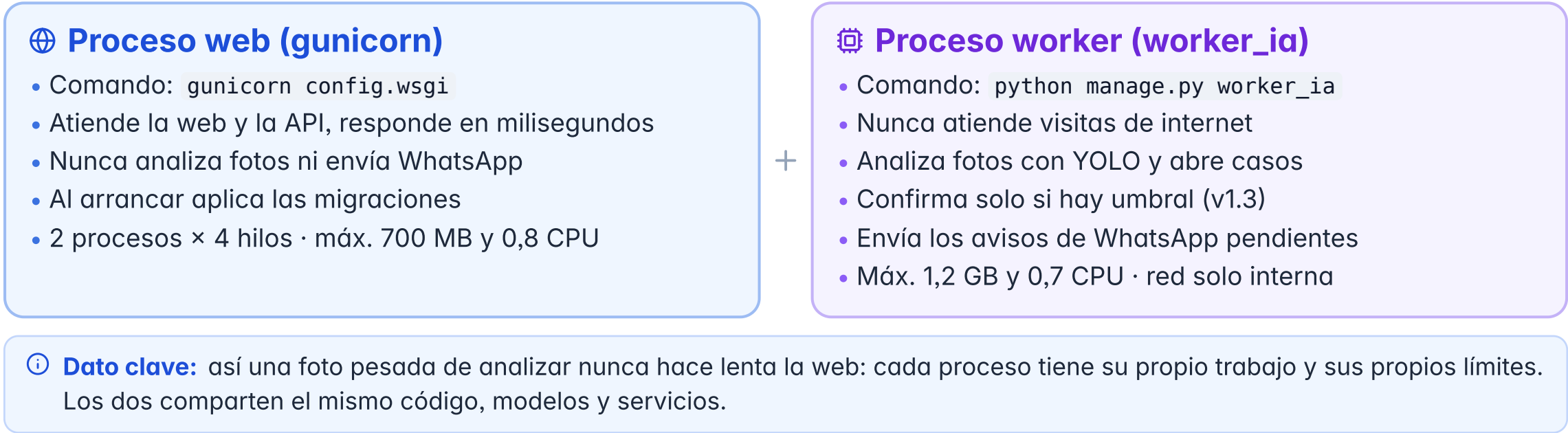


Backend en ejecución

Qué procesos corren, cómo se reparten el trabajo, cómo se configuran y cómo se diagnostican cuando algo falla.

Un solo código, dos procesos

La misma imagen de Docker arranca de dos maneras distintas.



🔧 EXPLICACIÓN TÉCNICA

La misma imagen Docker arranca de dos maneras (`command` distinto en `docker-compose.yml`). El worker solo está en la red interna (no tiene puerto publicado), espera a que el web esté «healthy» (para que las migraciones ya estén aplicadas) y tiene 30 s para terminar la foto en curso al detenerse (`stop_grace_period`). Si el VPS se reinicia, los dos se levantan solos (`restart: unless-stopped`).

🔧 HERRAMIENTA USADA Y POR QUÉ

🔧 **Un comando de Django como worker** — **Para qué:** procesar en segundo plano. **Por qué:** no hace falta Celery ni RabbitMQ: un bucle simple con la base como cola alcanza para el piloto.

La cola de análisis (sin RabbitMQ)

La cola es simplemente una tabla, `ai_tasks`, dentro de la misma base.

1

Se encola

Cuando llega una foto válida, se crea su tarea en la misma transacción: o se guardan las dos, o ninguna.

2

El worker la toma

Pide la siguiente tarea libre y la «reserva» (SKIP LOCKED) para que nadie más la tome.

3

La procesa

Descarga la foto de Cloudinary, ejecuta YOLO y guarda cajas, confianza y estado.

4

Si falla, reintenta

Un error de red no pierde la tarea: queda para otro intento (30 s y 60 s); tras 3, queda con error.

Para entenderlo: es una fila de tickets en una panadería: cada ticket se atiende una sola vez, en orden, y si el panadero se va, el ticket sigue en la fila.

EXPLICACIÓN TÉCNICA

`SELECT ... FOR UPDATE SKIP LOCKED` toma una fila sin bloquear a otros workers; la tarea se «arrienda» 5 minutos (`locked_until` , `locked_by`) y suma `attempts` . Si el worker muere, al vencer el arriendo otra vuelta la toma. `available_at` programa el reintento ($30\text{ s} \times 2^{\text{intento}-1}$). Cola ordenada por `available_at` y `requested_at` .

HERRAMIENTA USADA Y POR QUÉ

La tabla `ai_tasks` como cola — **Para qué:** repartir el análisis. **Por qué:** «encolar = guardar» en la misma transacción: imposible tener una foto sin tarea o una tarea sin foto.

La vuelta del worker, paso a paso

Lo que hace el worker una y otra vez, sin parar.



EXPLICACIÓN TÉCNICA

`ia/management/commands/worker_ia.py` . Al arrancar o al cambiar la versión activa, carga el detector configurado (`IA_DETECTOR=onnx`) una sola vez y escribe en la consola «Detector «onnx» listo» con las clases y el umbral. Entre vueltas cierra conexiones viejas con la base (`close_old_connections`). Descarga con tiempo máximo de 60 s (`IA_DESCARGA_TIMEOUT`). Cada análisis guarda `processing_ms` , `model_version` y el tamaño de la imagen analizada.

HERRAMIENTA USADA Y POR QUÉ

Pillow — **Para qué:** leer la foto y corregir su orientación. **Por qué:** las cámaras guardan la rotación en el EXIF; sin corregirla, las cajas quedarían giradas.

Cómo analiza YOLO una foto

De una foto de 12 MP a unas pocas cajas con su confianza.



EXPLICACIÓN TÉCNICA

`ia/inference/onnx.py` : «letterbox» = escalar sin deformar y rellenar con gris (114) hasta `imgsz` . La salida del modelo se decodifica en el formato crudo de YOLO `(1, 4+clases, N)` o en el formato «end2end» `(1, N, 6)` ; luego **NMS** (supresión de no máximos: si dos cajas de la misma clase se solapan más del IoU, queda la de mayor confianza). Con tiling se hace un NMS global para no duplicar cajas en los bordes. Las coordenadas vuelven a píxeles de la imagen original, que es lo que dibuja el visor de la web.

HERRAMIENTA USADA Y POR QUÉ

onnxruntime — **Para qué:** ejecutar el modelo. **Por qué:** rápido en CPU, sin GPU ni PyTorch en el servidor; el archivo pesa 9,8 MB.

Los avisos: una segunda cola

El WhatsApp también se envía desde una cola con reintentos.



🔧 EXPLICACIÓN TÉCNICA

`notificaciones/services.py` : misma técnica de cola que `ai_tasks` (SKIP LOCKED + arriendo), con `MAX_ATTEMPTS = 5` y espera de 2^{intento} minutos. Los avisos se crean en la misma transacción que la decisión, así nunca hay una decisión confirmada sin sus avisos. Hoy el cliente es `ConsoleClient` : el «envío» se escribe en el registro del worker.

La configuración: el archivo .env (1/2)

Las mismas líneas de código se comportan distinto según estas variables.

Variable	Qué controla	En el piloto
APP_ENV	Entorno: dev (tu PC) o piloto (servidor)	piloto
DJANGO_DEBUG · DJANGO_SECRET_KEY	Modo depuración · clave para firmar sesiones	false · la generó instalar.sh
ALLOWED_HOSTS · CSRF_TRUSTED_ORIGINS	Dominios aceptados	monitoreo.agricolariachuelo.org
PUBLIC_BASE_URL	Dirección pública (enlace de los avisos)	https://monitoreo.agricolariachuelo.org
DATABASE_URL · DB_SSLMODE	Dirección y clave de la base · cifrado	Supabase São Paulo · require
DB_LIMITE_MB	Límite del plan para el aviso del diagnóstico	500 (8192 en Pro)
CLOUDINARY_URL · CLOUDINARY_ENV_PREFIX	Cuenta de Cloudinary · carpeta por entorno	clave dedicada · piloto
API_SUBIDA_MULTIPART	Acepta la subida v1 (respaldo)	true (pasar a false)

 **Importante:** en el servidor este archivo se llama `deploy/.env.piloto` , tiene permisos 600 (solo root) y nunca va a GitHub. Se edita con `nano` y se aplica con `bash actualizar.sh` .

🔧 EXPLICACIÓN TÉCNICA

Existe además `deploy/.env` (sin secretos, lo genera `instalar.sh`) con las variables de Docker Compose: `DOMINIO` , `TRAEFIK_NETWORK` (riachuelo-proxy), `TRAEFIK_ENTRY_HTTPS` (websecure), `TRAEFIK_ENTRY_HTTP` (web) y `TRAEFIK_CERTRESOLVER` (letsencrypt). En la PC, `.env` se crea desde `.env.example` con una clave nueva.

La configuración: el archivo .env (2/2)

Variables de la IA, de WhatsApp y de los registros.

Variable	Qué controla	En el piloto
IA_DETECTOR · MODEL_PATH	Qué IA usar y dónde está el modelo	onnx · modelos/modelo.onnx
IA_DESCARGA_TIMEOUT	Espera máxima al bajar una foto	60 s
WHATSAPP_CLIENT	Enviar de verdad o solo escribir en consola	ConsoleClient (pendiente)
WHATSAPP_TOKEN · _PHONE_ID · _GRAPH_VERSION	Credenciales de Meta	vacías
WHATSAPP_TEMPLATE · _LANG	Plantilla aprobada e idioma	caso_confirmado · es
WHATSAPP_TEMPLATE_IA v1.3	Plantilla para «Confirmado por IA»	opcional (por defecto la misma)
LOG_LEVEL · LOG_REQUESTS · LOG_SLOW_MS	Cuánto registrar y desde cuándo un pedido es «lento»	INFO · true · 1500
LOG_COLOR	Colores en la consola	0

 **EXPLICACIÓN TÉCNICA**

En desarrollo (APP_ENV=dev) los valores por defecto cambian: IA_DETECTOR=simulado (20 % de fotos con indicio, IA_SIMULADO_PROBABILIDAD), Cloudinary simulado si CLOUDINARY_URL está vacío, SQLite si DATABASE_URL está vacío y WhatsApp en consola. Así se prueba el flujo completo sin ninguna cuenta externa.

 **HERRAMIENTA USADA Y POR QUÉ**

 **django-environ** — **Para qué:** leer el .env con tipos. **Por qué:** convierte texto en números, listas y booleanos, y evita escribir secretos en el código.

Errores y diagnóstico

Cuando algo falla, el sistema lo dice en lenguaje claro.

☰ Una línea por pedido

La consola muestra cada visita con su tiempo y su `traceId`. Avisa si una página tarda más de 1,5 s.

🌐 Errores del navegador

Si falla un JavaScript o no carga una foto, el navegador lo reporta y aparece en la consola del servidor.

🔧 Comando diagnostico

Revisa base, migraciones, RLS, TLS, trigger, CHECK, espacio, Cloudinary, IA, cola, WhatsApp y cuentas; dice qué está mal y cómo arreglarlo.

🛡 Bloqueos de seguridad

En piloto, Django no arranca si falta Cloudinary, si DEBUG está activo o si el dominio no está configurado.

</> Qué revisa (salida ilustrativa)

```
# docker compose exec web python manage.py diagnostico
[ok]
Base de datos: PostgreSQL, TLS activo
[ok]
Migraciones al día · 51 CHECK
[ok]
RLS y REVOKE en todas las tablas
[ok]
Trigger de auditoría activo
[ok]
Cloudinary: cuenta y prefijo «piloto»
[ok]
IA: modelo 2026.10-int activo
[!!] Cuentas: 0 especialistas activos
    → crea uno en Administración → Usuarios

# resultado real del 08/10: 24 correctos, 1 error
```

🔧 EXPLICACIÓN TÉCNICA

El middleware de `diagnostico/` asigna el `traceld` y escribe la línea de cada pedido; `diagnostico/checks.py` y `arranque.py` frenan el arranque en piloto si la configuración es peligrosa. Otros comandos: `comprobar_bd` (explica errores de conexión en lenguaje claro) y `diagnostico --red` (incluye pruebas de red). El ejemplo de la derecha está resumido; el error real del 08/10 fue «0 especialistas activos».

Rendimiento medido

Cuánto tarda cada pantalla con 100 000 fotos en la base.

Pantalla	Mediana (ms)	p95 (ms)	Consultas	Tamaño
Bandeja (pendientes, página 1)	39	42	6	25 KB
Bandeja (todos, por confianza, página 50)	60	70	6	25 KB
Caso	46	50	11	8 KB
Mapa: datos sin filtros (tope 5 000 puntos)	658	719	6	258 KB (gzip)
Mapa: un lote, confirmados	254	292	6	91 KB (gzip)
Plano (SWG 1)	26	27	10	19 KB
Panel (30 días, con caché)	5	6	4	5 KB
Panel (30 días, sin caché)	≈ 280	≈ 300	21	5 KB

Dato clave: la meta del curso (RNF04) es responder en menos de 1,5 s con más de 50 000 plantas; la web la cumple con holgura incluso con 100 000 fotos. Meta interna: bandeja y caso < 300 ms.

 **EXPLICACIÓN TÉCNICA**

Medición de referencia del 04/10/2026 en local: 100 576 capturas, 93 872 tareas de IA y 18 798 casos (4 759 pendientes), PostgreSQL 16, 12 pedidos por pantalla. Cómo se logra: número fijo de consultas, ubicación copiada al caso, índices, paginación, GeoJSON con tope y gzip, panel en caché 60 s. En Supabase cada consulta cruza la red (1–3 ms en la misma región): falta repetir la medición en el piloto.

 **HERRAMIENTA USADA Y POR QUÉ**

 **medir_rendimiento** — **Para qué:** medir tiempos con muchos datos.
Por qué: con docs/perf/datos_sinteticos.sql se prueba el peor caso antes de que ocurra.

11

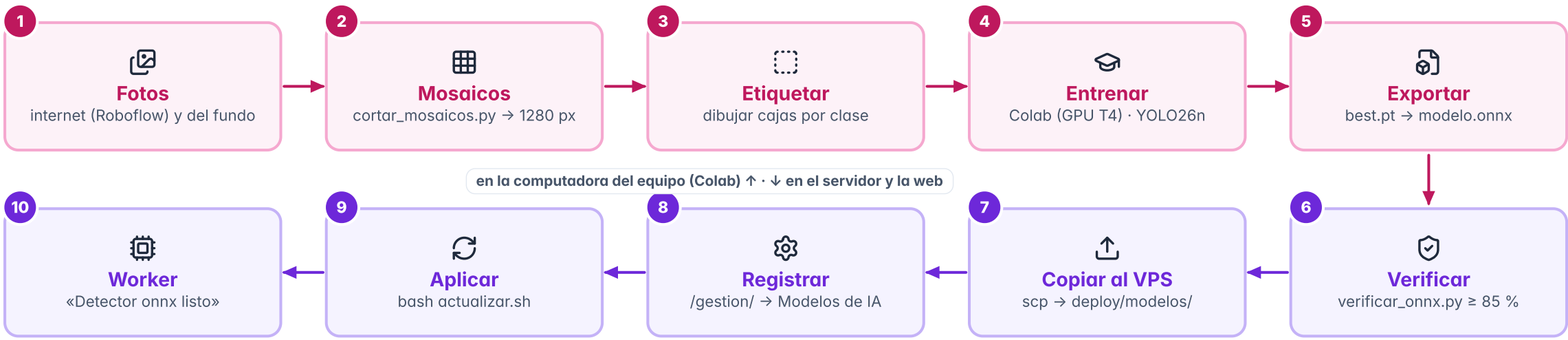


La IA en detalle

Cómo se entrena el modelo, cómo se mide si es bueno, cómo se activa en el servidor y qué falta para el modelo del piloto.

Del entrenamiento al servidor

El camino completo de un modelo: de fotos con cajas dibujadas a mano hasta el worker del VPS.



🔧 EXPLICACIÓN TÉCNICA

Decisiones del modelo v1: **YOLO26n** (Ultralytics 8.4+), clases en orden `["chanchito_blanco", "melaza_fumagina"]`, `imgsz 640` entrenado sobre recortes de 1280 px y el mismo tiling en producción (`{"tile": 1280, "overlap": 0.2}`). Se exporta a ONNX **con salida cruda** (sin NMS dentro del modelo): el NMS lo hace nuestro código, igual en la verificación y en el worker. Datos en Google Drive (`riachuelo_yolo/externas` y `propias`).

🔗 HERRAMIENTA USADA Y POR QUÉ

📄 **Google Colab (GPU T4)** — **Para qué:** entrenar sin comprar una GPU. **Por qué:** gratis por sesiones; el notebook `tools/ia/entrenar_yolo_colab.ipynb` repite siempre los mismos pasos.

Qué busca el modelo y con qué aprende

Dos clases y dos fuentes de fotos: de internet para empezar, del fondo para el piloto.

 **chanchito_blanco**

Planococcus ficus: colonias blancas y algodonosas en racimos, hojas y tronco. Es la plaga objetivo del proyecto.

 **melaza_fumagina**

La melaza pegajosa que deja el chanchito y el hongo negro (fumagina) que crece encima: un indicio visible a distancia.



 **Datos de internet**

Roboflow: «Mealybug by Grape V3» y «Mealybug by project404». Sirven para arrancar.

 **Datos propios**

Fotos del fondo tomadas con la app: SWG5 H31–H36 y SWG2 H01, H02 y H07, del 10 al 19 de octubre.

 **Validación honesta**

Sin fotos propias, el 15 % de las externas va a validación por grupo (no se mezclan recortes de una misma foto).

 **EXPLICACIÓN TÉCNICA**

El orden de las clases importa: el modelo devuelve un número (0, 1) y `model_configs.classes` lo traduce. Si el orden en `/gestion/` no es el del entrenamiento, el sistema llamaría «melaza» al chanchito. Una clase nueva se muestra legible sin cambiar la app (`labelText`). Las fotos rechazadas por calidad no se usan como «negativo» sin una decisión explícita (RN-14).

 **HERRAMIENTA USADA Y POR QUÉ**

 **Roboflow** — **Para qué:** conseguir datos etiquetados para empezar. **Por qué:** hay datasets públicos de chanchito; con ellos el sistema se pudo probar antes de tener fotos propias.

Cómo se mide si un modelo es bueno

Tres números que hay que saber leer, con los valores del modelo de integración.

Precisión (P)

0,69

De cada 100 cajas que dibuja, unas **69** son correctas. El resto son falsas alarmas.

Exhaustividad (R)

0,46

De cada 100 chanchitos que hay en las fotos, encuentra unos **46**. Se le escapan muchos.

mAP50

0,50

Una nota global de 0 a 1 que combina las dos, aceptando una caja si coincide al menos 50 % con la real.

⚠ **Importante:** estos números son del **modelo de integración** (fotos de internet, ~96 épocas, validación externa). Sirve para probar el sistema de punta a punta, no para decidir en el piloto. El modelo v1 se medirá con fotos del fondo.

🌟 **Para entenderlo:** la precisión es «cuando grita ¡plaga!, cuántas veces tiene razón»; la exhaustividad es «de todas las plagas que había, cuántas vio». Un buen vigilante necesita las dos.

🔧 EXPLICACIÓN TÉCNICA

Criterio de activación del modelo real: `verificar_onnx.py` con fotos de prueba del fondo debe detectar algo en **al menos 85 %** de las fotos que sí tienen plaga. El mismo script mide tiempos y sugiere `conf_threshold` . Con la IA como «filtro» que ordena el trabajo, importa más no perder plagas (exhaustividad) que evitar todas las falsas alarmas: el especialista descarta las falsas.

El modelo activo hoy

Lo que está corriendo en el servidor del piloto desde el 08/10/2026.

✓ Qué ya funciona

El worker carga el modelo y analiza las fotos que llegan; abre casos con cajas y confianza; la web los muestra con el aviso «indicio sugerido por IA».

⚠ Por qué no es el del piloto

Aprendió con fotos de otros viñedos y otras cámaras: en el fondo puede equivocarse más. Por eso el umbral de autoconfirmación sigue apagado.

🔄 Qué pasa al cambiarlo

Al activar el v1 con otra versión, el worker recarga el detector y vuelve a encolar las fotos que no tengan análisis de ese modelo.

⚙ Configuración activa en /gestion/

```
name          yolo-chanchito-melaza
version        2026.10-int  (integración)
arquitectura   YOLO26n · ONNX · 9,8 MB
classes        chanchito_blanco, melaza_fumagina
conf · iou      0.25 · 0.45
imgsz · tiling  640 · 1280 / 0.2
autoconfirmación  apagada (vacío)
active         sí
```

```
# en el VPS:
/srv/riachuelo/deploy/modelos/modelo.onnx
```

⚙ EXPLICACIÓN TÉCNICA

Hecho el 08/10: herramientas subidas a GitHub (rama prueba2 , «Herramientas de entrenamiento YOLO»), comando exportar_dataset , analizar_imagen() en onnx.py , modelo activado en el VPS y diagnóstico con 24 correctos y 1 error (0 especialistas activos). No se reutiliza la versión 2026.10-v1 en Drive porque ya tiene un entrenamiento terminado: el v1 real llevará otra versión.

Tres números que gobiernan la IA

Qué hace cada umbral del modelo, con un dibujo para cada uno.

 **conf_threshold · 0,25**

0 %

minimo 0,25

100 %

0,12

descarta

0,41

caja

0,88

caja

 **iou_threshold · 0,45**

0,81

solape

0,63

si el solape > 45 %,
queda solo la de 0,81

 **auto_confirm_threshold · vacío**

0 %

umbral (ejemplo 0,80)

100 %

Vacío = apagado. Con umbral, si la caja más segura lo alcanza, el caso nace «Confirmado por IA» y avisa por WhatsApp.

 **EXPLICACIÓN TÉCNICA**

Los tres viven en `model_configs` y se editan por modelo en `/gestion/`. Subir `conf_threshold` reduce falsas alarmas pero pierde plagas; bajarlo hace lo contrario.

`iou_threshold` solo afecta cajas que se pisan. `auto_confirm_threshold` (v1.3, CHECK `model_configs_umbral_auto_valido` : entre 0 y 1 o vacío) es el único que cambia el flujo de decisión: por eso nace vacío.

Cómo se activa o se cambia un modelo

Cinco pasos, siempre iguales, para el modelo v1 y los que vengan después.

1



Exportar

```
yolo export
model=best.pt format=onnx
imgsz=640 (en la máquina
del equipo de IA).
```

2



Verificar

```
verificar_onnx.py --
modelo modelo.onnx --
imagenes fotos_prueba : ≥
85 % y tiempos
razonables.
```

3



Copiar

```
scp modelo.onnx
root@179.198.103.82:/srv/
riachuelo/deploy/modelos
/ (no va a Git).
```

4



Registrar

```
/gestion/ → Modelos de
IA: nombre, versión única,
clases en orden, umbrales,
imgsz , tiling, activo.
```

5



Aplicar

```
cd /srv/riachuelo &&
bash
deploy/actualizar.sh ;
luego mirar docker
compose logs -f worker .
```

 **Dato clave:** solo puede haber **un modelo activo** (índice único parcial). Al marcar uno nuevo se desmarca el anterior; el worker detecta el cambio de versión, recarga y encola las fotos pendientes de ese modelo.

 **EXPLICACIÓN TÉCNICA**

Opcional: `weights_sha256` (huella del archivo, `sha256sum modelo.onnx`) para comprobar que es el archivo correcto. En la pantalla IA de la web (`/ia/`) se ve la cola, los tiempos y las tareas con error; el administrador puede reencolarlas (`TAREA_IA_REENCOLADA` en la auditoría). Vigilar `processing_ms` y la RAM (`docker stats`): el worker tiene 1,2 GB.

 **HERRAMIENTA USADA Y POR QUÉ**

`> scp y SSH` — **Para qué:** llevar el modelo al VPS. **Por qué:** los pesos no viajan por GitHub (son grandes y no son código); se copian directo y con cifrado.

Cómo mejora la IA con el uso

El trabajo diario del especialista produce los datos para el siguiente modelo.



EXPLICACIÓN TÉCNICA

```
python manage.py exportar_dataset --modo revisados
```

 exporta fotos con sus cajas según la decisión del especialista (`detections.review_status`), listas para etiquetar o entrenar. En el VPS se ejecuta con `docker compose run --rm -v /srv/riachuelo/datasets:/salida web ...` porque el contenedor es de solo lectura; luego se baja el `.tgz` con `scp` .

Lo que la IA nunca hace

Límites escritos en las reglas y en el código.

**No diagnostica**
Dice «indicio sugerido por IA». Nunca «plaga detectada», «infestación» ni «severidad» (W-11).

**No indica gravedad**
Los colores muestran el estado de revisión, nunca urgencia (W-12).

**No receta**
Sin tratamientos, dosis, calendarios ni recomendaciones agronómicas (W-13, R-17).

**No avisa sola por defecto**
Solo con umbral activado por modelo; si no, avisa el especialista.

**No juzga fotos malas**
Solo analiza fotos aceptadas por calidad; UTILIZABLE no significa «sin plaga».

**No cambia decisiones**
Un análisis nuevo nunca cambia una decisión ya tomada (RN-W06).

**EXPLICACIÓN TÉCNICA**
En la web, el aviso `messages.IA_AVISO` («Indicio sugerido por IA: no es un diagnóstico. La decisión es del especialista») aparece en la bandeja y en cada caso de origen IA. En la app, «Ubicar plaga» muestra la clase y el porcentaje como «Indicio de la IA: 93 %», nunca como diagnóstico.

El calendario de la IA

Del 10 de octubre al 20 de noviembre de 2026.



Hito	Fecha	Condición para avanzar
Fotos propias capturadas	10–19 oct	Hileras SWG5 H31–H36 y SWG2 H01, H02, H07, con la app en el piloto
Modelo v1 activo	2 nov	<code>verificar_onnx.py</code> detecta ≥ 85 % de las fotos con plaga
Postest	9–18 nov	App y modelo congelados; mismos 12 segmentos del pretest
Modelo v2	14–17 nov	Reentrenar con las decisiones del especialista (sin tocar el postest)

EXPLICACIÓN TÉCNICA

Fechas del plan del equipo (al 08/10/2026): captura 10–19 oct, etiquetado 13–25 oct, entrenamiento v1 26–31 oct, activación del v1 el 2 nov si cumple el 85 %, postest 9–18 nov (12 segmentos), v2 con las decisiones del especialista 14–17 nov y cierre el 20 nov. El postest usa una versión congelada de la app y del modelo.

HERRAMIENTA USADA Y POR QUÉ

Plan por fechas — **Para qué:** llegar al postest con un modelo validado. **Por qué:** el modelo v1 debe estar activo antes de medir; el v2 no cambia el postest en curso.

12

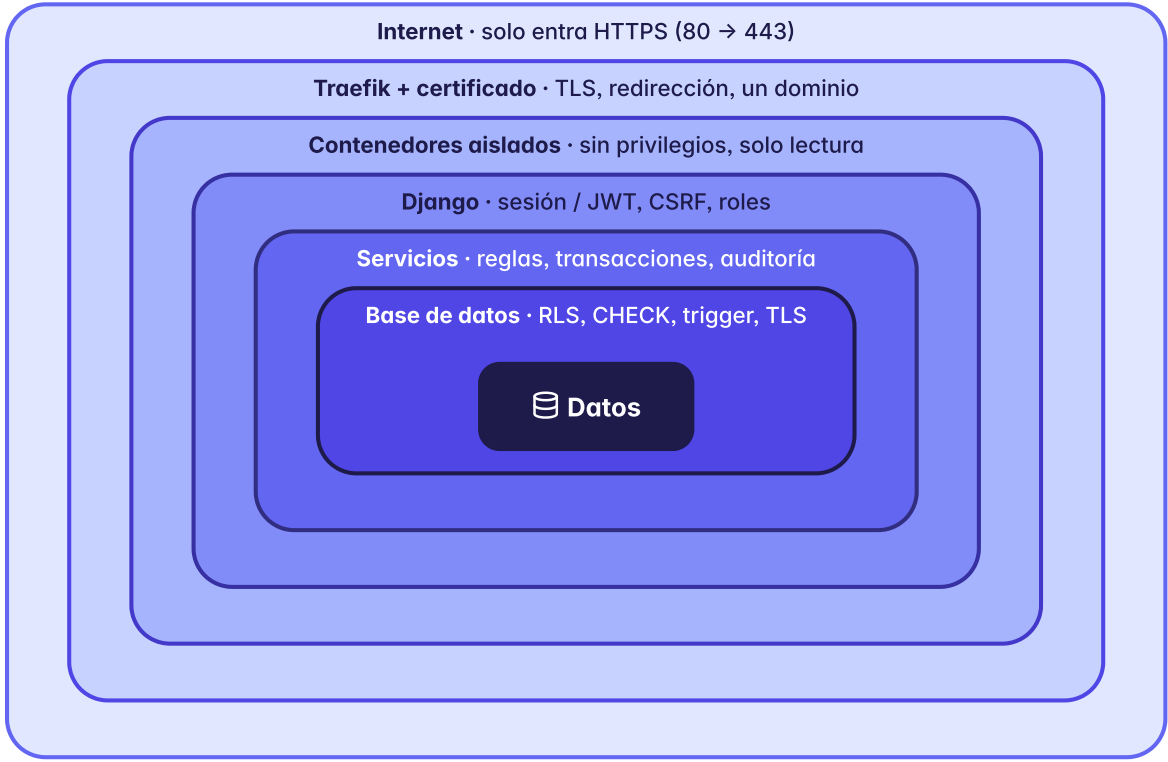


Seguridad

Cómo se protegen las cuentas, los datos, las fotos y el servidor, capa por capa, y qué falta reforzar.

Seguridad por capas

Como una cebolla: para llegar a los datos hay que atravesar todas las capas.



Defensa en profundidad

Cada capa supone que la anterior puede fallar: si alguien roba un token, igual choca con permisos, reglas y la base.

Mínimo privilegio

Cada pieza solo tiene el acceso que necesita: la app nunca tiene claves; el worker no recibe visitas.

Cifrado en el camino

Celular, navegador y servidor hablan por HTTPS; el servidor y Supabase, con TLS obligatorio (`DB_SSLMODE=require`).

Todo queda registrado

Cada pedido tiene traceld y cada acción importante queda en la auditoría.

EXPLICACIÓN TÉCNICA

Las capas no dependen unas de otras: los permisos están en la vista (`@web_view`) **y** en el servicio (RN-W01); los estados válidos en el código **y** en la base (CHECK); el acceso a Supabase lo bloquea la contraseña **y** RLS + REVOKE. Así un error en un lugar no se convierte en un agujero.

Cuentas y contraseñas

Quién puede entrar y cómo se protege su contraseña.

 **Contraseñas con hash**
Se guardan con PBKDF2 (miles de vueltas de cálculo): ni el administrador ni la base pueden ver la contraseña.

 **Política**
8 o más caracteres, con letras y números, sin espacios al inicio o al final; no parecida al usuario ni común. Igual en la web y en la app.

 **Intentos limitados**
Web: 5 fallos → bloqueo de 15 min.
API: login 10 por minuto. App sin internet: 5 intentos → 15 min.

 **Contraseña temporal**
Las cuentas nuevas (o con contraseña asignada) deben cambiarla al primer ingreso.

 **Solo el administrador**
Aprueba, bloquea, edita datos de ingreso y asigna roles. Nadie cambia su propia cuenta (RN-W14).

 **Sin registro público web**
En la web no hay «crear cuenta»: las crea el administrador o se piden desde la app.

 **EXPLICACIÓN TÉCNICA**

Validadores de Django: `UserAttributeSimilarityValidator` , `MinimumLengthValidator` (8), `CommonPasswordValidator` , `NumericPasswordValidator` y el propio `LettersAndDigitsValidator` (`cuentas/validators.py` , misma regla que `validatePassword` de la app). El bloqueo de ingreso web usa la caché en la base (`LOGIN_MAX_FAILED=5` , `LOGIN_LOCKOUT_MINUTES=15`). La contraseña temporal se muestra **una sola vez** para entregarla en persona; la auditoría guarda solo el hecho, nunca la contraseña.

 **HERRAMIENTA USADA Y POR QUÉ**

 **PBKDF2 de Django** — **Para qué:** guardar contraseñas. **Por qué:** es el estándar de Django, lento a propósito para que adivinar contraseñas sea muy caro.

Sesiones y tokens

Cómo se mantiene a una persona identificada sin pedirle la contraseña a cada rato.



Protecciones de la web

Las defensas que trae la plataforma contra los ataques más comunes.

Protección	Cómo	Para qué
HTTPS obligatorio	<code>SECURE_SSL_REDIRECT</code> + Traefik 80 → 443	nadie usa la web sin cifrado
HSTS	<code>SECURE_HSTS_SECONDS</code> (3600 por defecto; 31536000 por variable)	el navegador recuerda usar solo HTTPS
Anti-clickjacking	<code>X_FRAME_OPTIONS = "DENY"</code>	nadie puede meter la web dentro de otra página
Tipo de contenido	<code>SECURE_CONTENT_TYPE_NOSNIFF</code>	el navegador no «adivina» tipos peligrosos
Referer	<code>strict-origin-when-cross-origin</code>	no se filtran rutas internas a otros sitios
Dominios aceptados	<code>ALLOWED_HOSTS</code> · <code>CSRF_TRUSTED_ORIGINS</code>	solo responde a su dominio
Sin JavaScript en línea	W-17 + prueba automática	lista para una CSP estricta; menos superficie XSS
Sin CDN	HTMX, Leaflet, fuente e íconos locales (W-01)	ningún tercero puede cambiar el código que se ejecuta
Permisos	<code>@web_view</code> + servicios	ocultar un botón nunca es la única protección
Plantillas de Django	escapan todo el texto por defecto	un nombre con código no se ejecuta

 **EXPLICACIÓN TÉCNICA**

Configuración en `config/settings.py` . `SECURE_PROXY_SSL_HEADER` confía en `X-Forwarded-Proto` que pone Traefik (gunicorn arranca con `--forwarded-allow-ips=*` porque solo Traefik llega al contenedor). HSTS se limita al subdominio de la plataforma para no afectar otros servicios del dominio de Hostinger. Las consultas usan el ORM: sin SQL armado a mano, sin inyección SQL.

Seguridad de la app

Lo que la app hace para no ser el eslabón débil.

 **Sin claves en la app**

Ni Supabase, ni el secreto de Cloudinary, ni WhatsApp. Las variables `EXP0_PUBLIC_*` no son secretas (R-18, R-21).

 **Almacén seguro**

Tokens y verificador sin internet en SecureStore (Keystore de Android). Nunca en SQLite ni en los registros.

 **Permisos de un solo uso**

Para subir una foto, un ticket firmado para esa foto y por 1 hora (R-22).

 **Celulares revocables**

Cada celular tiene su `deviceId`; el administrador lo revoca desde la web.

 **Vinculación con código secreto**

El QR lleva un código aleatorio que vale hasta cerrar la sesión o generar otro; cada mensaje local se valida (zod) y se procesa una sola vez.

 **Diagnóstico sin secretos**

El archivo de diagnóstico no lleva contraseñas, tokens ni el verificador.

 **EXPLICACIÓN TÉCNICA**

La red local de la mochila funciona sin internet y solo dentro del Wi-Fi del campo; el emparejamiento exige el código del QR (`pairingTokenBytes = 16`) y la misma versión de protocolo; por regla, los tres celulares llevan la misma versión de app (RN-17). El login sin internet usa un verificador PBKDF2 (40 000 vueltas) con 5 intentos y bloqueo de 15 min (RN-03). «Ubicar plaga» solo lee: no puede cambiar nada en el servidor.

 **HERRAMIENTA USADA Y POR QUÉ**

 **expo-secure-store** — **Para qué:** guardar secretos en el celular. **Por qué:** usa el almacén cifrado del sistema operativo, no un archivo común.

Fotos y secretos

Las fotos son evidencia privada; los secretos viven en un solo lugar.

Fotos privadas

Tipo `authenticated` : sin una URL firmada por Django la foto no se ve, aunque alguien conozca su nombre.

Tamaños cerrados

Solo «miniatura» y «revisión» (y la original a pedido). Con «Strict transformations» nadie puede pedir otros tamaños ni gastar créditos.

URLs firmadas al mostrar

Se generan en el momento y nunca se guardan en la base ni en los registros (W-10).

El WhatsApp no lleva la foto

Lleva el enlace al caso, que pide iniciar sesión (DW-26).

Dónde viven los secretos

- ✓ `/srv/riachuelo/deploy/.env.piloto` en el VPS, permiso 600 (solo root)
- ✓ `.env` en la PC de desarrollo (excluido por `.gitignore`)
- ✗ Nunca en GitHub, en la app, en el navegador, en un chat ni en una captura
- ✗ Nunca las claves `anon` o `service_role` de Supabase

Secreto	Qué abre
<code>DJANGO_SECRET_KEY</code>	firma sesiones, tokens y CSRF
<code>DATABASE_URL</code>	la base de Supabase (incluye la contraseña)
<code>CLOUDINARY_URL</code>	<code>api_key</code> + <code>api_secret</code> de las fotos
<code>WHATSAPP_TOKEN</code>	enviar mensajes desde el número de la empresa

⚠ **Importante:** si una clave se filtra: rotarla en el panel del servicio (Supabase, Cloudinary, Meta), pegar la nueva en `.env.piloto` y ejecutar `bash actualizar.sh` .

EXPLICACIÓN TÉCNICA

En el plan Free de Cloudinary las URLs firmadas no caducan; por eso el aviso enlaza al caso (con login) y no a la imagen. La base guarda solo `cloudinary_public_id` , versión, bytes, formato y tamaño. El servidor verifica la firma de la respuesta de Cloudinary antes de aceptar una foto (`UPLOAD_SIGNATURE_INVALID`).

Auditoría: quién hizo qué, y cuándo

Un registro que solo crece: nunca se edita ni se borra.

Tema	Eventos que se registran
Ingresos	INGRESO_WEB · INGRESO_APP
Cuentas	CUENTA_REGISTRADA · CUENTA_CREADA · CUENTA_APROBADA · CUENTA_ESTADO · ROLES_CAMBIADOS · CUENTA_EDITADA · CONTRASENA_TEMPORAL · CONTRASENA_ASIGNADA · CONTRASENA_CAMBIADA_WEB · CONTRASENA_CAMBIADA_APP
Celulares y avisos	CELULAR_REVOCADO · DESTINATARIO_GUARDADO · AVISO_SIN_DESTINATARIOS
Campo	SESION_RECIBIDA · PASADA_RECIBIDA · SESION_CERRADA · CAPTURA_CONFIRMADA
Casos e IA	CASO_ABIERTO · CASO_NUEVO_ANALISIS · CASO_CONFIRMADO_POR_IA · CASO_DECIDIDO · CASO_CORREGIDO · ANALISIS_POSTERIOR_A_DECISION · TAREA_IA_REENCOLADA
Catálogos y mapa	CATALOGO_CREADO · _EDITADO · _DESACTIVADO · _REACTIVADO · CATALOGO_LOTE_HILERAS · LOTE_CONTORNO · MARCADOR_UBICADO · PUNTO_CREADO · _EDITADO · _ELIMINADO
Reportes	EXPORTACION_CASOS

Un evento por dentro (ejemplo ilustrativo)

```
{
  "timestamp": "2026-10-08 09:14:03",
  "user": "especialista (id 12)",
  "action": "CASO_DECIDIDO",
  "entity": "case · 7f3c...",
  "before": {"status": "PENDIENTE_REVISION"},
  "after": {"status": "CONFIRMADO_POR_ESPECIALISTA"},
  "trace_id": "b81e..."
}
```

🔧 EXPLICACIÓN TÉCNICA

Cada evento guarda usuario (o nulo si fue el sistema), acción, objeto, valores antes y después y el `trace_id` del pedido, **en la misma transacción** que el cambio (W-15). Un trigger de PostgreSQL (`audit_events_solo_insercion`) rechaza UPDATE y DELETE; la única excepción es poner `user_id` en NULL si se elimina una cuenta. La pantalla Auditoría (solo administrador) permite filtrar y revisar.

🔗 HERRAMIENTA USADA Y POR QUÉ

📖 **Trigger de PostgreSQL** — **Para qué:** que el historial no se pueda alterar. **Por qué:** ni siquiera un error del código o un administrador con el panel puede borrar la historia.

El servidor y su aislamiento

Aunque algo saliera mal dentro de un contenedor, el daño queda encerrado.



Sin privilegios

Corren como el usuario `riachuelo` (uid 10001), no como root.

Solo lectura

El disco del contenedor no se puede escribir; `/tmp` en memoria (64 MB).

Sin capacidades

`cap_drop: ALL` y `no-new-privileges`: aunque alguien entre, no puede escalar.

Redes separadas

Solo el web está en la red del proxy; el worker vive en la red interna.

Límites

RAM y CPU con tope: un problema no tumba al resto del VPS.

Registros con tope

10 MB × 3 archivos por contenedor: el disco no se llena.

Pendiente: activar el firewall del VPS (ufw) dejando solo 22, 80 y 443 —con cuidado de no cortar el SSH—: hoy está inactivo y un puerto de otro servicio (3000) está expuesto.

EXPLICACIÓN TÉCNICA

Todo esto está en `deploy/docker-compose.yml` (bloque común `x-comun`). La imagen no contiene secretos (`.dockerignore` deja fuera `.env`, `.venv`, `media` y la base local); los secretos llegan como variables desde `.env.piloto`. El modelo se monta de solo lectura. La plataforma no modifica Traefik ni los otros servicios del VPS.

Reglas de trabajo seguro

La seguridad también depende de cómo trabaja el equipo.

**Secretos fuera del chat**

Nunca se pegan claves en un chat, un correo, un commit o una captura de pantalla.

**La clave de root no se comparte**

La escribe solo quien tiene el acceso: ni una IA ni un compañero la piden o la guardan.

**Solo por Git**

El código nunca se edita en el VPS: PC → GitHub → VPS. Allí solo se tocan `.env.piloto` y `modelos/`.

**Sin datos de prueba**

En el piloto `sembrar_demo` está prohibido (W-22): solo datos reales.

**Nada a mano en Supabase**

Ni tablas, ni políticas RLS, ni datos desde el panel: todo por migraciones.

**No tocar lo que no es nuestro**

Traefik y los otros servicios del VPS no se modifican.

 Si una clave se filtra (paso a paso)

1

**Avisar**

al responsable del proyecto, sin reenviar la clave

2

**Rotar**

generar una nueva en el panel del servicio (Supabase, Cloudinary o Meta)

3

**Reemplazar**

pegarla en `.env.piloto` del VPS (lo hace quien tiene el acceso)

4

**Aplicar**

`bash actualizar.sh` reinicia web y worker con la clave nueva

5

**Revisar**

auditoría y registros: ¿alguien la usó mientras estuvo expuesta?

**EXPLICACIÓN TÉCNICA**

Antes de subir: `git status` para revisar que no vaya ningún `.env` ni modelo (el `.gitignore` ayuda, pero la revisión humana es la última barrera). Ojo: cambiar `DJANGO_SECRET_KEY` invalida todas las sesiones web y los tokens de la app (todos vuelven a iniciar sesión); cambiar la contraseña de Supabase exige actualizar `DATABASE_URL`.

Lo que falta reforzar en seguridad

Pendientes ordenados por prioridad.

Tarea	Dónde	Por qué	Prioridad
Restringir Supabase a la IP del VPS	Panel de Supabase → Network Restrictions	nadie más puede ni intentar conectarse	Alta
Firewall del VPS (ufw: 22, 80, 443)	VPS	cerrar puertos expuestos de otros servicios	Alta
Respaldo automático y prueba de restauración	VPS (cron + pg_dump)	el plan Free no da copias descargables	Alta
Verificar Data API apagada, SSL forzado y sign-ups de Auth apagados	Panel de Supabase	reducir superficie	Media
Supabase Pro para el piloto	Supabase	copias diarias y sin pausa por inactividad	Media
«Strict transformations» y sin presets sin firma	Panel de Cloudinary	nadie gasta créditos ni sube sin permiso	Media
API_SUBIDA_MULTIPART=false	.env.piloto	cerrar la vía de subida que ya no se usa	Media
HSTS de un año (SECURE_HSTS_SECONDS=31536000)	.env.piloto	tras confirmar que todo va por HTTPS	Baja
Mantenimiento semanal programado	cron del host	limpiar sesiones y tokens; evitar la pausa	Baja

🔧 **Para entenderlo:** el código ya trae sus protecciones (cuentas, permisos, tokens, auditoría, contenedores). Lo que falta es **configuración** de los servicios y del servidor: cerrar puertas que vienen abiertas por defecto y asegurar copias de los datos.

 **EXPLICACIÓN TÉCNICA**

Ninguna de estas tareas exige programar: son ajustes de configuración que hace la persona con acceso a cada panel o al VPS. Las de prioridad alta conviene hacerlas antes de la prueba de campo completa con datos reales.

13



Despliegue y operación

Dónde vive cada pieza, cómo llega un cambio al servidor y a los celulares, y qué hacer cuando algo falla.

Dónde vive cada cosa

El mapa de lugares: si algo hay que cambiar, aquí se ve dónde y cómo.

Pieza	Dónde vive	Cómo se actualiza
Código de la plataforma	PC del equipo · GitHub (prueba2) · VPS en /srv/riachuelo	git push → git pull en el VPS
Web, API y worker	VPS Hostinger KVM 1 (Ubuntu 24.04): riachuelo-web-1 y riachuelo-worker-1	bash deploy/actualizar.sh
Secretos y ajustes	deploy/.env.piloto en el VPS (permiso 600)	editar con nano + actualizar
Modelo de IA	deploy/modelos/modelo.onnx (solo lectura para los contenedores)	scp + /gestion/ + actualizar
Base de datos	Supabase riachuelo-piloto (São Paulo, plan Free)	migraciones al arrancar la web
Fotos	Cloudinary (prefijo piloto , tipo authenticated)	las sube la app; nadie las borra
Dominio y HTTPS	Hostinger (registro A monitoreo) · Let's Encrypt vía Traefik	el certificado se renueva solo
Código de la app	PC del equipo · GitHub (fase4-sincronizacion)	git push
APK de la app	EAS Build (nube de Expo) → enlace de descarga → 3 celulares	npm run build:apk:piloto
Entrenamiento de la IA	Google Colab (GPU T4) · datos en Roboflow y fotos propias	cuaderno + verificar_onnx.py
Mapas	Esri World Imagery, OpenStreetMap y Google Maps («Cómo llegar»)	servicios externos

🔧 EXPLICACIÓN TÉCNICA

El VPS (1 vCPU, 3,8 GB de RAM, 48 GB de disco) se comparte con otro servicio: por eso los contenedores tienen tope. En el disco del VPS solo hay código, secretos y el modelo; **los datos viven en Supabase y las fotos en Cloudinary**, así que reconstruir el servidor no pierde información.

🔗 HERRAMIENTA USADA Y POR QUÉ

🔗 **EAS Build** — **Para qué:** compilar el APK en la nube de Expo. **Por qué:** no hace falta Android Studio y el mismo perfil da la misma app a los 3 celulares.

La instalación del servidor (una sola vez)

Cinco pasos; el guion hace las comprobaciones difíciles.

1



Clonar

`git clone` del repositorio en `/srv/riachuelo` (rama `prueba2`)

2



Primera pasada

`bash deploy/installar.sh` : detecta Docker y Traefik, crea `.env.piloto` con clave nueva y se detiene

3



Secretos

`nano deploy/.env.piloto` : pegar `DATABASE_URL` (puerto 5432) y `CLOUDINARY_URL`

4



Segunda pasada

`bash deploy/installar.sh` : valida, revisa el DNS, construye, levanta y diagnostica

5



Primer administrador

`docker compose exec web python manage.py createsuperuser`

 **Se puede repetir**

Volver a ejecutar `instalar.sh` no rompe nada: reinstala con lo mismo.

 **No toca lo ajeno**

No modifica Traefik ni el otro servicio del VPS; todo queda en `/srv/riachuelo` .

 **Y para retirar**

`bash deploy/desinstalar.sh` borra contenedores, imágenes y la carpeta; después se quita el registro DNS.

 **Lo que instalar.sh revisa solo**

- ✓ Docker y el plugin `docker compose`
- ✓ El Traefik existente: red, entradas 80/443 y certificados
- ✓ Que estén las variables obligatorias (sin mostrarlas)
- ✓ Errores típicos: texto de ejemplo, puerto 6543, comillas
- ✓ Que el dominio apunte a la IP del VPS
- ✓ Que la web responda «healthy» (hasta 200 s)

 **EXPLICACIÓN TÉCNICA**

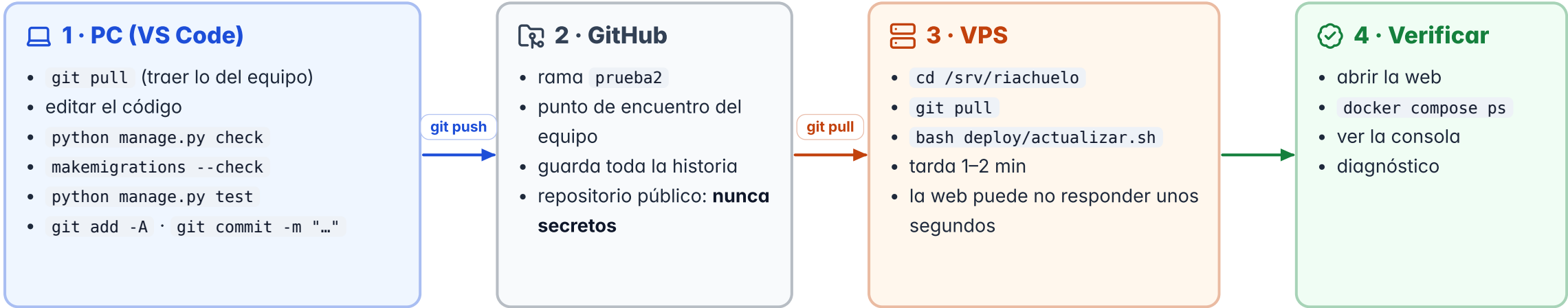
Traefik está en modo de red «host»: `instalar.sh` lo detecta, crea la red `riachuelo-proxy` y escribe `deploy/.env` (variables sin secretos: dominio, red, entradas y resoledor). La clave de Django se genera con `secrets.token_urlsafe(60)` . Ya se hizo una vez; queda como referencia.

 **HERRAMIENTA USADA Y POR QUÉ**

`>_ Bash` — **Para qué:** automatizar la instalación. **Por qué:** un guion repetible evita olvidos y deja los mismos pasos para cualquiera del equipo.

Cómo llega un cambio a la web real

Siempre en la misma dirección: PC → GitHub → VPS.



⚠ Importante: el código **nunca** se edita en el VPS: allí solo se tocan `.env.piloto` y `modelos/`. Si `git pull` se queja de cambios locales, revisar con `git status`. Si se cambió un `models.py`, subir también la migración nueva.

🔧 EXPLICACIÓN TÉCNICA

Las tres comprobaciones antes de subir evitan los errores más caros: `check` revisa la configuración, `makemigrations --check` avisa si falta una migración y `test` corre las 203 pruebas. Luego se agrega el reporte en `docs/REPORTE_AVANCE.md` (W-24). El mismo flujo sirve desde el panel de Git de VS Code (Commit → Sync Changes).

🔗 HERRAMIENTA USADA Y POR QUÉ

🔗 **Git + GitHub** — **Para qué:** llevar el mismo código a la PC, al equipo y al VPS. **Por qué:** cada cambio queda con autor, fecha y mensaje, y se puede volver atrás.

Qué hace actualizar.sh, paso a paso

Un comando que reconstruye, migra, reinicia y se revisa solo.

1



Construye

`docker compose build web` : imagen nueva (dependencias y estáticos)

2



Reinicia

`up -d --remove-orphans` : web y worker con la imagen nueva

3



Migra

al arrancar, la web ejecuta `migrate --noinput` y luego `unicorn`

4



Espera

hasta 200 s a que `/api/v1/health` diga «healthy»

5



Si falla

muestra las últimas 60 líneas de la web y termina con error

6



Si arranca

estado, limpia imágenes viejas y corre el diagnóstico

</> Así se ve en nuestro código

```
# deploy/actualizar.sh (resumido)
docker compose build web
docker compose up -d --remove-orphans
# ... espera el estado «healthy» (40 × 5 s)
docker compose logs --tail 60 web # solo si falló
docker compose ps
docker image prune -f
docker compose exec -T web \
  python manage.py diagnostico --sin-color
```



Siempre el mismo comando

- después de un cambio de código
- después de editar `.env.piloto`
- después de copiar un modelo nuevo a `modelos/`
- si el certificado no salió: `docker compose up -d --force-recreate web`



EXPLICACIÓN TÉCNICA

El worker espera a que la web esté «healthy» (`depends_on: service_healthy`), así nunca trabaja con tablas viejas. La imagen se llama `riachuelo-plataforma:latest` y es la misma para los dos procesos; Docker reutiliza capas, por eso solo reinstala dependencias si cambió `requirements.txt` .



HERRAMIENTA USADA Y POR QUÉ

 **Docker Compose** — **Para qué:** describir y levantar los dos contenedores juntos. **Por qué:** un solo archivo define imagen, límites, redes y salud; el mismo comando sirve siempre.

Qué le pasa a cada parte al actualizar

Para actualizar sin miedo.

Parte	¿Cambia?	Qué pasa
Web y API	Sí	imagen nueva y reinicio; unos segundos sin respuesta
Worker	Sí	se reinicia; una tarea a medias se libera al vencer su arriendo (5 min) y se retoma
Base de datos	Solo si hay migraciones	se aplican solas; los datos existentes no se pierden
Avisos pendientes	No	siguen en la cola y se envían al volver el worker
Fotos	No	siguen en Cloudinary, igual que antes
.env.piloto y modelos/	No	no están en Git: <code>git pull</code> no los pisa
Sesiones de la web	No	siguen abiertas: la sesión vive en la base, no en el contenedor
Celulares	No	si justo sincronizaban, reintentan solos sin duplicar (idempotencia)
Certificado HTTPS	No	lo mantiene Traefik

 **Dato clave:** actualizar no borra datos: todo lo importante vive fuera del VPS. Lo peor que pasa es un minuto sin web, y la app ni se entera porque guarda todo y reintenta.

 **EXPLICACIÓN TÉCNICA**

El arriendo (lease) de `ai_tasks` es de 5 min y el de `notifications` de 2 min: si un proceso muere a mitad de trabajo, la fila vuelve a estar disponible y otro la toma (`SELECT ... FOR UPDATE SKIP LOCKED`). La app usa IDs propios (UUID) en cada envío, así que repetir un pedido no crea filas nuevas.

Publicar una versión nueva de la app

De la PC a los tres celulares con el mismo APK.

1

Subir la versión

en `app.json` : `version`,
`versionCode` y
`buildNumber` (hoy 0.5.0 · 10)

2

Validar

`npm run validate` :
typecheck + lint + 139 pruebas

3

Entrar a Expo

`npx eas-cli@latest login` (una sola vez)

4

Compilar

`npm run build:apk:piloto` (perfil «piloto»)

5

Instalar

enlace o QR de EAS →
`.apk` en los 3 celulares (permitir apps desconocidas)

6

Comprobar

Ajustes → versión: los 3 iguales (RN-17)

</> Así se ve en nuestro código

```
// eas.json – perfil del piloto
"piloto": {
  "distribution": "internal",
  "env": {
    "EXPO_PUBLIC_API_URL":
      "https://monitoreo.agricolariachuelo.org",
    "EXPO_PUBLIC_USE MOCK_API": "0",
    "EXPO_PUBLIC_APP_ENV": "piloto"
  },
  "android": { "buildType": "apk" }
}
```

Perfil	Para qué
<code>piloto</code>	la app real, contra el servidor del piloto
<code>preview</code>	APK con servidor simulado (demostraciones)
<code>development</code>	cliente de desarrollo (programar con el celular)

Importante: una dependencia nativa nueva (en 0.5.0, `react-native-webview`) exige **APK nuevo**: no llega por Expo Go.

EXPLICACIÓN TÉCNICA

La dirección del servidor queda fija al compilar (`EXPO_PUBLIC_API_URL`): por eso un APK «preview» no sirve en el piloto. `appVersionSource: local` hace que la versión salga de `app.json` . Antes: `npm install` si cambió `package.json` .

HERRAMIENTA USADA Y POR QUÉ

Expo + EAS — **Para qué:** empaquetar y firmar la app Android. **Por qué:** mismo proceso para todos; el APK se instala sin Play Store (RNF-18).

Puesta en marcha de la web v1.3 y la app v0.5.0

Lo que falta hacer para que lo nuevo funcione en el piloto, en orden.



Los comandos del VPS

La «chuleta» de operación del día a día.

Para qué	Comando (desde /srv/riachuelo/deploy)
Publicar cambios	<code>cd /srv/riachuelo && git pull && bash deploy/actualizar.sh</code>
Ver el estado	<code>docker compose ps</code>
Ver la consola en vivo	<code>docker compose logs -f --tail 50 web worker</code> (Ctrl + C para salir)
Revisar todo	<code>docker compose exec web python manage.py diagnostico</code> (--red incluye la red)
Reiniciar solo el worker	<code>docker compose restart worker</code>
Crear un administrador	<code>docker compose exec web python manage.py createsuperuser</code>
Limpiar sesiones y tokens	<code>docker compose exec web python manage.py mantenimiento</code>
Respaldo (falta probar)	<code>docker compose exec web python manage.py respaldo</code>
Reintentar el certificado	<code>docker compose up -d --force-recreate web</code>
Ver memoria y CPU	<code>docker stats --no-stream</code>
Retirar todo al final	<code>bash deploy/desinstalar.sh</code>

 **EXPLICACIÓN TÉCNICA**

Cada línea de la consola lleva el `traceId` del pedido: con ese código se encuentra en los registros exactamente qué pasó con un pedido de la app o de la web. La consola de Django (`manage.py shell`) existe, pero se usa con cuidado: lo que se cambie ahí no pasa por los servicios ni queda en la auditoría.

 **HERRAMIENTA USADA Y POR QUÉ**

 **SSH / consola de Hostinger** — **Para qué:** entrar al VPS. **Por qué:** la contraseña de root la escribe solo quien tiene el acceso.

Trabajar en la PC: iniciar.bat

La plataforma completa en la laptop, para programar y probar.

1



Entorno

crea `.venv` e instala dependencias (Python 3.10 a 3.14)

2



Configuración

crea `.env` desde `.env.example` con clave nueva (primera vez)

3



Base

sin `DATABASE_URL` usa SQLite; aplica migraciones y carga la demo la primera vez

4



Revisa

`python manage.py diagnostico`

5



Arranca

worker en otra ventana y web + API en `http://127.0.0.1:8000`

Opción	Qué hace
<code>iniciar.bat</code>	doble clic: todo lo de arriba
<code>-Demo</code>	vuelve a cargar los datos de demostración
<code>-IA</code>	instala onnxruntime y numpy para probar el modelo
<code>-SinWorker</code>	solo la web y la API
<code>-Puerto 8080</code>	usa otro puerto

 **Celulares contra la PC**

En la misma Wi-Fi usan la IP que muestra la ventana; si Windows pregunta por el Firewall, permitir en «Redes privadas».

 **La demo, solo aquí**

Los datos de demostración nunca van al piloto (W-22).

 **EXPLICACIÓN TÉCNICA**

En la PC la web corre con `runserver` (servidor de desarrollo) y en el VPS con gunicorn. Con `IA_DETECTOR=simulado` el worker inventa indicios con probabilidad `IA_SIMULADO_PROBABILIDAD` (0,2) para probar todo el flujo sin modelo; con `WHATSAPP_CLIENT=ConsoleClient` los avisos se escriben en la consola en vez de enviarse.

 **HERRAMIENTA USADA Y POR QUÉ**

 `iniciar.bat` / `iniciar.ps1` — **Para qué:** levantar todo en Windows con un doble clic. **Por qué:** cualquier integrante prueba la plataforma sin saber Docker.

Los comandos de la app

Lo que se usa para programar, probar y compilar la app.

Para qué	Comando (en la carpeta de la app)
Instalar dependencias	<code>npm install</code> (después de cada cambio de <code>package.json</code>)
Revisar todo	<code>npm run validate</code> (typecheck + lint + jest)
Probar en Expo Go	<code>npm start</code> (sin servidor TCP: cámaras simuladas)
Probar con el APK de desarrollo	<code>npm run start:dev</code>
Compilar el APK del piloto	<code>npm run build:apk:piloto</code>
APK con servidor simulado	<code>npm run build:apk</code>
Instalar un paquete nuevo	<code>npx expo install <paquete></code> (versión compatible con SDK 57)
Contrato con el servidor	<code>npx tsx ... tools/verificacion/integracion.ts</code> (plataforma en la laptop)
Red local simulada	<code>npx tsx ... tools/verificacion/redlocal.ts</code>

 **Dato clave:** antes de entregar un cambio de la app: `npm run validate` ; si se tocó la sincronización o el contrato, también las pruebas de `tools/verificacion` (simulado 9/9, redlocal 9/9, integración 30/30).

 **EXPLICACIÓN TÉCNICA**

El servidor TCP del controlador (`react-native-tcp-socket`) no existe en Expo Go: por eso en Expo Go las cámaras se simulan y la prueba real necesita el APK. Las pruebas de contrato se ejecutan con Node 22.13 o superior. Contra el piloto real existe `piloto.ts` , que crea una sesión pequeña de prueba y exige `PERMITIR_PILOTO=1` .

Errores que ya ocurrieron y cómo se resolvieron

Lo aprendido al desplegar, para no repetirlo.

Síntoma	Causa	Solución
<code>collectstatic</code> fallaba al construir	<code>leaflet.js</code> apuntaba a un archivo de mapa de fuente inexistente	se quitó la línea <code>sourceMappingURL</code> del archivo copiado
<code>instalar.sh</code> no hallaba la red de Traefik	Traefik corre en modo de red «host»	el guion detecta ese modo y crea <code>riachuelo-proxy</code>
Cloudinary no funcionaba	<code>CLOUDINARY_URL</code> pegada con el texto de ejemplo del panel	pegar los valores reales; <code>instalar.sh</code> ahora lo detecta
Certificado HTTPS inválido	Let's Encrypt respondió NXDOMAIN: el DNS aún no existía	<code>docker compose up -d --force-recreate web</code> con el registro A activo
«La web no arrancó» sin explicación	el guion no mostraba la causa	<code>actualizar.sh</code> ahora imprime los últimos registros
El editor DNS de Hostinger fallaba («Order not found»)	problema del panel	el registro A lo creó el soporte
Base en el puerto 6543	es el «transaction pooler» (sin sentencias preparadas)	usar el Session pooler 5432

 **EXPLICACIÓN TÉCNICA**

Cada error se convirtió en una comprobación automática o en una línea del guion, para que no se repita. La lista de problemas conocidos de la app (GPS, permisos, red local) está en la sección 6.

Si algo falla: qué revisar primero

Una guía rápida de síntomas y primeras acciones.

Síntoma	Qué mirar primero	Cómo
La web no abre	¿web «healthy»?	<code>docker compose ps</code> · <code>docker compose logs --tail 60 web</code>
Aviso de certificado	¿el dominio apunta al VPS?	<code>docker compose up -d --force-recreate web</code>
Las fotos no se analizan	¿worker vivo? ¿modelo cargado?	<code>docker compose logs -f worker</code> · diagnóstico
Una tarea quedó en error	motivo en la tarea	reencolar desde la web (queda <code>TAREA_IA_REENCOLADA</code>)
No llegan los WhatsApp	¿destinatarios del lote? ¿cliente real?	estado del aviso en el caso · diagnóstico
La app no sincroniza	¿internet? ¿hora del celular? ¿cuenta activa?	diagnóstico de la app · buscar su traceld
La app dice 403	cuenta bloqueada o celular revocado	revisar en Usuarios de la web
La base se llena o se pausa	aviso al 80 % (<code>DB_LIMITE_MB</code>)	<code>mantenimiento</code> · plan Pro

🌟 **Para entenderlo:** casi todo se resuelve mirando tres cosas en orden: **¿está vivo?** (`ps`), **¿qué dice?** (`logs`) y **¿qué revisa el diagnóstico?** (`diagnostico`).

🔧 **EXPLICACIÓN TÉCNICA**

`diagnostico` revisa configuración, base, RLS, permisos, TLS, trigger de auditoría, CHECK, espacio usado, Cloudinary, IA y WhatsApp, y explica cada problema en lenguaje claro. `comprobar_bd` prueba solo la conexión a la base y traduce el error (contraseña, puerto, red).

🔗 **HERRAMIENTA USADA Y POR QUÉ**

🔗 **Comando diagnostico** — **Para qué:** revisar todo el sistema en un minuto. **Por qué:** el mismo chequeo en la PC y en el VPS; lo corren `instalar.sh` y `actualizar.sh` al final.

14



Pendientes y por pulir

Lo que falta para que el piloto funcione de punta a punta, ordenado por prioridad, y los detalles que conviene mejorar después.

Resumen del estado

Dónde estamos hoy, parte por parte.

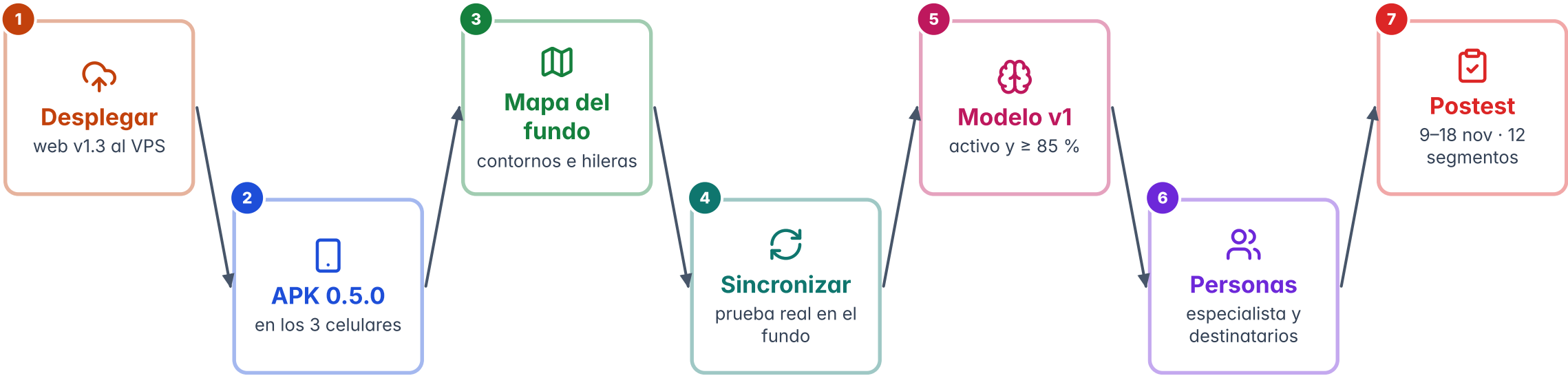
Parte	Estado	Detalle
Plataforma web y API en HTTPS	Listo	en monitoreo.agricolariachuelo.org; v1.3 escrita y probada (203 pruebas), falta subirla al VPS
Base de datos del piloto	Listo	39 tablas, RLS, 51 CHECK, auditoría de solo inserción
Fotos en Cloudinary	Listo	clave dedicada, fotos privadas, subida directa con ticket
Despliegue y actualizaciones	Listo	PC → GitHub → VPS con un comando
App: captura y red local	En campo	fases 0–3 hechas; probándose en el fundo con 3 Android
App: sincronización	Falta confirmar	hecha y verificada en la laptop (29/29); falta la prueba real en el fundo
«Ubicar plaga» (app 0.5.0)	Listo en código	139 pruebas e integración 30/30; falta compilar el APK
IA (YOLO)	Parcial	modelo de integración activo; falta el v1 entrenado con fotos del fundo
WhatsApp	Pendiente	los avisos se escriben en la consola; falta la cuenta de Meta y la plantilla
Datos reales del fundo	Parcial	83 hileras cargadas (SWG1 31, SWG2 16, SWG5 36) sin coordenadas
iPhone (fase 5)	Pendiente	el código es el mismo; falta compilar y probar

 **EXPLICACIÓN TÉCNICA**

El estado corresponde al 08–09/10/2026. «Listo en código» significa que está programado y con pruebas, pero aún no llega a los usuarios (falta desplegar o compilar). La lista completa y ordenada sigue en las próximas diapositivas.

La ruta crítica hasta el posttest

Siete pasos en orden; ninguno se puede saltar.



⚠ Importante: cada paso depende del anterior: sin el despliegue la app 0.5.0 no tiene datos; sin sincronización real no hay fotos del fondo para la IA; sin el especialista nadie confirma y no hay avisos. El postest no debe empezar con nada a medio terminar.

🔧 EXPLICACIÓN TÉCNICA
El postest compara los mismos 12 segmentos del pretest (diseño $O_1 \rightarrow X \rightarrow O_2$), con la app y el modelo congelados. Si un paso se atrasa, lo que se mueve es la activación del modelo, nunca la regla de que la persona decide: sin modelo v1 la IA queda con el de integración y el especialista revisa todo.

Pendiente 1: el modelo YOLO del piloto

El servidor ya está listo; falta el modelo entrenado con fotos del fundo.

1



Capturar

fotos propias con la app: SWG5 H31–H36 y SWG2 H01, H02, H07 (10–19 oct)

2



Etiquetar

cajas de chanchito blanco y melaza (13–25 oct)

3



Entrenar v1

Colab, YOLO26n (26–31 oct)

4



Verificar

`verificar_onnx.py` : detecta ≥ 85 % de las fotos con plaga

5



Subir y registrar

`scp` a `modelos/` + configuración activa en `/gestion/`

6



Aplicar

`bash actualizar.sh` (2 nov): las fotos en cola se analizan solas

 Medir con el modelo real

- tiempo por foto en el worker (1 vCPU)
- memoria: tope de 1,2 GB del contenedor
- precisión frente a las decisiones del especialista

 La autoconfirmación sigue apagada

- solo se enciende si el v1 demuestra pocos falsos positivos
- lo decide el equipo con el especialista
- se fija en `auto_confirm_threshold`

 Después: modelo v2

- reentrenar con las decisiones del especialista (14–17 nov)
- sin cambiar el modelo durante el postest

 **EXPLICACIÓN TÉCNICA**

Hoy corre el modelo de integración `2026.10-int` (P 0,69 · R 0,46 · mAP50 0,50), entrenado con fotos de internet (Roboflow). Sirve para probar el flujo completo, no para medir: el v1 del piloto llevará otra versión y aprenderá con fotos del fundo, tomadas con las mismas cámaras de la mochila.

 **HERRAMIENTA USADA Y POR QUÉ**

 **Google Colab + Ultralytics** — **Para qué:** entrenar con GPU gratis.
Por qué: la PC del equipo no tiene GPU; el resultado se exporta a ONNX para el servidor.

Agrícola Riachuelo · Plataforma de monitoreo fitosanitario

175 / 183

Pendiente 2: la app en el campo

Lo que solo se puede comprobar en el fundo, con los tres celulares.

**APK 0.5.0 en los 3 celulares**

Compilar con el perfil piloto e instalar; los tres con la misma versión (RN-17).

**Sincronización real**

Confirmar en el fundo que sesiones, pasadas y fotos llegan a la web.

**Prueba sin internet**

Jornada completa en modo avión y sincronizar al volver.

**Cerrar la subida antigua**

Con los 3 en ticket v2.0: `API_SUBIDA_MULTIPART=false` .

**Login sin internet lento**

Calibrar las vueltas de PBKDF2 (40 000) en el Galaxy A03 (Q-10).

**iPhone (fase 5)**

Compilar con Xcode y probar permisos, cámara y red local (Q-07).

**Autonomía**

Medir una jornada con power bank (RNF-12).

**Calibrar calidad**

Umbrales Q1 e intervalo del modo automático con fotos reales (Q-09).

⚠ **Pendiente:** la captura y la coordinación solo funcionan con la app en primer plano y la pantalla encendida (RNF-17); si en el campo se apaga la pantalla, el controlador deja de coordinar hasta volver.

**EXPLICACIÓN TÉCNICA**

La prueba de campo es la que valida RNF-03 (cero pérdida de evidencia ante cierres, cortes y reintentos) y RNF-04 (sin duplicados). Para revisar un problema, la app genera un archivo de diagnóstico sin secretos que se comparte desde Ajustes; con su `traceId` se encuentra el pedido en los registros del servidor.

Pendiente 3: operación y datos

Tareas de configuración y de cuidado del sistema.

Tarea	Dónde	Por qué	Prioridad
Crear las cuentas de especialistas y supervisores	Web → Usuarios	hoy hay 0 especialistas activos: nadie confirma	Alta
Destinatarios de WhatsApp con consentimiento y lotes	Web → Destinatarios	a quién avisar en cada lote	Alta
Respaldo con <code>pg_dump</code> y prueba de restauración	VPS	el plan Free no da copias descargables	Alta
Seguridad: IP de Supabase y firewall	Supabase · VPS	ver «Lo que falta reforzar» (sección 12)	Alta
Mantenimiento semanal programado	cron del VPS	limpia sesiones y tokens; evita la pausa del plan Free	Media
Supabase Pro para el piloto (y <code>DB_LIMITE_MB=8192</code>)	Supabase	copias diarias y sin pausa	Media
Revisar créditos de Cloudinary cada semana	Panel de Cloudinary	Free: 25 al mes (~1 000 fotos ≈ 10)	Media
Medir el rendimiento con datos reales	VPS	confirmar RNF04 (< 1,5 s) en el piloto	Baja
Política de retención de evidencia (Q-05)	Equipo	hasta decidir, no se borra nada (R-11)	Baja

 **EXPLICACIÓN TÉCNICA**

El diagnóstico del VPS del 08/10 dio 24 comprobaciones correctas y 1 error: «0 especialistas activos». Ninguna de estas tareas exige programar; el respaldo sí necesita un detalle: el contenedor es de solo lectura, así que `pg_dump` debe escribir en una carpeta montada o correr desde el host.

Pendiente 4: el mapa y las ubicaciones

Para que «Ubicar plaga» lleve a la planta, no solo a la hilera.

**Contornos de los lotes**

Dibujar SWG1, SWG2 y SWG5 en el mapa satelital de la web (Q-W05).

**Inicio y fin de las 83 hileras**

Coordenadas reales de los marcadores -INI y -FIN (Q-01): dan una referencia fija aunque el GPS de la foto sea malo.

**Importar por CSV**

Cargar catálogos y coordenadas desde un archivo en vez de uno por uno.

**Ubicación estimada**

Sin GPS, estimar la posición interpolando entre el inicio y el fin de la hilera.

**Mejor lectura de GPS**

Guardar la mejor lectura de los últimos segundos; hoy se avisa con más de 15 m.

**Calibrar «llegaste»**

`arrivedRadiusM` (15 m) con pruebas en el fondo.

**Número de planta**

Mostrar la planta aproximada dentro del segmento cuando se pueda calcular.

**Hileras y segmentos en el mapa**

Dibujarlos además de los contornos (opcional).

**EXPLICACIÓN TÉCNICA**

Sin coordenadas, un caso sin GPS solo se ubica por lote, hilera y segmento; con los marcadores cargados, la web y «Ubicar plaga» pueden mostrar un punto aunque el GPS haya fallado (`locationSource = MARCADOR`). Las coordenadas se guardan con 6 decimales (unos 10 cm) en `double precision` .

**HERRAMIENTA USADA Y POR QUÉ**

 **Esri World Imagery** — **Para qué:** ver el fondo desde el satélite para dibujar. **Por qué:** permite trazar lotes sin caminar el perímetro con un GPS.

Pendiente 5: los avisos por WhatsApp

Hoy el aviso funciona de punta a punta, pero en «modo consola».

1

Cuenta y número

WhatsApp Business de la empresa en Meta (Q-W08)

2

Plantillas

`caso_confirmado` y la de IA, categoría Utility, aprobadas por Meta

3

Credenciales

`WHATSAPP_TOKEN` , `WHATSAPP_PHONE_ID` y versión de Graph en `.env.piloto`

4

Cliente real

`WHATSAPP_CLIENT` = `CloudApiClient` y `actualizar.sh`

5

Destinatarios

con consentimiento escrito y sus lotes

6

Probar

confirmar un caso de prueba y ver «Enviado» en la web

Costo

Fuera de la ventana de 24 h solo se envían plantillas aprobadas, y se cobran. Hay que conocer el costo por mensaje antes del piloto.

Qué nunca lleva

Ni la foto ni su URL, ni tratamientos o urgencia: solo lote, hilera, lado, fecha y el enlace al caso con login.

Pregunta abierta

Si un caso avisado se corrige, ¿se envía un mensaje de corrección? Propuesto: no en esta versión (Q-W07).

EXPLICACIÓN TÉCNICA

Mientras tanto, `ConsoleClient` escribe en la consola del worker el mensaje que se habría enviado: todo el flujo (cola, reintentos, estados) se prueba sin gastar ni molestar a nadie. Un error de envío nunca cambia la decisión del caso (RN-W03).

HERRAMIENTA USADA Y POR QUÉ

WhatsApp Cloud API — **Para qué:** avisar al responsable del lote. **Por qué:** es la app que el personal del fundo ya usa todos los días.

Detalles por pulir

Mejoras que no bloquean el piloto, pero que conviene hacer.



Web

- estado «Atendido» para cerrar el ciclo en campo
- aviso de privacidad también en la web (cuentas creadas por el administrador)
- HSTS de un año cuando todo esté estable
- imprimir / PDF oficial de un caso (Q-W06)



App

- notificaciones en el celular cuando hay un caso nuevo
- «Ubicar plaga»: ver qué casos ya se atendieron
- mostrar el lado de la hilera y la planta aproximada
- mejorar la lectura de GPS antes de cada foto



Código y documentos

- subir a GitHub las dos ramas con v1.3 y v0.5.0
- AGENTS.md y el Maestro de la app: actualizar a CFG-8 y ADR 0009
- docs/REPORTE_AVANCE.md y la correspondencia de tablas del informe del curso
- confirmar que v1.2 ya estaba aplicada en el VPS

🌟 **Para entenderlo:** nada de esto bloquea el piloto: son mejoras que hacen el sistema más cómodo o más completo. Se hacen después de la ruta crítica, empezando por las que pidan los usuarios en la prueba de campo.

🔧 EXPLICACIÓN TÉCNICA

Varias mejoras de la app (casos atendidos, notificaciones) necesitan un cambio en el contrato /api/v1 ; la regla es agregar rutas o campos, nunca quitar ni renombrar, porque puede haber celulares con versiones anteriores. Las notificaciones del celular exigirían además un servicio de envío (por ejemplo, el de Expo) y su configuración.

Preguntas abiertas

Decisiones que no son de programación: las toma el equipo con la empresa.

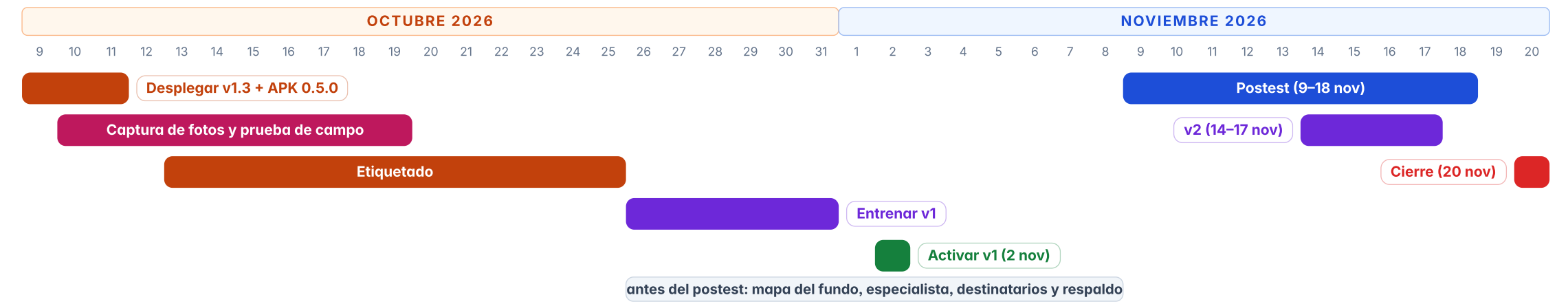
ID	Pregunta	Quién decide
Q-01	Códigos definitivos y coordenadas de lotes, hileras, segmentos y marcadores	Equipo + empresa
Q-03	Texto final del aviso de privacidad y datos que se piden al registrarse	Equipo + empresa
Q-05	Política de retención de las fotos y espacio mínimo	Equipo
Q-W02	Supervisores de zona: cuántos y qué lotes cubre cada uno (a quién avisar)	Equipo + empresa
Q-W03	Qué clases puede confirmar el especialista (¿solo chanchito blanco?)	IA + especialista
Q-W07	Si se corrige un caso ya avisado, ¿mensaje de corrección?	Equipo + empresa
Q-W08	Número de WhatsApp Business, plantillas aprobadas, costo y consentimiento	PM + empresa
Q-W10	Cómo revisar una muestra de fotos «sin indicios» (falsos negativos)	Equipo + especialista
—	Valor del umbral de autoconfirmación, si se llega a encender	Equipo + especialista

 **EXPLICACIÓN TÉCNICA**

Las preguntas viven en los Maestros (web §25.3 y app §25.3) con su fecha límite. Mientras no se respondan, el sistema usa la opción más prudente: no borrar evidencia, no enviar correcciones y no confirmar sin persona.

El calendario hasta el cierre

Del 9 de octubre al 20 de noviembre de 2026.



Hito	Fecha	Condición
Web v1.3 y APK 0.5.0 en uso	lo antes posible	diagnóstico sin errores y los 3 celulares en 0.5.0
Modelo v1 activo	2 nov	<code>verificar_onnx.py</code> ≥ 85 %
Postest	9–18 nov	app y modelo congelados; mismos 12 segmentos
Cierre del proyecto	20 nov	informe final con los resultados del postest

 **EXPLICACIÓN TÉCNICA**

Fechas de la IA y del postest según el plan del equipo al 08/10/2026. Las tareas sin fecha fija (mapa, cuentas, destinatarios, respaldo) tienen una sola condición: estar hechas antes del postest. Al terminar, `desinstalar.sh` retira la plataforma del VPS sin tocar lo demás.

Para llevarse

Si recuerdas solo seis ideas, que sean estas.

 **Dos repositorios, una API**

La app captura; la plataforma guarda, analiza y muestra. Solo hablan por `/api/v1` , que solo agrega rutas y campos.

 **La IA sugiere, la persona decide**

Ningún aviso sale sin confirmación: del especialista o, si el equipo enciende el umbral, de una IA con confianza muy alta y siempre auditada.

 **Un solo guardián de los datos**

Solo Django entra a la base; los secretos viven solo en el servidor; la app nunca tiene claves.

 **El campo nunca espera a internet**

Todo se guarda primero en el celular y se sincroniza después, sin perder ni duplicar fotos.

 **Del aviso a la planta**

«Ubicar plaga» lleva al encargado al lugar exacto con el mapa, la distancia y «Cómo llegar».

 **Cambiar es seguro**

PC → GitHub → VPS con un comando; las migraciones cuidan la base y la auditoría recuerda todo.

⚠ **Pendiente:** para el posttest faltan: desplegar v1.3 y el APK 0.5.0, la sincronización real en el fundo, el modelo v1, el mapa del fundo y las personas (especialista y destinatarios).

 **EXPLICACIÓN TÉCNICA**

Las seis ideas resumen las reglas que no se negocian: contrato que solo crece (sección 8), decisión humana y auditoría (secciones 4, 11 y 12), aislamiento de los datos (secciones 9 y 12), funcionamiento sin conexión (secciones 2 y 6), ubicación en campo (secciones 4 y 6) y despliegue repetible (sección 13).